# Wallet Guide

The Zcash wallet layer: keys, addresses, fees, and transaction construction

m@rek.onl

**Abstract**

This is the fifth volume of the series — *Math Guide*, *Crypto Guide*, *Halo 2 Guide*, *Orchard Guide*, *Wallet Guide*. Assuming the Orchard protocol of the *Orchard Guide*, it documents the deployed wallet layer above it: hierarchical key derivation (ZIP-32), diversified and unified addresses (ZIP-316), the conventional fee (ZIP-317), the transaction-construction pipeline of librustzcash, partially created transactions (PCZT), the transaction lifecycle from broadcast through expiry (ZIP-203), and payment requests (ZIP-321). Every derived quantity carries its exact derivation, and every claim about deployed behaviour cites the implementing crate and file.

# Contents

# 1 Hierarchical key derivation (ZIP-32)

The *Orchard Guide* constructs every Orchard key — ask, nk, $\mathsf{r_{ivk}}$, the full viewing key, ivk, dk, ovk, and the diversified addresses — from a single 32-byte spending key sk, which it treats as "simply a uniform 256-bit string", deferring its origin to ZIP-32 (§"Keys: capabilities for spending and for viewing" there, Definition "Orchard key hierarchy"). This section supplies that origin. A deployed wallet does not draw an independent sk per account: it draws one *seed* and derives every account's sk from it deterministically, so that a single backup recovers every key the wallet will ever hold. We develop ZIP-32's Orchard instantiation: the hardened-only derivation tree, the standard account path $m_{\mathsf{Orchard}}/32'/coin\_type'/account'$, the *internal* keys that stand in for a change path level, the fingerprints that name keys and seeds, and the serialised encodings — together with where the deployed stack (orchard, zip32, zcash_keys, zcash_client_backend, zcash_client_sqlite) enforces each rule, and where its documentation and its code disagree.

## 1.1 Wallet seeds

The root secret of the whole wallet is a byte string $S$, the *seed*. ZIP-32 imposes two requirements (ZIP-32, "Wallet seeds"): the seed MUST be 32 to 252 bytes long, and it MUST be generated with at least 256 bits of entropy — a requirement that extends to any mnemonic phrase from which the seed is obtained (ZIP-315 gives further guidance). The rationale is the seed's position in the hierarchy: every key of every account derives deterministically from $S$, so a lower-entropy seed is a weak link for all of them at once, and — unlike an attack on one key — a brute-force search over low-entropy seeds runs against many wallets simultaneously.

**Remark 1.1** (Where the deployed stack enforces the seed bounds)**.** Enforcement is patchier than the MUST suggests, and one doc comment is wrong. The derivation function itself does not check: neither `ExtendedSpendingKey::master` (orchard, `src/zip32.rs`) nor `HardenedOnlyKey::master` (zip32 0.2.0, `src/hardened_only.rs`) inspects the seed length — BLAKE2b hashes whatever it is given — although the former's doc comment claims "Panics if the seed is shorter than 32 bytes or longer than 252 bytes"; the claimed panic does not exist. One layer up, `UnifiedSpendingKey::from_seed` (zcash_keys, `src/keys.rs`) panics on seeds shorter than 32 bytes and checks no upper bound. The trait documentation of `WalletWrite::create_account` and `import_account_hd` (zcash_client_backend, `src/data_api.rs`) declares a panic for seeds outside 32..252 bytes, and zcash_client_sqlite enforces both bounds — indirectly, because it computes a seed fingerprint (Definition 1.13) via `SeedFingerprint::from_seed`, which returns `None` outside 32..252 bytes (zip32 0.2.0, `src/fingerprint.rs`; zcash_client_sqlite, `src/lib.rs`). The ZIP requirement therefore holds at the top of the deployed stack, not in the derivation function beneath it.

## 1.2 Hardened-only derivation

ZIP-32 obtains Orchard keys by instantiating its generic *hardened-only key derivation* process with two domain parameters: the master-key personalisation Orchard.MKGDomain = ``ZcashIP32Orchard'' (16 bytes) and the child-derivation lead byte Orchard.CKDDomain = 0x81 (ZIP-32, "Orchard key derivation"). Non-hardened derivation — the BIP-32 mode in which a public key derives child public keys — is not defined for Orchard at all. That choice fixes the shape of the extended keys: an *Orchard extended spending key* is the pair $(\mathsf{sk}, c)$ of a normal 32-byte spending key and a 32-byte *chain code* $c$, and ZIP-32 defines *no* Orchard extended full viewing key, because with only hardened derivation a full viewing key confers no child-derivation capability and so has no use for a chain code (ZIP-32, "Orchard extended keys"; contrast the Sapling sections of the same ZIP, which do define extended FVKs). Deployed, `ExtendedSpendingKey` carries the pair inside a `HardenedOnlyKey` together with bookkeeping fields `depth`, `parent_fvk_tag`, and `child_index` (orchard, `src/zip32.rs`).

**Definition 1.2** (Orchard master key generation)**.** Given a seed $S$ of 32 to 252 bytes, compute

$$I := \text{BLAKE2b-512}\big(\texttt{``ZcashIP32Orchard''},\ S\big), \qquad I = I_L \,\|\, I_R$$

with $I_L, I_R$ the first and last 32 bytes — unkeyed BLAKE2b-512 with the 16-byte personalisation above — and set $\mathsf{sk}_m := I_L$, $c_m := I_R$. The *master node* of the Orchard tree is $m_{\mathsf{Orchard}} := (\mathsf{sk}_m, c_m)$ (ZIP-32, "Hardened-only master key generation" instantiated per "Orchard master key generation"). Deployed master generation additionally requires $\mathsf{sk}_m$ to be valid in the sense of Definition 1.9, else it fails with `Error::InvalidSpendingKey` (orchard, `src/zip32.rs`; zip32 0.2.0, `src/hardened_only.rs`).

**Definition 1.3** (Orchard child key derivation)**.** Write $i' := i + 2^{31}$ for the *hardened* index derived from $0 \le i < 2^{31}$ (ZIP-32). Child derivation is defined only for hardened indices: for $i \ge 2^{31}$,

$$\begin{aligned}\mathsf{CKDsk}\big((\mathsf{sk}_{\text{par}}, c_{\text{par}}),\ i\big):\quad & I := \mathsf{PRFexpand}_{c_{\text{par}}}\big(\texttt{0x81} \,\|\, \mathsf{sk}_{\text{par}} \,\|\, \text{I2LEOSP}_{32}(i)\big),\\ & \mathsf{sk}_i := I_L, \qquad c_i := I_R,\end{aligned}$$

where $\text{I2LEOSP}_{32}(i)$ is the 4-byte little-endian encoding of $i$ and $\mathsf{PRFexpand}$ is the BLAKE2b-512 expansion function of the *Orchard Guide* (personalisation `` ``Zcash_ExpandSeed'' ``); derivation for $i < 2^{31}$ fails (ZIP-32, "Hardened-only child key derivation" and "Orchard child key derivation").

Note the division of labour in the PRF call: the parent *chain code* $c_{\text{par}}$ is the PRF key, and the parent spending key is message data. Continuing the tree therefore requires the pair — the 32-byte $\mathsf{sk}$ alone is a leaf capability, which is exactly the property the account path below exploits when it discards the chain code. Deployed derivation also caps the depth: the `depth` field is a byte incremented with `checked_add`, so a child of a depth-255 key cannot exist and the attempt fails with `Error::MaxDerivationDepth` (orchard, `src/zip32.rs`).

**Remark 1.4** (The general form: triples, and sibling instantiations)**.** ZIP-32's hardened-only child derivation in full generality takes a triple $(i, \mathsf{lead}, \mathsf{tag})$ and appends $\|\,[\mathsf{lead}]\,\|\,\mathsf{tag}$ to the PRF message; each distinct triple yields an independent output. For $\mathsf{lead} = 0$ and $\mathsf{tag} = []$ the encoding *omits* both fields, making it byte-compatible with the earlier definition of $\mathsf{CKDh}$ that lacked them; Orchard child derivation always uses $\mathsf{lead} = 0$, $\mathsf{tag} = []$, which is why Definition 1.3 shows no such suffix (ZIP-32, "Hardened-only child key derivation" and "Orchard child key derivation"). ZIP-32 defines no path *notation* for non-zero $\mathsf{lead}$ or non-empty $\mathsf{tag}$. The deployed encoder implements the omission literally: `PrfExpand::with` skips $[\mathsf{lead}]\,\|\,\mathsf{tag}$ when the lead is zero and the tag empty (zcash_spec 0.2.1, `src/prf_expand.rs`; zip32 0.2.0, `src/hardened_only.rs`). The same hardened-only process has two sibling instantiations outside the Orchard tree: *registered* key derivation (personalisation `` ``ZIPRegistered_KD'' ``, lead byte `0xAC`, subtree rooted at $ZipNumber'$ from an IKM of [len] ‖ *ContextString* ‖ [len] ‖ $S$) and the deprecated *ad-hoc* derivation (personalisation `` ``ZcashArbitraryKD'' ``, lead byte `0xAB`) (ZIP-32, "Specification: Registered key derivation" and "Specification: Ad-hoc key derivation (deprecated)"; zip32 0.2.0, `src/registered.rs`, `src/arbitrary.rs`). They matter here only as neighbours in the domain-separation registry of Table 1.

## 1.3 The standard account path

**Definition 1.5** (Orchard account path)**.** The account-level key for account number $account \in \{0, \ldots, 2^{31} - 1\}$ is derived along the path $m_{\mathsf{Orchard}}/32'/coin\_type'/account'$:

$$(\mathsf{sk}_{\text{acct}}, c_{\text{acct}}) := \mathsf{CKDsk}\Big(\mathsf{CKDsk}\big(\mathsf{CKDsk}\big(\mathsf{MKGh}^{\mathsf{Orchard}}(S),\ 32'\big),\ coin\_type'\big),\ account'\Big),$$

after which the wallet keeps $\mathsf{sk}_{\text{acct}}$ and discards $c_{\text{acct}}$. The *purpose* level is $32' =$ `0x80000020`, registered per BIP 43; the *coin type* follows the SLIP 44 registry — 133 on Mainnet, 1 on Testnet

and Regtest (all testnets share coin type 1) — and accounts are numbered sequentially from 0. Wallets MUST support this path for any account below $2^{31}$ (ZIP-32, "Key path levels" and "Orchard key path"; coin-type constants deployed in zcash_protocol, `src/constants/mainnet.rs`, `testnet.rs`, `regtest.rs`).

The path terminates at the account. Addresses are not further path children: each account exposes $2^{88}$ diversified addresses per scope, images of the diversifier indices under FF1 (ZIP-32, "Orchard key path"; the construction is in the *Orchard Guide*). Nor is there a change level between account and address, where BIP-44-style transparent paths put one: shielded change returning to the originating account is indistinguishable on-chain from any other output, so a change subtree would separate nothing an observer can see, and the separation the wallet itself needs — telling its own change apart from external receipts — is achieved one level lower by the *internal key derivation* of §1.5 (ZIP-32, "Key path levels").

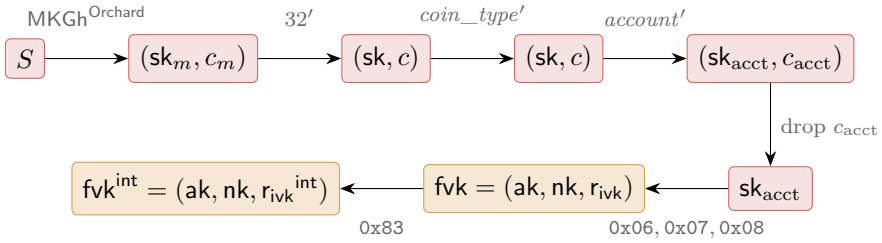


Figure 1: The deployed Orchard key path. Each horizontal arrow in the top row is one CKDsk call (Definition 1.3): a PRFexpand invocation with lead byte `0x81`, keyed by the parent chain code. The chain code travels only as far as the account node $(\mathsf{sk}_{\mathrm{acct}}, c_{\mathrm{acct}})$; the wallet layer keeps the bare $\mathsf{sk}_{\mathrm{acct}}$, from which the external full viewing key fvk (*Orchard Guide*, lead bytes `0x06`/`0x07`/`0x08`) and the internal $\mathsf{fvk}^{\mathsf{int}}$ (Definition 1.6, lead byte `0x83`) both derive. Red marks spend capability, orange full-viewing capability, per the series palette.

**Deployed path derivation.** Function `SpendingKey::from_zip32_seed(seed, coin_type, account)` (orchard, `src/keys.rs`) rejects *coin_type* $\geq 2^{31}$ with `Error::InvalidChildIndex`, runs `ExtendedSpendingKey::from_path` over the three hardened indices of Definition 1.5, and returns only the 32-byte account spending key — the chain code is discarded, so the wallet layer retains no Orchard child-derivation capability below the account level. Above it, `UnifiedSpendingKey::from_seed` stores that plain `SpendingKey`, and the unified full viewing key's Orchard component is the `FullViewingKey` computed from it (zcash_keys, `src/keys.rs`). The zip32 crate types the path levels: `AccountId` is a 31-bit integer whose `TryFrom<u32>` rejects values at or above $2^{31}$ and which always enters paths hardened, and `ChildIndex` is hardened-only — `from_index` returns `None` unless the hardened bit is set, and `hardened(v)` asserts $v < 2^{31}$ and stores $v + 2^{31}$ (zip32 0.2.0, `src/lib.rs`). A distinguished index `ChildIndex::PRIVATE_USE` = `0x7fffffff`′ is reserved for private-use subtrees; zcashd's legacy Sapling account uses it, and we find no Orchard use (ZIP-32, "Sapling key path"; zip32 0.2.0, `src/lib.rs`).

## 1.4 From the account key to addresses: the layer already built

Everything below $\mathsf{sk}_{\mathrm{acct}}$ is the hierarchy the *Orchard Guide* constructs in "Keys: capabilities for spending and for viewing" (Definition "Orchard key hierarchy"), and we do not re-derive it: ask, nk, $\mathsf{r}_{\mathsf{ivk}}$ from PRFexpand lead bytes `0x06`, `0x07`, `0x08` with the ToScalar/ToBase reductions and their bias analysis; the sign-canonicalised $\mathsf{ak} = [\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$; $\mathsf{fvk} = (\mathsf{ak}, \mathsf{nk}, \mathsf{r}_{\mathsf{ivk}})$; $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r}_{\mathsf{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$; the $(\mathsf{dk}, \mathsf{ovk})$ split of a single PRFexpand output with lead byte `0x82`;

and the FF1 diversified addresses with default index 0 (protocol specification §4.2.3, "Orchard Key Components"; orchard, `src/keys.rs`).

What ZIP-32 adds at the wallet level is the capability boundary of the full viewing key. The holder of fvk obtains, for *both* the external and the internal scope of §1.5: the incoming viewing keys, the diversifier keys and outgoing viewing keys, and hence every diversified address of the account; the *index* behind any of its addresses, since the diversifier map is a keyed permutation and $\mathsf{FF1\text{-}AES256}_{\mathsf{dk}}^{-1}(d)$ — FF1 decryption under the empty tweak — recovers the index (`DiversifierKey::diversifier_index`, orchard, `src/keys.rs`); and the scope of any of its addresses, by trying external then internal (`FullViewingKey::scope_for_address`, orchard, `src/keys.rs`). Two contrasts with Sapling sharpen the picture (ZIP-32, "Orchard diversifier and OVK derivation"): the diversifier key derives from the *full viewing key* rather than the extended spending key, so viewing capability suffices to locate an address's position in the sequence; and all $2^{88}$ diversifiers are valid, so no rejection loop exists and the default diversifier is $d_0$. What the full viewing key can *not* do: derive ask, or derive any child account — the former by the one-wayness of the `0x06` expansion, the latter because only hardened derivation exists and no extended FVK is defined (§1.2). Finally, no mechanism derives an outgoing viewing key per *diversified address*: payments are sent from accounts, with external or internal scope, not from individual addresses (ZIP-32, "Orchard diversifier and OVK derivation", citing the ZIP-316 rationale "Usage of Outgoing Viewing Keys").

## 1.5 Internal keys: change and auto-shielding

The path of Definition 1.5 reserved no change level, so the separation between externally visible receiving and wallet-internal operations — change outputs, and the auto-shielding of transparent funds — happens below the account instead: every account key yields a second, *internal* full viewing key. The mechanism was implemented in zcashd as part of ZIP-316 support and applies to keys that are children of the account level, i.e. account keys (ZIP-32, "Orchard internal key derivation").

**Definition 1.6** (Orchard internal key derivation)**.** Given an external full viewing key $\mathsf{fvk} = (\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}})$, let $K := \mathrm{LE}_{256}(\mathsf{r_{ivk}})$ and

$$\mathsf{r_{ivk}}^{\mathsf{int}} := \mathrm{ToScalar}\big(\mathsf{PRFexpand}_K\big(\mathtt{0x83} \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{ak}) \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{nk})\big)\big) \in \mathbb{F}_{p_{\mathsf{Vesta}}},$$
$$\mathsf{fvk}^{\mathsf{int}} := (\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}}^{\mathsf{int}}),$$

i.e. $\mathsf{DeriveInternalFVK}^{\mathsf{Orchard}}$ modifies *only* $\mathsf{r_{ivk}}$ (ZIP-32, "Orchard internal key derivation"; protocol specification §4.2.3). The downstream keys then follow the standard FVK-level rules of the *Orchard Guide*, applied to $\mathsf{fvk}^{\mathsf{int}}$: $\mathsf{ivk}^{\mathsf{int}} := \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}^{\mathsf{int}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$ and $(\mathsf{dk}^{\mathsf{int}}, \mathsf{ovk}^{\mathsf{int}})$ from the `0x82` split keyed by $\mathrm{LE}_{256}(\mathsf{r_{ivk}}^{\mathsf{int}})$. The *expanded internal spending key* is $(\mathsf{ask}, \mathsf{nk}, \mathsf{r_{ivk}}^{\mathsf{int}})$ — the same ask — and no 32-byte internal spending key exists (ZIP-32, "Orchard internal key derivation").

The shape is deliberately that of the $(\mathsf{dk}, \mathsf{ovk})$ derivation: the same key $K$, the same message body $\mathrm{I2LEOSP}_{256}(\mathsf{ak}) \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{nk})$, a different lead byte. Its consequences: ak and nk are unchanged (the protocol specification notes $\mathsf{ak}^{\mathsf{int}} = \mathsf{ak}$ and $\mathsf{nk}^{\mathsf{int}} = \mathsf{nk}$, §4.2.3), so spend authorisation and nullifier derivation are common to both scopes, while ivk, dk, and ovk all separate because they derive from $\mathsf{r_{ivk}}^{\mathsf{int}}$. The two scopes are therefore two disjoint families of diversified addresses under one spend authority — and since $\mathsf{ivk}^{\mathsf{int}} \neq \mathsf{ivk}$, a note addressed under the internal scope does not trial-decrypt under the external ivk (*Orchard Guide*, "Transmission and detection of a note"), which is precisely the separation a wallet wants when it hands its external incoming viewing key to a payment processor. Note that the internal FVK is *computable from* the external FVK — the derivation consumes only $(\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}})$ — so disclosing the full viewing key disclosed both scopes all along; the boundary runs at the incoming-viewing tier, not the

full-viewing tier. Unlike its Sapling counterpart, the construction does not rest on non-hardened derivation, which is undefined for Orchard (ZIP-32, "Orchard internal key derivation").

Deployed, the zip32 crate's `Scope` enum {`External`, `Internal`} names the two scopes and orchard re-exports it; `FullViewingKey::rivk(Scope)`, `to_ivk`, `to_ovk`, and `derive_internal` implement Definition 1.6 (zip32 0.2.0, `src/lib.rs`; orchard, `src/keys.rs`). The change path is concrete: zcash_client_backend sends Orchard change to `ufvk.orchard().address_at(0u32, Scope::Internal)` — the internal-scope address at diversifier index 0 (zcash_client_backend, `src/data_api/wallet.rs`).

The transparent pool needs the same external/internal ovk split — its auto-shielding transactions carry shielded outputs whose outgoing ciphertexts want a viewing key — but P2PKH keys have no protocol-level ovk to scope, so ZIP-316 synthesises the pair.

**Definition 1.7** (Transparent internal and external OVK)**.** For a transparent P2PKH account with BIP-32 account-level extended public key $(c, pk)$ — chain code $c$, keying PRFexpand as a 256-bit key, and $\mathsf{ser}(pk)$ the 33-byte compressed public key —

$$I_{\mathsf{ovk}} := \mathsf{PRFexpand}_c\big(\texttt{0xd0} \,\|\, \mathsf{ser}(pk)\big), \qquad \mathsf{ovk}_{\text{external}} := I_{\mathsf{ovk}}[0..32), \quad \mathsf{ovk}_{\text{internal}} := I_{\mathsf{ovk}}[32..64)$$

(ZIP-316, *Deriving Internal Keys*; lead byte in Table 1).

**Remark 1.8** (Byte-level normalisation between ZIP and code)**.** ZIP-32 writes the PRF key as the bit sequence $\text{I2LEBSP}_{256}(\mathsf{r_{ivk}})$ where PRFexpand consumes bytes; the code passes `rivk.to_repr()`, the 32-byte little-endian encoding. The two are identical under the $\text{LE}_{256}$ convention this series uses, and we write $\text{LE}_{256}$ throughout, as the *Orchard Guide* already does. Likewise the ak bytes fed to the `0x82` and `0x83` messages are `SpendValidatingKey::to_bytes()`, the 32-byte RedPallas point encoding — which, because key generation forces the sign bit to zero (the canonicalisation of the *Orchard Guide*), coincides with $\text{I2LEOSP}_{256}(\mathsf{Extract}(\mathsf{ak}))$, matching the specification's $\text{I2LEOSP}_{256}(\mathsf{ak})$ for $\mathsf{ak} \in \{0, \ldots, p_{\mathsf{Pallas}} - 1\}$ (orchard, `src/keys.rs`; protocol specification §4.2.3 and the AuthSignPublic type). The nk bytes are the base-field representation.

## 1.6 Key validity and derivation failure

**Definition 1.9** (Valid Orchard spending key)**.** A 32-byte string sk is a *valid* spending key iff all three hold: (i) $\mathsf{ask} \neq 0$; (ii) the external $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}) \notin \{0, \bot\}$; (iii) the internal $\mathsf{ivk}^{\mathsf{int}} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}^{\mathsf{int}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}) \notin \{0, \bot\}$. Function `SpendingKey::from_bytes` returns `None` exactly when one of the three fails (orchard, `src/keys.rs`), and the specification's key-generation algorithm mandates exactly these three discard conditions — "discard this key and repeat with a new sk" (protocol specification §4.2.3, "Orchard Key Components"). `FullViewingKey::from_bytes` re-checks both scopes when parsing an FVK from bytes (orchard, `src/keys.rs`).

**Remark 1.10** (Derivation failure has no specified recovery)**.** ZIP-32 notes that a child spending key may yield an invalid external or internal full viewing key "with small probability" (ZIP-32, "Orchard child key derivation") but prescribes no wallet behaviour when it does; the specification's "repeat with a new sk" addresses a freshly random key, not a fixed HD path, where there is nothing to redraw. The deployed stack hard-errors: `master` and `derive_child` return `Err(Error::InvalidSpendingKey)`, `from_path` propagates it, and zcash_keys surfaces it as `DerivationError::Orchard` (orchard, `src/zip32.rs`; zcash_keys, `src/keys.rs`); no layer implements a skip-to-next-index convention. This behaviour is normative in practice: an account index at which derivation fails simply has no key, and no deployed convention reassigns it. The event is negligibly rare — neither ZIP-32 nor the specification quantifies "small probability", so Proposition 1.11 derives a bound — and the gap is theoretical.

**Proposition 1.11** (Probability of an invalid key)**.** *Model the* PRFexpand *outputs and the* GroupHash*-derived Sinsemilla generators as uniform and independent (the random-oracle model*

*in which the* Orchard Guide *analyses both primitives). A uniformly random* 32*-byte* $\mathsf{sk}$ *then fails the conditions of Definition 1.9 with probabilities*

$$\Pr[\mathsf{ask} = 0] \;<\; 2^{-254}, \qquad \Pr\big[\mathsf{ivk} \in \{0, \bot\}\big] \;\le\; 307/p_{\mathsf{Vesta}} \;<\; 2^{-245} \quad \textit{(per scope)},$$

*and is invalid with probability below* $2^{-244}$.

*Proof.* The scalar $\mathsf{ask}$ is the ToScalar image of the `0x06` expansion (the conditional negation that canonicalises $\mathsf{ak}$ preserves zeroness), so $\mathsf{ask} = 0$ iff a uniform 512-bit integer reduces to 0 modulo $p_{\mathsf{Vesta}}$. Exactly $\lceil 2^{512}/p_{\mathsf{Vesta}} \rceil$ of the $2^{512}$ inputs do, and $p_{\mathsf{Vesta}} > 2^{254}$ gives $\lceil 2^{512}/p_{\mathsf{Vesta}} \rceil < 2^{258}$, hence $\Pr[\mathsf{ask} = 0] < 2^{258} \cdot 2^{-512} = 2^{-254}$.

For $\mathsf{ivk}$, the two memberships separately. *Zero*: conditioned on the Sinsemilla hash not returning $\bot$, the commitment point is the hash point plus $[\mathsf{r_{ivk}}]\, H_D$ — the blinding term of the *Orchard Guide*'s "Sinsemilla commitment and short commitment" — uniform on the $p_{\mathsf{Vesta}}$-element Pallas group by perfect hiding, $\mathsf{r_{ivk}}$ being uniform up to the $O(2^{-258})$ ToScalar bias. The extractor sends only the identity to 0, because $y^2 = 0^3 + 5$ has no solution in $\mathbb{F}_{p_{\mathsf{Pallas}}}$ — 5 is a non-square there (protocol specification §5.4.9.7, note) — so $\Pr[\mathsf{ivk} = 0] \le 1/p_{\mathsf{Vesta}} < 2^{-254}$. *Bottom*: the $\bot$ output arises only inside the Sinsemilla hash, since the blinding addition is complete (§5.4.8.4). On the 510-bit $\mathsf{Commit}^{\mathsf{ivk}}$ message — two 255-bit field encodings, $\lceil 510/10 \rceil = 51$ ten-bit chunks — the hash performs $2 \cdot 51 = 102$ incomplete additions (§5.4.1.9). Each fails only when one operand is the identity or shares the other's $x$-coordinate — at most 3 of the $p_{\mathsf{Vesta}}$ group elements — so, with each accumulator modelled as uniform, the union bound gives $\Pr[\bot] \le 306/p_{\mathsf{Vesta}} < 2^9 \cdot 2^{-254} = 2^{-245}$, and per scope $\Pr[\mathsf{ivk} \in \{0, \bot\}] \le 307/p_{\mathsf{Vesta}}$. The internal scope repeats the argument with the independent $\mathsf{r_{ivk}}^{\mathsf{int}}$ (the `0x83` expansion). In total, $\Pr[\mathsf{sk}\text{ invalid}] < (1 + 2 \cdot 307) \cdot 2^{-254} < 2^{10} \cdot 2^{-254} = 2^{-244}$. □

The bound makes ZIP-32's "small probability" concrete: below $2^{-244}$ per derived key, and below $2^{-242}$ for the union over the four nodes of the standard account path (master included, Definition 1.2). Only the $\bot$ term is heuristic, and it is doubly guarded: any concrete input exhibiting $\bot$ would constitute a nontrivial discrete-log relation among the Sinsemilla generators (*Orchard Guide*, the Sinsemilla collision-resistance proof), so none is expected ever to be observed.

## 1.7 Fingerprints and tags

Two identifiers name objects of this section without revealing them; both are BLAKE2b-256 digests with dedicated personalisations.

**Definition 1.12** (Orchard FVK fingerprint and tag)**.** For a full viewing key $\mathsf{fvk} = (\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}})$, the *fingerprint* is

$$\begin{aligned} \mathsf{FVFP}(\mathsf{fvk}) &:= \text{BLAKE2b-256}\big(\texttt{``ZcashOrchardFVFP''},\ \mathsf{fvk}_{\text{raw}}\big), \\ \mathsf{fvk}_{\text{raw}} &:= \text{I2LEOSP}_{256}(\mathsf{ak}) \,\|\, \text{I2LEOSP}_{256}(\mathsf{nk}) \,\|\, \text{I2LEOSP}_{256}(\mathsf{r_{ivk}}), \end{aligned}$$

the digest of the 96-byte raw FVK encoding $\mathsf{fvk}_{\text{raw}}$ (protocol specification §5.6.4.4). The *tag* is its first 4 bytes and MUST NOT be assumed unique; the master key's parent tag is the 4 zero bytes (ZIP-32, "Orchard Full Viewing Key Fingerprints and Tags"). Deployed as `FvkFingerprint` and `FvkTag`; child derivation stamps each child with `parent_fvk_tag` = the tag of the parent's FVK (orchard, `src/zip32.rs`).

**Definition 1.13** (Seed fingerprint)**.** For a seed $S$ of 32 to 252 bytes, with $[\text{length}(S)]$ its single length byte,

$$\mathsf{SeedFP}(S) := \text{BLAKE2b-256}\big(\texttt{``Zcash\_HD\_Seed\_FP''},\ [\text{length}(S)] \,\|\, S\big);$$

the string form is Bech32m with human-readable part `zip32seedfp`, and no short tag is defined; the length prefix was chosen to match zcashd's implementation (ZIP-32, "Seed Fingerprints"). Deployed, `SeedFingerprint::from_seed` returns `None` outside the 32..252-byte bounds (zip32 0.2.0, `src/fingerprint.rs`), and zcash_client_sqlite uses the fingerprint to bind each stored account to the seed that produced it (zcash_client_sqlite, `src/lib.rs`).

### 1.8 Extended-key encoding and test vectors

**Definition 1.14** (Orchard extended spending key encoding)**.** An extended spending key at depth depth, child index $i$ (hardened offset included), with parent tag $\mathsf{tag}_{\mathrm{par}}$, serialises as the 73-byte string

$$\mathsf{xsk} := \mathrm{I2LEOSP}_8(\mathsf{depth}) \,\|\, \mathsf{tag}_{\mathrm{par}} \,\|\, \mathrm{I2LEOSP}_{32}(i) \,\|\, c \,\|\, \mathsf{sk} \qquad (1+4+4+32+32 \text{ bytes}),$$

with the master key at depth 0, tag `0x00000000`, index 0. The Bech32 human-readable parts are `secret-orchard-extsk-main` (Mainnet) and `secret-orchard-extsk-test` (Testnet) (ZIP-32, "Orchard extended spending keys").

ZIP-32 defines this encoding "for completeness" and expects most wallets to expose only the plain sk encoding of the protocol specification (§5.6.4.5) — consistent with the deployed stack, which discards the chain code at the account level (§1.3). Indeed we find *no* deployed librustzcash code path that emits or parses the Bech32 form: the 73-byte layout is exercised only by the orchard crate's embedded test-vector checks, and zcash_keys stores the raw spending key inside the unified spending key encoding (orchard, `src/zip32.rs`, `src/test_vectors/zip32.rs`; zcash_keys, `src/keys.rs`). We classify the encoding as specified but unused in the scanned stack.

**Remark 1.15** (Test vectors)**.** The ZIP-32 Orchard vectors live in zcash-test-vectors under `orchard_zip32`; the orchard crate embeds four of them — the nodes $m$, $m/1'$, $m/1'/2'$, $m/1'/2'/3'$ from the 32-byte seed `0x00`, …, `0x1f` — each giving sk, $c$, the 73-byte xsk, and the FVK fingerprint (ZIP-32, "Test Vectors"; orchard, `src/test_vectors/zip32.rs`, `src/zip32.rs`). They do *not* cover the standard path of Definition 1.5. Separate key-component vectors (sk → ask, ak, nk, $\mathsf{r_{ivk}}$, ivk and the internal-scope components) sit in `src/test_vectors/keys.rs`, exercised by the `keys.rs` tests.

**Example 1.16** (The standard path, worked)**.** No published vector covers the path of Definition 1.5, so we compute one, from the embedded vectors' seed $S =$ `0x00`, …, `0x1f`, for Mainnet ($coin\_type = 133$) and account 0:

| | | |
|---|---|---|
| $m$ | sk | `7eee3c1017870990a3dd6891b82f80be8976c1e7dc20d60817a5e88e8b2cd4b8` |
| | $c$ | `ab8b7a00509ef20e469b5292b61d474b7cffcb1657924cda720250ae40526677` |
| $m/32'$ | sk | `93af0a87c2e4704d8825e145557c6a5d4dcbc9016170e276a2080a2dc9eb0f87` |
| | $c$ | `6e1e808d410d6d1f84e62e67c8b60e80e5ee4de5021d575c0433a625d609c45c` |
| $m/32'/133'$ | sk | `149fcee594a2a03de277ab919f22611d0ef6aa8e8e9a0bef32f01a0bcb0e8c4c` |
| | $c$ | `07d13fdeccd21da16b467f8e785edc32f74ceeb541bede0394647f10fdb7114f` |
| $m/32'/133'/0'$ | sk | `b67d8d87cab9189500afb45dbca9f92c924c1de9d9ee1451be4a78313cb223b4` |
| | $c$ | `7ff0a6392e144d4c8f45bca21fb7827b508446ac069890aefbebbe388b7a84c0` |

The bottom sk is exactly what `SpendingKey::from_zip32_seed` returns for $(S, 133, 0)$; the bottom $c$ is what it discards. The account key's FVK fingerprint (Definition 1.12) is

`da8dab741dc4f6ff8964fbaa8840270caf47d286e9a0722e6c89e7f2c38189d5`

(tag `da8dab74`), its parent's fingerprint begins `7010c19b`, and the account's 73-byte encoding of Definition 1.14 is therefore

$$\mathsf{xsk} = \mathsf{03} \,\|\, \mathtt{7010c19b} \,\|\, \mathtt{00000080} \,\|\, c \,\|\, \mathsf{sk}, \qquad \mathtt{00000080} = \mathrm{I2LEOSP}_{32}(0'),$$

with $c$ and sk the bottom table row. The numbers are script-computed per the series conventions: a standalone reimplementation of $\mathsf{MKGh}^{\mathsf{Orchard}}$ and CKDsk first reproduces all four embedded vectors byte for byte (sk, $c$, xsk); the orchard crate, driven as a library, reproduces their FVK fingerprints, returns the same account sk from `from_zip32_seed`, and validates every node above per Definition 1.9.

### 1.9 The PRFexpand lead-byte registry

One PRF underlies every derivation above and in the *Orchard Guide*, but domain separation is per PRF *key*, not protocol-wide: under a fixed key, distinct lead bytes make the derived secrets mutually independent, while separate consumers may reuse a byte under different keys. Sapling derives rcm and esk from lead bytes `0x04` and `0x05` with no $\rho$ suffix — the assignment swapped relative to Orchard's, which the specification attributes to an oversight (protocol specification §4.7.3, note) — and the Sapling instantiation of ZIP-32 intentionally reuses `0x00`–`0x02` from the Sapling key components under a different key. Table 1 collects the accounting for the Orchard, Orchard-ZIP-32, and ZIP-316 bytes this series constructs; the Sapling-only bytes (`0x00`–`0x03`, `0x10`–`0x18`, of which §2.2 cites `0x10` and `0x16`) are accounted in the specification's list (protocol specification, the $\mathsf{PRF}^{\mathsf{expand}}$ accounting under "Pseudo Random Functions" and §4.2.3; ZIP-32; zcash_spec 0.2.1, `src/prf_expand.rs`). The Sprout instantiation of ZIP-32 reserves, permanently, the lead byte `0x80`, the personalisation ``` ``ZcashIP32_Sprout'' ```, and the HRPs `zxsprout`/`zxtestsprout` (ZIP-32, "Values reserved due to previous specification for Sprout"). All Orchard and ZIP-32 constants in this section match between the specification text and zcash_spec 0.2.1 — the version locked by both the orchard and librustzcash workspaces (`Cargo.lock` in each; likewise zip32 0.2.0).

| Lead byte | PRF key | Message after the lead byte | Output |
|---|---|---|---|
| `0x04` | rseed | $\rho$ | esk |
| `0x05` | rseed | $\rho$ | rcm |
| `0x06` | sk | — | ask |
| `0x07` | sk | — | nk |
| `0x08` | sk | — | $\mathsf{r_{ivk}}$ |
| `0x09` | rseed | $\rho$ | $\psi$ |
| `0x80` | Sprout ZIP-32 child (reserved; never reuse) | | |
| `0x81` | $c_{\mathrm{par}}$ | $\mathsf{sk_{par}} \,\|\|\, \mathrm{I2LEOSP}_{32}(i)$ | $\mathsf{sk}_i \,\|\|\, c_i$ |
| `0x82` | $\mathrm{LE}_{256}(\mathsf{r_{ivk}})$ (per scope) | ak $\|\|$ nk | dk $\|\|$ ovk |
| `0x83` | $\mathrm{LE}_{256}(\mathsf{r_{ivk}})$ | ak $\|\|$ nk | $\mathsf{r_{ivk}}^{\mathsf{int}}$ |
| `0xAB` | ad-hoc ZIP-32 child (deprecated) | | |
| `0xAC` | registered ZIP-32 child | | |
| `0xD0` | $c$ (chain code) | $\mathsf{ser}(pk)$ | $\mathsf{ovk_{ext}} \,\|\|\, \mathsf{ovk_{int}}$ |

Table 1: Lead-byte registry of PRFexpand messages. Point and field arguments are encoded with $\mathrm{I2LEOSP}_{256}$; $\rho$ is the 32-byte nullifier of the Action's spent note (*Orchard Guide*, "Transmission and detection of a note"). Rows `0x04`–`0x09` are constructed in the *Orchard Guide* and carry the Orchard meanings of `0x04`/`0x05`; Sapling reuses both bytes, without the $\rho$ suffix and with the outputs swapped (`0x04` → rcm, `0x05` → esk), under its own per-note rseed keys (protocol specification §4.7.3, note). Rows `0x81` and `0x83` are constructed in Definitions 1.3 and 1.6; `0xD0` in Definition 1.7; `0x80`, `0xAB`, `0xAC` belong to other subsystems (ZIP-32) and appear for the completeness of the accounting.

## 2 Diversified and unified addresses (ZIP-316)

An address layer must reconcile two demands that pull apart. Each payee should receive their own address, mutually unlinkable with every other — yet all of them spendable by one key and

recoverable from one seed; each shielded protocol solves this with *diversified addresses*. And Zcash is several protocols at once: transparent, Sapling, and Orchard receivers coexist, each with its own format, so a recipient wants to hand a sender *one* string from which the sender's wallet picks the best receiver both sides support; ZIP-316 solves this above the protocols with the *unified* container encodings. The *Orchard Guide* ("Keys: capabilities for spending and for viewing") already constructs the Orchard-side machinery — the key tiers, the FF1-AES256 diversifier map $d := \mathsf{FF1\text{-}AES256}_{\mathsf{dk}}(d_{\text{plain}})$, the diversified base $\mathtt{g_d} = \mathsf{GroupHash}^{\mathsf{Pallas}}(\mathtt{z.cash} : \mathtt{Orchard\text{-}gd}, d)$, and the rationale for a keyed permutation over the index space — and we cite it throughout. This section adds what the wallet layer builds on top: the index discipline shared across Sapling, Orchard, and transparent derivation, the raw and unified encodings, viewing-key containers, and the deployed address-generation code.

As of this writing ZIP-316 is organised into revisions: Revision 0 is Active and is what the deployed stack implements; Revision 1 was withdrawn; Revision 2 is a Draft (ZIP-316, *Revisions*). This volume documents Revision 0 and states Revision 2 in §2.8, classified *specified, not deployed*; its reference implementation is librustzcash PR #2199, absent from the deployed crates. Deployed-behaviour claims below are verified against librustzcash commit `62ee526`, the orchard repository at `bef8a27`, and the crates-io releases zcash_spec 0.2.1, zip32 0.2.0, and sapling-crypto 0.6.0 (exact versions pinned by the librustzcash workspace `Cargo.lock`); ZIP text is quoted from the zips repository at commit `c6b7035`.

## 2.1 Accounts: ZIP-32 key paths and wallet seeds

Every address below hangs off an *account*, a node of the ZIP-32 hierarchical-deterministic key tree. Sapling and Orchard share the path shape

$$m \,/\, \mathtt{purpose}' \,/\, \mathtt{coin\_type}' \,/\, \mathtt{account}',$$

with $\mathtt{purpose} = 32'$ (`0x80000020`), the SLIP-44 coin type (133 on mainnet, 1 on all testnets), the account index counting from 0, and every level hardened (ZIP-32, *Key path levels*; *Sapling key path*; *Orchard key path*; deployed constants in zcash_protocol, `constants/`). Two absences are deliberate. There is no change level: shielded change pays the wallet's own address and is indistinguishable from any other output, so the transparent external/internal split buys nothing (ZIP-32, *Key path levels*). And there is no non-hardened tail: Orchard ZIP-32 defines hardened-only derivation, instantiated with $\mathsf{MKGDomain} = \mathtt{"ZcashIP32Orchard"}$ and $\mathsf{CKDDomain} = \mathtt{0x81}$ over the extended spending key $(\mathsf{sk}, c)$; a child may, with small probability, yield a spending key whose external or internal full viewing key is invalid, in which case derivation at that index simply fails — ZIP-32 prescribes no recovery, and the deployed stack surfaces an error (§1.6; ZIP-32, *Specification: Orchard key derivation*). Wallets MUST support this path for accounts 0 to $2^{31} - 1$ and MUST support generating each account's default payment address; a stream of diversified addresses MAY be supported (ZIP-32, *Key path levels*; *Sapling key path*; *Orchard key path*). The seed feeding the tree MUST be 32–252 bytes with at least 256 bits of entropy: an attack on a lower-entropy mnemonic can be run against many wallets at once, by matching derived addresses and viewing keys against observed ones (ZIP-32, *Specification: Wallet seeds*).

## 2.2 Diversified addresses

A diversified address multiplies addresses without multiplying keys: one incoming viewing key $\mathsf{ivk}$, one diversifier key $\mathsf{dk}$, and one 88-bit *diversifier* $d$ per address. The *Orchard Guide* ("Keys: capabilities for spending and for viewing") constructs the map and its design rationale — a keyed permutation of the index space, rather than a hash or a random draw; the wallet layer fixes the index discipline that ZIP-32 shares between Sapling and Orchard.

**Definition 2.1** (Diversifier derivation, Sapling and Orchard)**.** For an account's diversifier key $\mathsf{dk}$, the diversifier at *diversifier index* $j$ is

$$d_j := \mathsf{FF1\text{-}AES256.Encrypt}\big(\mathsf{dk},\ \texttt{""},\ \mathrm{I2LEBSP}_{88}(j)\big), \qquad j \in \{0, \ldots, 2^{88} - 1\},$$

where $\mathsf{FF1\text{-}AES256}$ is NIST SP 800-38G FF1 with a 256-bit AES key, radix 2, *minlen* = *maxlen* = 88, and the tweak always the empty string; input and output are 88-bit sequences. The protocol specification abbreviates the keyed permutation as $\mathrm{PRP}^d_K(d) := \mathsf{FF1\text{-}AES256}_K(\texttt{""}, d)$ (ZIP-32, *Sapling diversifier derivation*; *Orchard diversifier and OVK derivation*; protocol specification §5.4.4, *Pseudo Random Permutations*).

The Orchard diversified payment address at index $j$ is then

$$d := \mathrm{PRP}^d_{\mathsf{dk}}\big(\mathrm{I2LEBSP}_{88}(j)\big), \qquad \mathsf{g_d} := \mathsf{DiversifyHash}^{\mathsf{Orchard}}(d), \qquad \mathsf{pk_d} := [\mathsf{ivk}]\,\mathsf{g_d},$$

the pair $(d, \mathsf{pk_d})$, with the address at index 0 the *default* diversified payment address; the last step is the key-agreement $\mathsf{KA}^{\mathsf{Orchard}}.\mathsf{DerivePublic}$ of the protocol specification, and the *Orchard Guide* constructs all three maps. Diversifier indices SHOULD NOT be chosen at random, and an address generator MAY encode recoverable information in the index — the $\mathsf{dk}$ holder recovers it by FF1 decryption (protocol specification §4.2.3, *Orchard Key Components*).

The deployed cipher is the `fpe` crate's `FF1<Aes256>` at radix 2; the `orchard` crate pins `fpe` 0.6 in its `Cargo.toml`. Both protocols encrypt the index bytes `j.as_bytes()` wrapped as a `BinaryNumeralString` via `from_bytes_le` — the 11-byte little-endian index with bits least-significant-first within each byte, which is exactly $\mathrm{I2LEBSP}_{88}(j)$ — under the empty tweak, and convert the 88-bit output back with `to_bytes_le` into the 11-byte diversifier, exactly $\mathrm{LEBS2OSP}_{88}$ (`orchard`, `src/keys.rs`; `sapling-crypto` 0.6.0, `src/zip32.rs`).

The permutation earns its keep quantitatively. An 88-bit diversifier is too short to be cryptographically unguessable at the 128-bit security level, and diversifiers chosen at random begin to collide by the birthday bound as the number of draws approaches $2^{44}$; a keyed permutation reaches the full $2^{88}$ range with no repetition, provided the input index never repeats for a given node (protocol specification §5.4.1.6, notes). The remaining rationale — per-account sequences pseudorandom and mutually uncorrelated, and the issue-count leak a bare counter would cause — is the *Orchard Guide*'s diversifier construction, cited above.

**Remark 2.2** (FF1 parameter security)**.** The property the design requires is that FF1-AES256, with the tweak fixed to the empty string, is a secure pseudo-random permutation. As parameterized here — radix 2, an 88-bit domain, 10 rounds per NIST SP 800-38G — it is unaffected by the DKLS2020 attacks on small-domain FF1: at their parameters $r = 5$ (half the number of rounds) and $n = 44$ (half the domain size in bits), the distinguishing attack has data complexity $2^{2n((r-1)-1/2)-n} = 2^{264}$ and time complexity $2^{2n((r-1)-1/2)} = 2^{308}$, no better than brute-forcing the 256-bit key (protocol specification §5.4.4).

**Definition 2.3** (Diversifier validity and default diversifiers)**.** For Sapling,

$$\mathsf{DiversifyHash}^{\mathsf{Sapling}}(d) := \mathsf{GroupJHash}^{\mathsf{URS}}\big(\texttt{"Zcash\_gd"},\ \mathrm{LEBS2OSP}_{88}(d)\big),$$

valued in the prime-order subgroup of Jubjub excluding zero, or $\perp$; the diversifier $d_j$ is *valid* iff the result is not $\perp$, which for a given $\mathsf{dk}$ holds for approximately half of all indices $j$. The Sapling *default diversifier* is $d_j$ for the least nonnegative $j$ yielding a valid diversifier, and a Sapling account has at most approximately $2^{87}$ payment addresses (ZIP-32, *Sapling diversifier derivation*; *Sapling key path*). For Orchard, with $P := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-gd}, \mathrm{LEBS2OSP}_{88}(d))$,

$$\mathsf{DiversifyHash}^{\mathsf{Orchard}}(d) := \begin{cases} \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-gd}, \texttt{""}) & \text{if } P = \mathcal{O}, \\ P & \text{otherwise,} \end{cases}$$

which never returns $\perp$: every one of the $2^{88}$ Orchard diversifiers is valid, the default diversifier is $d_0$, and an account has exactly $2^{88}$ payment addresses (protocol specification §5.4.1.6; ZIP-32, *Orchard diversifier and OVK derivation*; *Orchard key path*).

The totality of the Orchard hash — and why it removes Sapling's try-the-next-index rejection loop — is discussed in the *Orchard Guide* ("Keys: capabilities for spending and for viewing"). The residual Sapling failure rate of $\frac{1}{2}$ per index is not a curiosity: it resurfaces when a single index must serve several protocols at once (§2.7).

The two protocols also place $\mathsf{dk}$ at different depths of the key tree. In Sapling the diversifier key is a component of the *extended spending key*, derived down the ZIP-32 tree beside the spending authority:

$$\mathsf{dk}_m := \mathrm{truncate}_{32}\big(\mathsf{PRFexpand}_{\mathsf{sk}_m}([\texttt{0x10}])\big), \qquad \mathsf{dk}_i := \mathrm{truncate}_{32}\big(\mathsf{PRFexpand}_{I_L}([\texttt{0x16}] \,\|\, \mathsf{dk}_{\mathrm{par}})\big),$$

where $I_L$ is the ZIP-32 child-derivation entropy (ZIP-32, *Sapling master key generation*; *Sapling child key derivation*; domain bytes also in `zcash_spec` 0.2.1, `src/prf_expand.rs`). Orchard instead derives $\mathsf{dk}$ directly from the full viewing key $(\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}})$, as one half of a single $\mathsf{PRFexpand}$ output whose other half is $\mathsf{ovk}$:

$$R := \mathsf{PRFexpand}_{\mathrm{LE}_{256}(\mathsf{r_{ivk}})}\big(\texttt{0x82} \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{ak}) \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{nk})\big),$$
$$\mathsf{dk} := R[0..32), \qquad \mathsf{ovk} := R[32..64),$$

the derivation the *Orchard Guide* types in full ($\mathsf{PRFexpand}_K(t)$ is BLAKE2b-512 with personalisation `Zcash_ExpandSeed` over $K \,\|\, t$; ZIP-32, *Conventions*; protocol specification §4.2.3). The consequence is capability parity with a simpler key structure: any holder of the Orchard full viewing key can determine the position of any diversifier in the account's sequence, as in Sapling only a holder of the *extended* full viewing key can (ZIP-32, *Orchard diversifier and OVK derivation*).

**Remark 2.4** (Reuse of $\mathsf{r_{ivk}}$)**.** The randomizer $\mathsf{r_{ivk}}$ serves twice: as the $\mathsf{Commit}^{\mathsf{ivk}}$ trapdoor and as the $\mathsf{PRFexpand}$ key deriving $\mathsf{dk}$ and $\mathsf{ovk}$. If $\mathsf{dk}$ or $\mathsf{ovk}$ is known to an adversary, this reuse prevents proving *perfect* hiding for $\mathsf{Commit}^{\mathsf{ivk}}$ in this context, and modelling $\mathsf{PRFexpand}$ merely as a PRF is insufficient; the specification instead makes a joint assumption on Pallas scalar multiplication and $\mathsf{PRFexpand}$ (protocol specification §4.2.3, security note).

Inverting the permutation recovers the index:

$$j = \mathsf{FF1\text{-}AES256.Decrypt}\big(\mathsf{dk},\ \texttt{""},\ d\big).$$

Decryption alone proves nothing about ownership: every valid diversifier decrypts to *some* index under *every* diversifier key (sapling-crypto 0.6.0, `src/zip32.rs`, doc comment: "all valid diversifiers can be produced by all diversifier keys"), so the ownership test re-derives the address at the decrypted index under the candidate $\mathsf{ivk}$ and compares it with the given address. Deployed: the method `DiversifierKey::diversifier_index` performs the raw FF1 decryption in both protocols, and the checked variants are the Orchard `IncomingViewingKey::diversifier_index` and the Sapling `IncomingViewingKey::decrypt_diversifier` (orchard, `src/keys.rs`; sapling-crypto 0.6.0, `src/zip32.rs`).

**Remark 2.5** (Unlinkability, and an oracle to avoid)**.** The security requirement: given two randomly selected shielded payment addresses of *different* spend authorities and a third address derivable from either, all three using different diversifiers, it is not possible to tell from which of the two authorities the third address derives. For Sapling this holds under DDH on the Jubjub prime-order subgroup when $\mathsf{GroupJHash}$ is modelled as a random oracle, via IK-CPA key privacy of ElGamal; the specification applies the same property and notes to Orchard (protocol specification §5.4.1.6). One pitfall is normative: an implementation SHOULD avoid providing a *chosen-diversifier* oracle — deriving a new address for a given $\mathsf{ivk}$ at an adversary-supplied diversifier — since such an oracle trivially breaks unlinkability for the other diversified addresses of that $\mathsf{ivk}$ (same section, final note).

## 2.3 Raw encodings

An Orchard shielded payment address encodes raw in 43 bytes,

$$\text{LEBS2OSP}_{88}(d) \,\|\, \mathsf{pk_d}^{\star},$$

the 11-byte diversifier followed by the 32-byte star-encoding of $\mathsf{pk_d}$; a decoder rejects the encoding if the point fails to parse or is the zero point (protocol specification §5.6.4.2, *Orchard Raw Payment Addresses*). A Sapling address is likewise $11+32 = 43$ bytes under Jubjub ctEdwards compression, rejected if parsing fails, the encoding is non-canonical, or $\mathsf{pk_d}$ lies outside the prime-order subgroup minus zero — the exclusion of zero dates from specification version 2025.6.0 (protocol specification §5.6.3.1, *Sapling Payment Addresses*).

The Orchard raw *full viewing key* is 96 bytes,

$$\text{I2LEOSP}_{256}(\mathsf{ak}) \,\|\, \text{I2LEOSP}_{256}(\mathsf{nk}) \,\|\, \text{I2LEOSP}_{256}(\mathsf{r_{ivk}}),$$

invalid if any component is a non-canonical field element, if ak is not a valid affine $x$-coordinate of a Pallas point, or if either the external or the internal ivk derived from it is 0 or $\perp$; the deployed `FullViewingKey::from_bytes` enforces exactly these checks, including both scopes' ivk validity (protocol specification §5.6.4.4, *Orchard Raw Full Viewing Keys*; orchard, `src/keys.rs`). The Orchard raw *incoming viewing key* is 64 bytes, $\mathsf{dk} \,\|\, \text{I2LEOSP}_{256}(\mathsf{ivk})$, where ivk MUST be a nonzero canonical Pallas base field element; deployed as `IncomingViewingKey::{to_bytes, from_bytes}` over a `NonZeroPallasBase` (protocol specification §5.6.4.3, *Orchard Raw Incoming Viewing Keys*; orchard, `src/keys.rs`).

None of these raw Orchard encodings has a string form of its own: no Bech32 or Bech32m encoding is defined for an individual Orchard address, incoming viewing key, or full viewing key — the unified encodings below are their only string carrier (protocol specification §5.6.4.2). Sapling, by contrast, retains standalone address strings (mainnet HRP `zs`, testnet `ztestsapling`; protocol specification §5.6.3.1), and the deployed renderer honours the asymmetry: the method `Receiver::to_zcash_address` emits a Sapling receiver as a bare `zs` address and a transparent receiver as a `t`-address, but an Orchard receiver only as a single-receiver unified address (zcash_keys, `address.rs`).

## 2.4 Unified addresses

A unified address (UA) bundles at most one *receiver* per pool into one string, so the recipient distributes a single address and the sender's wallet chooses the pool. The choice is normative, by the *priority list* of Table 2: the Sender of a payment to a UA MUST use the receiver of the most preferred type it supports — Orchard over Sapling over P2SH over P2PKH. A wallet MAY let its user configure which receiver types it can send to, but MUST NOT allow the priority order itself to be changed, except via experiments (ZIP-316, *Encoding of Unified Addresses*; mirrored in protocol specification §5.6.4.1, *Unified Payment Addresses and Viewing Keys*).

**Definition 2.6** (Unified container, raw encoding)**.** The raw encoding of a UA, UFVK, or UIVK is the concatenation, over its items in *ascending* typecode order, of

$$\text{compactSize}(\textit{typecode}) \,\|\, \text{compactSize}(\ell_{\textit{item}}) \,\|\, \textit{item},$$

where typecode and length values MUST be at most `0x02000000` (the deployed bound is `MAX_COMPACT_SIZE` in zcash_encoding, `src/lib.rs`). A transparent receiver is the bare 20-byte hash — the P2PKH validating-key hash or the P2SH script hash — excluding the two version bytes of the base58check raw encoding (ZIP-316, *Encoding of Unified Addresses*).

Encoding in a fixed, non-priority order makes the byte encoding canonical; the deployed container preserves the parse order in `items_as_parsed` for round-trip serialization and separately offers `items()` sorted by preference (ZIP-316, *Rationale for Item ordering*; zcash_address, `kind/unified.rs`).

| Typecode | Item type | Receiver | UFVK item | UIVK item |
|---|---|---|---|---|
| 0x00 | transparent P2PKH | 20 | 65 | 65 |
| 0x01 | transparent P2SH | 20 | — | — |
| 0x02 | Sapling | 43 | 128 | 64 |
| 0x03 | Orchard | 43 | 96 | 64 |

Table 2: Revision 0 item typecodes with byte lengths of the receiver, UFVK-item, and UIVK-item encodings. *Priority* is descending typecode among these four — 0x03 is most preferred — while encoding order is *ascending* typecode, deliberately not priority order. P2SH viewing-key items do not exist in Revision 0 (ZIP-316, *Encoding of Unified Addresses*; *Encoding of Unified Full/Incoming Viewing Keys*).

Revision 0 imposes containment rules. A UA or UVK MUST contain at least one non-metadata item; a Revision 0 UA MUST contain at least one *shielded* item (typecode 0x02 or 0x03), and a Revision 0 UVK likewise (Revision 2 drops the requirement for UVKs). A consumer MUST reject: duplicate typecodes; both P2SH and P2PKH present; only metadata items; items out of ascending typecode order; trailing bytes; or any constituent item failing its own validation. A consumer MUST ignore items with unrecognized typecodes (except the MUST-understand metadata of §2.8), and every wallet must still parse a container holding unrecognized items (ZIP-316, *Requirements for both Unified Addresses and Unified Viewing Keys*; *Receivers*). There is intentionally no typecode for Sprout addresses or viewing keys: ZIP-211 disabled sending funds into the Sprout pool at Canopy (ZIP-316; protocol specification §5.6.4.1).

The deployed parser implements these rules once, identically for all three container types (`Address`, `Ufvk`, `Uivk`): the function `try_from_items_internal` rejects out-of-order typecodes (`InvalidTypecodeOrder`), duplicates (`DuplicateTypecode`), the P2PKH/P2SH combination (`BothP2phkAndP2sh`), and only-transparent containers (`OnlyTransparent`); typecodes 0x04 through 0x02000000 parse as `Typecode::Unknown`, and larger values fail as `InvalidTypecodeValue` (`zcash_address`, `kind/unified.rs`).

**Remark 2.7** (Unknown items versus the shielded requirement)**.** The deployed code renders "at least one shielded item" as "not only transparent items", and deliberately counts unknown typecodes as *not* transparent (comment in `zcash_address`, `kind/unified.rs`): a UA containing only a P2PKH receiver plus one unknown-typecode item parses successfully, although it contains no item of typecode 0x02 or 0x03. The ZIP's parenthetical "currently Typecodes 0x02 and 0x03" suggests this forward-compatible reading is intended, but the ZIP nowhere states that an unknown item may satisfy the shielded requirement. We flag the divergence and do not resolve it.

Within a single container, all items obtained by hierarchical deterministic derivation SHOULD represent the same account — the ZIP-32 and BIP-44 account level (ZIP-316, *Requirements for both Unified Addresses and Unified Viewing Keys*). Above the parser, the shielded requirement is enforced at construction as well: `UnifiedAddress::from_receivers` returns `None` without a Sapling or Orchard receiver, and address generation fails with `ShieldedReceiverRequired` (`zcash_keys`, `address.rs`, `keys.rs`).

The deployed sendable-address surface is the enum `zcash_keys::address::Address`: a bare Sapling `PaymentAddress`, a `TransparentAddress` (P2PKH or P2SH), a `Unified` address, or a `Tex` address — the ZIP-320 transparent-source-only form wrapping a 20-byte P2PKH hash. A unified address can receive a memo iff it has a Sapling or Orchard receiver (`unified::Address::can_receive_memo`; `zcash_address`, `kind/unified/address.rs`).

### 2.5 String encoding: F4Jumble and Bech32m

The raw container still needs a string form, and the obvious choice — Bech32m directly over the raw bytes — has a human-factors flaw at UA lengths. The Bech32m checksum does not *cascade*: a user who visually verifies only the beginning and end of a long string can miss an adversarial substitution of an internal receiver. ZIP-316 therefore passes the whole payload through *F4Jumble*, an unkeyed four-round Feistel construction approximating a random permutation of the entire byte string, before the checksummed encoding: any local change diffuses over the whole string, giving near-second-preimage resistance and nonmalleability — for example, an adversary cannot delete a shielded receiver so that only a transparent one remains. Three rounds would not suffice: a controlled xor-difference attack fixes the input to $H_0$. The HRP, zero-padded to 16 bytes, is appended *inside* the jumbled payload, extending the same anti-malleation protection to the HRP itself and defending against cross-network confusion and address-versus-viewing-key confusion (ZIP-316, *Jumbling*; f4jumble, `src/lib.rs`, crate documentation).

**Definition 2.8** (Unified string encoding)**.** Let *raw* be the encoding of Definition 2.6 and *padding* the container's HRP in US-ASCII, padded to exactly 16 bytes with zero bytes; the total length must satisfy $\ell(raw) \le \ell_M^{\mathrm{MAX}} - 16$. The string encoding is

$$\mathrm{Bech32m}\big(\mathrm{HRP},\ \mathsf{F4Jumble}(raw \,\|\, padding)\big),$$

ignoring the BIP-350 length restrictions (Bech32m was chosen over Bech32 for better checksum behaviour at variable lengths). Decoding MUST: Bech32m-decode, rejecting bad checksums; reject out-of-range lengths; apply $\mathsf{F4Jumble}^{-1}$; verify and strip the trailing 16-byte padding; then parse the items, rejecting the whole encoding on any failure (ZIP-316, *Encoding of Unified Addresses*; deployed as `to_jumbled_bytes` and `parse_items` in zcash_address, `kind/unified.rs`).

| Container | Mainnet | Testnet | Regtest |
|---|---|---|---|
| Unified address | `u` | `utest` | `uregtest` |
| Unified FVK | `uview` | `uviewtest` | `uviewregtest` |
| Unified IVK | `uivk` | `uivktest` | `uivkregtest` |

Table 3: Revision 0 human-readable parts, as deployed. ZIP-316 defines the mainnet and testnet rows (ZIP-316, *Revisions*); the regtest column is a librustzcash extension, present only in zcash_protocol, `constants/regtest.rs`.

**Definition 2.9** (F4Jumble)**.** Let $M$ be a message of $\ell_M$ bytes with $38 \le \ell_M \le \ell_M^{\mathrm{MAX}} := (2^{16} + 1) \cdot 64 = 4194368$, and let $\ell_H := 64$. Set $\ell_L := \min(\ell_H, \lfloor \ell_M/2 \rfloor)$ and $\ell_R := \ell_M - \ell_L$, and split $M = a \,\|\, b$ with $\ell(a) = \ell_L$. Then, with all letters local to this definition,

$$x := b \oplus G_0(a), \qquad y := a \oplus H_0(x), \qquad d := x \oplus G_1(y), \qquad c := y \oplus H_1(d),$$

and $\mathsf{F4Jumble}(M) := c \,\|\, d$. The round functions are

$$H_i(u) := \text{BLAKE2b-}(8\ell_L)\big(\texttt{"UA\_F4Jumble\_H"} \,\|\, [i, 0, 0],\ u\big),$$
$$G_i(u) := \text{first } \ell_R \text{ bytes of } \Big\|_{j=0}^{\lceil \ell_R/\ell_H \rceil - 1} \text{BLAKE2b-512}\big(\texttt{"UA\_F4Jumble\_G"} \,\|\, [i] \,\|\, \mathrm{I2LEOSP}_{16}(j),\ u\big),$$

each personalisation filling the 16-byte BLAKE2b personal field. The inverse applies the four XOR rounds in reverse order (ZIP-316, *Jumbling*).

The deployed implementation keeps the state as *left* ‖ *right* with *left* the first $\min(64, \lfloor \ell_M/2 \rfloor)$ bytes, and runs `g_round(0)` ($right \oplus= G_0(left)$, emulating the XOF chunkwise in 64-byte blocks), `h_round(0)`, `g_round(1)`, `h_round(1)`, with the inverse running the same rounds in reverse; this

matches the formulas above under the identification of $(a, b, x, y, c, d)$ with the successive half-states (f4jumble, `src/lib.rs`). One bound differs: the crate accepts message lengths $48 \le \ell_M \le 4194368$ (`VALID_LENGTH`) — the Revision 0 minimum of 48 rather than Revision 2's 38. ZIP-316 states the difference is unobservable for Revision 0-valid item sets, since no such set parses to 22–31 content bytes, and Revision 2 consumers MAY apply either minimum to Revision 0 containers (ZIP-316, *Rationale for length restrictions*; f4jumble, `src/lib.rs`). On the checksum side, the deployed encoder uses a custom checksum type `Bech32mZip316`, identical to Bech32m except that `CODE_LENGTH` $= 4194368 = \ell^{\mathrm{MAX}}$ lifts the BIP-350 length cap; decoding rejects any string whose HRP does not match a known network (main, test, regtest) for the container type (zcash_address, `kind/unified.rs`).

The jumble is cheap at address lengths: 4 BLAKE2b compressions for $\ell_M \le 128$ bytes — which covers both common UA shapes, since a Sapling-plus-Orchard UA has $\ell_M = 45 + 45 + 16 = 106$ bytes and a transparent-plus-Sapling-plus-Orchard UA $\ell_M = 22 + 45 + 45 + 16 = 128$ — rising to 6 for $\ell_M \le 192$, 10 for $\ell_M \le 256$, and 196608 at $\ell_M = \ell_M^{\mathrm{MAX}}$. A consumer unable to handle maximum-length inputs MUST still support $\ell_M \ge 256$ bytes: two conformance levels (ZIP-316, *Efficiency*).

Display is normative too: if a UA or UVK is abridged, at least the first 20 characters MUST be shown. The worst case — the longest mainnet HRP, `uview`, plus the separator — leaves 14 characters, about 70 bits of jumbled data, below the $2^{125}$ security target, so 20 is an absolute minimum; and only *initial* characters count, since different displays may otherwise show disjoint substrings of the same string (ZIP-316, *Requirements for both Unified Addresses and Unified Viewing Keys*, and its rationale).

**Example 2.10** (A unified address, end to end)**.** We trace the pipeline on the smallest deployed shape — an Orchard-only mainnet UA — using the test vector derived from the 32-byte seed `000102...1f` at account 9, diversifier index $j = 1$ (zcash_address, `kind/unified/address/test_vectors.rs`). Every value below is script-computed; the key half runs the deployed `SpendingKey::from_zip32_seed` and `FullViewingKey::address_at` (orchard, `src/keys.rs`).

*Receiver.* The account key at $m/32'/133'/9'$ yields the full viewing key, whose PRFexpand image under domain byte `0x82` gives dk; FF1 encryption of the index bytes (Definition 2.1) gives $d_1$, and the key agreement of §2.2 gives $\mathsf{pk_d}$:

| | |
|---|---|
| dk | 82cc9d79742fe5ae9a142b9336a98677b154fe20401eb18998dbed915b0453ce |
| $\mathrm{I2LEOSP}_{88}(1)$ | 010000000000000000000 |
| $d_1$ | 3fadf8edb20a3301e8260a |
| $\mathsf{pk_d}^\star$ | a311f4cbd54d7d6a76baac88c244b0b121c6dc22a8bcce15898e267829fc1e01 |

Decryption inverts the permutation: FF1-AES256.Decrypt(dk, `""`, $d_1$) returns the index bytes above, recovering $j = 1$ — subject to the ownership caveat of §2.2. The neighbouring diversifiers $d_0$ = `e340636542ece1c81285ed` and $d_2$ = `987fd74a2256c596a66f83` share no visible structure with $d_1$: the sequence is pseudorandom in the index.

*Container.* The UA has one item, so the raw encoding of Definition 2.6 is the two header bytes compactSize(`0x03`) $\|$ compactSize(43) = `032b` followed by the 43-byte receiver $d_1 \| \mathsf{pk_d}^\star$, 45 bytes in all. Appending the padded HRP, `75` (`"u"`) then fifteen zero bytes, gives the 61-byte F4Jumble input $M$ with $\ell_L = 30$, $\ell_R = 31$. The four rounds of Definition 2.9, letters as there:

| | |
|---|---|
| $a = M[0..30)$ | 032b3fadf8edb20a3301e8260aa311f4cbd54d7d6a76baac88c244b0b121 |
| $b = M[30..61)$ | c6dc22a8bcce15898e267829fc1e017500000000000000000000000000000000 |
| $x = b \oplus G_0(a)$ | e6f1529b03a0ad0a4209da8e4365ca14a5988bf1d03084a1c326f929c574ca |
| $y = a \oplus H_0(x)$ | 37b358eb784bb2b315fd35c595628cfa9aa8045ef271728dd0cbf84dc81a |
| $d = x \oplus G_1(y)$ | ad26d7fb043e09cbf18e8f4d80d2b423ecb01719170463d644be4c76daab6b |
| $c = y \oplus H_1(d)$ | 98979f9a7d2dfce7b1616a8d2b9975f87878171b6fcf33b303f770b4aa64 |

The plaintext structure is legible in $a$ and $b$ — the TLV header `032b`, the diversifier, the padding — and none of it in $\mathsf{F4Jumble}(M) = c \| d$.

*String.* Regrouping the 61 jumbled bytes into 98 five-bit symbols (the last padded with two zero bits) and appending the 6-symbol Bech32m checksum yields, with the HRP and separator, $1 + 1 + 98 + 6 = 106$ characters:

```
u1nztelxna9h7w0vtpd2xjhxt4lpu8s9cmdl8n8vcr7actf2ny45nd07cy8cyuhuvw3axcp545y0ktq9cezuzx84jyhex8dk4tdvwhu4dl
```

Decoding reverses each stage per Definition 2.8, the inverse jumble running the same rounds in reverse order. The generating scripts reproduce this string through the deployed encoder (zcash_address, `unified::Address::encode`), re-encode all 60 vectors of the file above, and match all 8 vectors of f4jumble, `src/test_vectors.rs`, in both directions.

## 2.6 Unified viewing keys

A unified full viewing key (UFVK) and unified incoming viewing key (UIVK) reuse the container wholesale: the same item layout (Definition 2.6), the same containment rules, and the same jumbled Bech32m pipeline under their own HRPs (Table 3), with viewing-key items in place of receivers at the same typecodes (Table 2; ZIP-316, *Encoding of Unified Full/Incoming Viewing Keys*). The items:

- *Orchard* (`0x03`): the UFVK item is the 96-byte raw full viewing key of §2.3; the UIVK item is the 64-byte raw incoming viewing key $\mathsf{dk} \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{ivk})$.
- *Sapling* (`0x02`): the UFVK item is 128 bytes, $\mathsf{EncodeExtFVKParts}(\mathsf{ak}, \mathsf{nk}, \mathsf{ovk}, \mathsf{dk}) = \mathsf{ak}^\star \,\|\, \mathsf{nk}^\star \,\|\, \mathsf{ovk} \,\|\, \mathsf{dk}$ under Jubjub compression, and SHOULD be derived from the account-level ZIP-32 extended full viewing key (ZIP-316; the encoding function is defined in ZIP-32). The UIVK item is 64 bytes, $\mathsf{dk} \,\|\, \mathrm{I2LEOSP}_{256}(\mathsf{ivk})$ with a zero ivk invalid (per protocol specification version 2025.6.0) — deliberately *more* than the protocol specification's Sapling incoming viewing key, which is ivk alone: carrying dk is what makes UAs derivable from a UIVK (sapling-crypto 0.6.0, `src/zip32.rs`).
- *Transparent P2PKH* (`0x00`): the UFVK item is 65 bytes, the chain code $c$ followed by the 33-byte compressed public key — the last 65 bytes of the BIP-32 extended public key — at the account level $m/44'/\texttt{coin\_type}'/\texttt{account}'$; the UIVK item has the same shape one level down, at the external change level $m/44'/\texttt{coin\_type}'/\texttt{account}'/0$ (the deployed doc comment pins $m/44'/133'/\texttt{account}'/0$ for mainnet). Items of type P2SH (`0x01`) MUST NOT appear in Revision 0 UFVKs or UIVKs (ZIP-316, *Encoding of Unified Full/Incoming Viewing Keys*).

The deployed item types mirror the sizes exactly: `unified::Fvk::{Orchard([u8; 96]), Sapling([u8; 128]), P2pkh([u8; 65])}` and the `unified::Ivk` analogues (64/64/65) (zcash_address, `kind/unified/fvk.rs`, `kind/unified/ivk.rs`).

**Remark 2.11** (Viewing-key items are pruned)**.** The transparent items deliberately omit the BIP-32 extended-key metadata — depth, parent fingerprint, child number — because the child number would reveal the wallet account index; deserialization reconstitutes dummy attributes (depth 3 for the account key, depth 4 for the IVK, parent fingerprint `0xffffffff`) (zcash_address, `kind/unified/fvk.rs`; zcash_transparent, `keys.rs`). The Sapling item is pruned as well: the deployed `DiversifiableFullViewingKey` is exactly $(\mathsf{ak}, \mathsf{nk}, \mathsf{ovk}, \mathsf{dk})$ with no chain code, so round-tripping a UFVK loses the ability to perform non-hardened Sapling child derivation below the account (sapling-crypto 0.6.0, `src/zip32.rs`) — consistent with the ZIP, which only says the item SHOULD be *derived from* the account-level extended full viewing key.

The priority list of Table 2 does not govern viewing keys the way it governs receivers: a consumer of a UVK normally takes the *union* of the information in all items rather than selecting one, and although the current FVK and IVK types reuse the receiver typecodes, future viewing-key types need not correspond to receiver types (ZIP-316, *Encoding of Unified Full/Incoming Viewing*

*Keys*). The deployed container nevertheless returns `items()` sorted in preference order for UVKs just as for UAs, keeping `items_as_parsed` for round-trips (zcash_address, `kind/unified.rs`); the one place preference genuinely orders viewing-key material is the selection of outgoing viewing keys, which ZIP-316 specifies separately.

Namely, ZIP-316 refines which ovk a sender uses for its outgoing ciphertexts (the $C_{\text{out}}$ construction of the *Orchard Guide*, "Transmission and detection of a note"): the Sender determines a sending account — preferably from the wallet API, else when all inputs belong to a single account — and a *preferred sending protocol*, the most preferred receiver type among the funds being sent, and SHOULD use the external or internal ovk (according to the transfer type) of that protocol from the account-level full viewing key; if no account can be determined, it SHOULD pick one source address of the preferred protocol and use its FVK's ovk (ZIP-316, *Usage of Outgoing Viewing Keys*). Deployed: `UnifiedFullViewingKey::select_ovk(scope, input_sources)` picks the OVK of the highest-preference pool represented among the inputs (zcash_keys, `keys.rs`).

The external/internal split just used comes with a strict capability separation: a full viewing key holder can derive both the external and the internal incoming viewing keys and ovks; the external IVK holder can derive neither the internal IVK nor any ovk; the external ovk holder can derive neither the internal ovk nor any IVK. For the shielded protocols this follows from one-wayness of PRFexpand and of $\mathsf{CRH}^{\mathsf{ivk}}/\mathsf{Commit}^{\mathsf{ivk}}$, with the internal-key derivations specified in ZIP-32 (ZIP-316, *Deriving Internal Keys*). The Orchard internal full viewing key replaces only the commitment randomizer — the `0x83` derivation of Definition 1.6, from which the internal dk, ivk, and ovk follow by the standard component derivations. Transparent P2PKH accounts, having no protocol-level ovk, get the synthetic `0xd0` pair of Definition 1.7 (ZIP-316, *Deriving Internal Keys*; domain bytes in zcash_spec 0.2.1, `src/prf_expand.rs`).

A UIVK derives from a UFVK item by item, preserving typecodes: the Sapling FVK maps to its IVK per protocol specification §4.2.2, the Orchard FVK to its external-scope IVK per §4.2.3, and the transparent P2PKH account key to the external-change key by non-hardened child derivation at index 0; items with unrecognized typecodes MUST be dropped (as they must be again when deriving a UA from a UIVK). The deployed method `UnifiedFullViewingKey::to_unified_incoming_viewing_key` maps Sapling via `to_external_ivk`, Orchard via `to_ivk(Scope::External)`, transparent via `derive_external_ivk`, and sets the unknown-item list to empty, matching the ZIP (ZIP-316, *Deriving a UIVK from a UFVK*; zcash_keys, `keys.rs`).

## 2.7 Deriving a unified address from a UIVK

A UA derives from a UIVK at a *single* diversifier index $j$, and $j$ must be valid simultaneously for every item: the Sapling component requires $j$ to yield a valid Sapling diversifier (Definition 2.3); the transparent component requires $0 \le j \le 2^{31} - 1$, since $j$ becomes the BIP-44 non-hardened index-level child, giving the receiver path $m/44'/\texttt{coin\_type}'/\texttt{account}'/0/j$; the Orchard component imposes no constraint (ZIP-316, *Deriving a Unified Address from a UIVK*). The deployed index map makes the transparent constraint concrete: the function `to_transparent_child_index` splits the 11 little-endian index bytes into the low 4 and the high 7, requires the high 7 to be zero, and parses the low 4 as a `NonHardenedChildIndex`, whose maximum is $2^{31} - 1$ — exactly the ZIP range (zcash_keys, `keys.rs`; zcash_transparent, `keys.rs`).

Which receivers a generated UA carries is decided per call, by the request type `UnifiedAddressRequest`: either `AllAvailableKeys` or `Custom` over `ReceiverRequirements`, one setting of `Require`, `Allow`, or `Omit` for each of Orchard, Sapling, and P2PKH; the constructor rejects any combination in which both Orchard and Sapling are `Omit`, since no shielded receiver could result. The preset constants are `ALLOW_ALL` = (`Allow`, `Allow`, `Allow`), `SHIELDED` = (`Allow`, `Allow`, `Omit`), and `ORCHARD` = (`Require`, `Omit`, `Omit`). Intersecting two requirements prefers `Require` and `Omit` over `Allow` and errors on `Require` meeting `Omit` (zcash_keys, `keys.rs`).

Generation at an index, `UnifiedIncomingViewingKey::address(j, request)`, derives each requested receiver at the same $j$: the Orchard receiver via `address_at(j)`, which is total and never fails; the Sapling receiver via the diversifiable IVK's `address_at(j)`, which yields no receiver at an invalid diversifier — an error only when Sapling is `Required`, a silent omission under `Allow`; the transparent receiver via `to_transparent_child_index` and public derivation, where an out-of-range index errors under `Require` and omits under `Allow`. The result must retain at least one shielded receiver. The search `find_address(j, request)` increments $j$ *only* on `InvalidSaplingDiversifierIndex`, returns `DiversifierSpaceExhausted` on overflow past $2^{88} - 1$, and aborts on any other error; `default_address` is `find_address(0)`. The deployed *default unified address* therefore sits at the smallest index valid for all required receivers (zcash_keys, `keys.rs`).

**Remark 2.12** (Whose convention the default address is)**.** ZIP-316 does not specify address-search semantics at all; the smallest-valid-index default above is a zcash_keys convention, not a ZIP mandate — only ZIP-32's per-protocol default diversifiers (Definition 2.3) are normative. The search's tolerance is also narrow: it skips an index only for an invalid Sapling diversifier, so with a `Required` transparent receiver a search that starts or arrives at an index of $2^{31}$ or beyond — or a failed BIP-32 child derivation, probability about $2^{-127}$ — aborts generation rather than advancing (zcash_keys, `keys.rs`).

**Remark 2.13** (Index 0 and cross-wallet recovery)**.** Diversifier index 0 fails to yield a valid Sapling diversifier with probability $\frac{1}{2}$, so a wallet requiring a Sapling receiver often issues its first UA at a higher index — yet some wallets always generate a transparent P2PKH address at index 0. All Zcash wallets MUST therefore assume there may be transparent funds at diversifier index 0 of each ZIP-32 account, even if the wallet itself would never generate a UA at that index; reliable cross-wallet fund recovery depends on this (ZIP-316, note in *Deriving a Unified Address from a UIVK*).

## 2.8 Revision 2 (specified, not deployed)

Revision 2 of ZIP-316 is a Draft as of the zips snapshot `c6b7035`; everything in this subsection is classified *specified*, and none of it exists in the deployed crates — the reference implementation is librustzcash PR #2199, and a search of the local zcash_address sources finds none of the new HRPs, typecodes, or expiry logic. Revision 2 adds (ZIP-316, *Revisions*; Revision 2 changelog):

- *New HRPs.* Unified addresses split into `zu` (shielded-only; transparent items prohibited) and `tu` (transparent-enabled); UFVKs use `uvf` and UIVKs `uvi`; testnet forms append `test` to each HRP.
- *Metadata items.* Typecodes `0xC0`–`0xFC` are reserved for metadata, which MUST NOT be selected as receivers and MUST NOT confer viewing authority. The subrange `0xE0`–`0xFC` is *MUST-understand*: a consumer that does not recognize such an item MUST treat the whole container as unsupported. Typecodes `0xFFFA`–`0xFFFF` are experimental, and registry additions come via 2xxx-series ZIPs (ZIP-316, *Metadata Items*; *Typecode Registry*; *Adding new types*).
- *Address expiration.* Typecode `0xE0` carries a little-endian `u32` Address Expiry Height — the address is expired once the chain height exceeds it — and `0xE1` a little-endian `u64` Unix-time Address Expiry Time. A Revision 2 Sender MUST set a nonzero `nExpiryHeight`; MUST NOT send if its normal `nExpiryHeight` would exceed the Address Expiry Height, which would leak the address's expiry into the transaction; and SHOULD arrange expiry within 24 hours when only an expiry time is given. Expiry metadata is retained when deriving a UIVK and must not increase when deriving a UA; because shared expiry metadata can link diversified addresses, per-address variation is RECOMMENDED (ZIP-316, *Address Expiration Metadata*).

- *Viewing keys.* The at-least-one-shielded-item requirement is dropped for UVKs, and P2SH viewing-key items (BIP-388 wallet policies) are added (ZIP-316, Revision 2 changelog).

**Remark 2.14** (A retroactive rule the deployed parser does not enforce)**.** Revision 2 also legislates for Revision 0 strings: a Revision 0 container — identified by its `u`/`uview`/`uivk` HRP — MUST NOT contain MUST-understand typecodes, and consumers MUST reject violations (ZIP-316, *Metadata Items*). The deployed parser predates this amendment and does not enforce it: every typecode from `0x04` to `0x02000000` parses as ignorable `Typecode::Unknown`, so a `u`-prefixed address carrying an `0xE0` expiry item parses successfully today (`zcash_address`, `kind/unified.rs`). This is correct Revision 0 behaviour under the pre-amendment text and a violation of the Revision 2-amended text; we flag the divergence rather than resolve it.

# 3 Fees (ZIP-317)

Every transaction pays a fee, and the fee is public: it is the transparent value the transaction leaves unclaimed. For a shielded protocol this publicity imposes two requirements that an ordinary fee market does not. The fee must be *computable from public transaction fields alone* — a fee that depended on private data (which notes were spent, what change was returned) would leak that data through the amount paid (ZIP-317, *Requirements*) — and it should be *uniform across wallets*, because a wallet paying an idiosyncratic fee fingerprints every transaction it builds. ZIP-317 resolves both with a single *conventional fee*: a deterministic function of the transaction's public shape that every wallet is expected to pay.

**Remark 3.1** (A convention, not a consensus rule)**.** The conventional fee is a wallet convention, not a consensus requirement. ZIP-317 (*Fee calculation*) states: "It is not a consensus requirement that fees follow this formula; however, wallets SHOULD create transactions that pay this fee, in order to reduce information leakage, unless overridden by the user." Enforcement is economic, not validity-level: nodes are expected to relay, and the recommended block-template algorithm to preferentially mine, transactions paying at least the conventional fee (§3.6).

As of this writing ZIP-317 is organised into *revisions* (Last-Updated 2026-06-26): Revision 0 is Active and is the deployed fee mechanism; Revision 1 (a fee contribution for Ironwood-pool Actions, NU6.3) and Revision 2 (memo bundles, depending on the undeployed ZIP 248) are both Draft. A contribution defined by a revision is included in all later revisions. This volume documents Revision 0 as the deployed rule; we state Revisions 1–2 below and classify them *specified, not deployed*.

## 3.1 The conventional fee

The unit of account is the *logical action*: a pool-neutral measure of how much verification work and chain space a transaction demands. Its inputs are public transaction fields defined in the protocol specification §7.1 (*Transaction Encoding and Consensus*): the total byte sizes *tx_in_total_size* and *tx_out_total_size* of the transparent `tx_in` and `tx_out` fields, and the counts *nJoinSplit*, *nSpendsSapling*, *nOutputsSapling*, *nActionsOrchard*.

**Definition 3.2** (Logical actions, ZIP-317 Revision 0)**.** The transaction's pools contribute

$$\begin{aligned} \mathit{contribution}_{\text{Transparent}} &= \max\left(\left\lceil \frac{\mathit{tx\_in\_total\_size}}{150} \right\rceil, \left\lceil \frac{\mathit{tx\_out\_total\_size}}{34} \right\rceil\right), \\ \mathit{contribution}_{\text{Sprout}} &= 2 \cdot \mathit{nJoinSplit}, \\ \mathit{contribution}_{\text{Sapling}} &= \max(\mathit{nSpendsSapling},\ \mathit{nOutputsSapling}), \\ \mathit{contribution}_{\text{Orchard}} &= \mathit{nActionsOrchard}, \end{aligned}$$

and *logical_actions* is the sum of the contributions of the applicable revision (ZIP-317, *Fee calculation*).

**Definition 3.3** (Conventional fee)**.** The conventional fee, in zatoshis, is

$$conventional\_fee \;=\; marginal\_fee \cdot \max(grace\_actions,\; logical\_actions),$$

with $marginal\_fee = 5000$ zatoshis per logical action and $grace\_actions = 2$ (ZIP-317, *Fee calculation*).

The two byte-size divisors are the standard P2PKH sizes, chosen so that one typical transparent input or output weighs one logical action. The standard input size $150 = 36 + 110 + 4$ decomposes as the outpoint (36), the script — 1 overall-length byte, 1 signature-length byte, a 72-byte signature, 1 sighash-type byte, 1 pubkey-length byte, a 33-byte pubkey, and 1 byte of margin, totalling 110 — and the sequence field (4). The standard output size $34 = 8 + 26$ decomposes as the value (8) and the script — 1 script-length byte plus the 25-byte P2PKH script (ZIP-317, *Rationale for the chosen parameters*).

**Why logical actions, and why these constants.** A naïve schedule would charge per input plus per output. But an Orchard Action spends one note *and* creates one (the *Orchard Guide*, "A single shielded payment, end to end"), and Orchard bundles are padded to a minimum shape (§3.2); charging inputs and outputs separately would penalise exactly that forced padding. Logical actions count an Action once and, via the max in each contribution, do not discriminate between pools (ZIP-317, *Rationale for logical actions*). The transparent contribution is size-based rather than count-based so that large scripts cannot be undercounted: a 2-of-3 P2SH multisig input is roughly 70% larger than a P2PKH input and is charged proportionally (ZIP-317, *Rationale for the chosen parameters*). The grace window $grace\_actions = 2$ exists so that padding a one-action transaction to two actions — the standard behaviour of zcashd and the mobile SDKs — costs nothing; it also makes a low-value input worth sweeping: within the window, an input worth less than the marginal fee is still economic to include when a second input covers the payment plus fee. Two is also *min_actions*, the smallest economically relevant change-producing transaction (ZIP-317, *Rationale for the chosen parameters*). Finally, $marginal\_fee = 5000$ makes the minimal two-action fee 10,000 zatoshis, deliberately restoring the conventional fee that predated ZIP-313 (which had lowered it to 1,000).

**Later revisions (specified, not deployed).** Revision 1 (Draft, NU6.3) adds

$$contribution_{\text{Ironwood}} = nActionsIronwood;$$

Ironwood-pool Actions use the same Action structure as Orchard, so they are counted identically (ZIP-317, Revision 1). Revision 2 (Draft, dependent on ZIP 248 memo bundles) adds

$$contribution_{\text{Memos}} = \max(0,\; nMemoChunks - free\_memo\_chunks),$$

$$free\_memo\_chunks = \begin{cases} 2 & \text{if } nOutputsSapling + nActionsOrchard + nActionsIronwood > 0, \\ 0 & \text{otherwise,} \end{cases}$$

bounding the additional fee for a ZIP-231 memo bundle by 0.0019 ZEC (ZIP-317, Revision 2). Neither revision is implemented in the deployed stack (§3.3).

## 3.2 The fee is computed on padded counts

The counts in Definition 3.2 are those of the final, serialised transaction — *after* the builder pads each shielded bundle to its minimum shape. The deployed padding rules are those of the crates librustzcash builds against (commit 62ee526: sapling-crypto 0.6.0 and orchard 0.12.0, per its `Cargo.lock`):

- *Sapling* (sapling-crypto 0.6.0, `builder.rs`): a `Transactional` bundle with any spend or output requested (or with `bundle_required` set) uses $num\_outputs = \max(outputs, 2)$, the constant 2 being `MIN_SHIELDED_OUTPUTS`. Spends are never padded under `BundleType::DEFAULT`: $num\_spends$ equals $spends$ when $spends > 0$ and 0 otherwise, so an output-only bundle has zero spend descriptions; the floor $\max(spends, 1)$ applies only when `bundle_required` is set. An empty, non-required bundle is omitted entirely.
- *Orchard* (orchard 0.12.0, `builder.rs`): a `Transactional` bundle with $requested = \max(num\_spends, num\_outputs) > 0$ (or `bundle_required`) uses $num\_actions = \max(requested, \texttt{MIN\_ACTIONS})$ with $\texttt{MIN\_ACTIONS} = 2$.

Consequently any nonempty Sapling or Orchard bundle contributes at least two logical actions, and a one-input, one-output payment within a single shielded pool pays exactly the 10,000-zatoshi minimum.

The builder wires the padded counts into the fee call: the method `get_fee` of the transaction builder passes the raw count of Sapling spends added, the Sapling output count via `BundleType::num_outputs` (padding to 2 when the bundle is nonempty), and the Orchard action count via `BundleType::num_actions` (padding to $\texttt{MIN\_ACTIONS} = 2$ when nonempty) (zcash_primitives, `transaction/builder.rs`). The proposal-stage balance computation in zcash_client_backend does the same — additionally counting the prospective change outputs — using `sapling::builder::BundleType::DEFAULT` and `orchard::builder::BundleType::DEFAULT` (zcash_client_backend, `fees/common.rs` and `data_api/wallet/input_selection.rs`). Fee estimation and final construction therefore agree on the shielded counts by construction.

**Remark 3.4** (A forthcoming change to Orchard padding)**.** The unreleased orchard working copy (commit `bef8a27`, NU6.3-era, `src/builder.rs`) changes the requested-action count: when a bundle's flags disable cross-address transfers, a spend and an output can never share an Action, so $requested = num\_spends + num\_outputs$ replaces the max. Its documentation warns explicitly that ZIP-317 fee estimation must account for the larger count. This is future behaviour — *designed, not deployed*; librustzcash 62ee526 builds against orchard 0.12.0, which uses the max.

### 3.3 The deployed fee rule

Module `transaction::fees::zip317` of zcash_primitives implements Definition 3.3 as the type `FeeRule`; the constructor `FeeRule::standard()` instantiates the four ZIP constants,

$$\texttt{MARGINAL\_FEE} = 5000, \quad \texttt{GRACE\_ACTIONS} = 2,$$
$$\texttt{P2PKH\_STANDARD\_INPUT\_SIZE} = 150, \quad \texttt{P2PKH\_STANDARD\_OUTPUT\_SIZE} = 34,$$

and the derived $\texttt{MINIMUM\_FEE} = 10{,}000$ zatoshis is $\texttt{MARGINAL\_FEE} \cdot \texttt{GRACE\_ACTIONS}$. Non-standard parameters are gated behind the `non-standard-fees` feature and reject zero input or output sizes (zcash_primitives, `transaction/fees/zip317.rs`). The generic trait method is

```
fee_required(params, target_height,
transparent_input_sizes, transparent_output_sizes,
    sapling_input_count, sapling_output_count,
        orchard_action_count) -> Zatoshis
```

(zcash_primitives, `transaction/fees.rs`); the ZIP-317 implementation ignores `params` and `target_height` and computes

$$fee \;=\; 5000 \cdot \max\Big(2,\; \max\big(\lceil \tfrac{t_{\text{in}}}{150} \rceil, \lceil \tfrac{t_{\text{out}}}{34} \rceil\big) + \max(s_{\text{in}},\; s_{\text{out}}) + a\Big),$$

where $t_{\text{in}}$ is the sum of the *known* transparent input sizes, $t_{\text{out}}$ the sum of the transparent output sizes, $s_{\text{in}}, s_{\text{out}}$ the Sapling spend and output counts, and $a$ the Orchard action count; overflow yields a `BalanceError` (zcash_primitives, `transaction/fees/zip317.rs`).

Two terms of Definition 3.2 are absent, both scope restrictions rather than divergences. There is no Sprout term: the `FeeRule` trait has no JoinSplit parameter at all, because the deployed wallet stack cannot construct Sprout spends; the $2 \cdot nJoinSplit$ contribution is spec-only. There is no Ironwood term: Revision 1 is not implemented in this checkout (a search of the librustzcash sources finds no Ironwood fee code). The deployed rule is exactly ZIP-317 Revision 0 minus Sprout.

**Transparent input sizing.** Transparent inputs are sized along two paths. At *proposal* stage, a UTXO whose `script_pubkey` is P2PKH is counted as `InputSize::STANDARD_P2PKH` = `Known`(150); any unrecognised script yields `InputSize::Unknown` (zcash_client_backend, `wallet.rs`, `WalletTransparentOutput::serialized_size`). An `Unknown` input makes the fee incomputable: `fee_required` returns `FeeError::UnknownP2shInputs`. The recognised forms are P2PKH, and multisig-in-P2SH where the redeem script is known (zcash_primitives, `transaction/fees/zip317.rs`; zcash_transparent, `builder.rs`). At *build* stage, the method `TransparentInputInfo::serialized_len` computes the P2PKH size exactly: $36 + \big(1 + 74 + 34\big) + 4 = 149$ bytes — the outpoint, a scriptSig of one CompactSize byte plus a 74-byte push of a 73-byte placeholder signature (`MAX_SIG_SIZE` = 72 plus the sighash-type byte) plus a 34-byte push of a 33-byte pubkey, and the sequence field — one byte under the ZIP's 150, which includes a one-byte margin; P2SH multisig inputs are computed from the redeem script (zcash_transparent, `builder.rs`).

**Remark 3.5** (149 versus 150)**.** The proposal path therefore prices a P2PKH input one byte above its serialised size. The discrepancy is deliberate: the `FeeRule` documentation states that the rule "may slightly overestimate fees in case where the user is attempting to spend a large number of transparent inputs. This is intentional and is relied on for the correctness of transaction construction algorithms in the `zcash_client_backend` crate" (zcash_primitives, `transaction/fees/zip317.rs`). An overestimate can only leave a gratuity, never underfund the transaction.

**Transparent output sizing.** The trait method `OutputView::serialized_size` returns 8 (value) plus the script bytes plus the CompactSize length prefix; a P2PKH output is $8{+}1{+}25 = 34$ bytes, matching the ZIP constant (zcash_primitives, `transaction/fees/transparent.rs`).

**The standard rule at the wallet layer.** The crate zcash_client_backend exposes `StandardFeeRule`, an enum whose sole variant `Zip317` delegates to `FeeRule::standard()`, and the extension trait `Zip317FeeRule`, which exposes `marginal_fee()` and `grace_actions()` to the change strategies (zcash_client_backend, `fees.rs`, `fees/standard.rs`, `fees/zip317.rs`). There is no deployed alternative: paying the conventional fee is the only strategy the stack offers.

## 3.4 Change and dust under the fee rule

The fee rule does not act alone; it is consulted by a *change strategy* that turns a tentative selection of inputs and payments into a balanced transaction. Two ZIP-317 strategies exist: `SingleOutputChangeStrategy`, producing one change output, and `MultiOutputChangeStrategy`, splitting change across several notes according to a `SplitPolicy` so the wallet maintains a target number of spendable notes (zcash_client_backend, `fees/zip317.rs`).

**Change-pool selection.** Change goes to Orchard if the transaction has any Orchard input or output; otherwise to Sapling if it has any Sapling input or output; otherwise — fully transparent

flows — to a caller-specified fallback pool. The source marks this a "naive strategy" (TODO) (zcash_client_backend, `fees/common.rs`, `select_change_pool`).

**The balance computation.** The function `single_pool_output_balance` first computes *min_fee*, the fee required with *zero* change outputs (zcash_client_backend, `fees/common.rs`). Then:

1. if $total\_in < subtotal\_out + min\_fee$, it fails with `ChangeError::InsufficientFunds`, reporting $available = total\_in$ and $required = subtotal\_out + min\_fee$ — the figure the input-selection loop of §3.5 feeds back;
2. at exact equality, when all flows are transparent and no change memo is requested, it emits no change output and the fee is *min_fee*;
3. otherwise it computes *max_fee* with the target change-output count and derives $change = total\_in - (subtotal\_out + total\_fee)$.

In the shielded case a *zero-valued* change output is deliberately retained even at exact balance: omitting it would let an adversary who knows the recipients' outputs conclude, from the absence of change, that the inputs sum exactly to payments plus fee; the zero-valued output also carries the change memo (zcash_client_backend, `fees/common.rs`).

**Dust policy.** A change value at or below a dust threshold is handled by a `DustOutputPolicy`. The default is (`DustAction::Reject`, `None`), with the unset threshold defaulting to the marginal fee, 5000 zatoshis (`default_dust_threshold = fee_rule.marginal_fee()`); `Reject` still permits exactly-zero change per the previous paragraph. The `AllowDustChange` action emits the dust change output anyway. The `AddDustToFee` action folds the dust into the fee — with a guard: if the resulting fee exceeds the computed fee by more than $10 \cdot \texttt{MINIMUM\_FEE} = 100{,}000$ zatoshis the policy is rejected, protecting the user from a mis-set threshold; the documentation notes that `AddDustToFee` inherently leaks more information (zcash_client_backend, `fees.rs`, `fees/zip317.rs`, `fees/common.rs`).

**Uneconomic inputs.** An input is classified as dust iff its value is *at most* the marginal fee (zcash_client_backend, `fees/common.rs`, `check_for_uneconomic_inputs`). Dust is admitted only where it rides along free. Per pool $p$,

$$allowed_p = \min\big(\#dust_p,\ (\#outputs_p + \#change_p) - \#nondust_p\big) \quad \text{(saturating at 0)},$$

i.e. dust inputs that do not increase the padded action count. Then, if the baseline logical-action count is below *grace_actions*, at most *one* extra dust input may be added free — pools tried in the order transparent, Sapling, Orchard — admitted iff the hypothetical action count with it does not exceed the baseline, where the hypothetical count is $\max(t_{\text{in}}, t_{\text{out}}) + \max(\text{padded Sapling spends}, \text{padded Sapling outputs}) + \text{padded Orchard actions}$ with transparent inputs assumed P2PKH. When the presence of a change output is uncertain, the minimum allowance over both manifests is used (`possible_change` construction, same file). Dust beyond the allowance is returned as `ChangeError::DustInputs`, which the input selector converts into an exclusion list (§3.5).

**Remark 3.6** (Strictness divergence from the ZIP wording)**.** The ZIP speaks of inputs "with value *below* the marginal fee" (ZIP-317, *Requirements* and *Rationale for the chosen parameters*), while the deployed check classifies value $\leq$ *marginal_fee* — a note of exactly 5000 zatoshis included — as dust, and the SQL layer (§3.5) likewise never offers notes with value $\leq 5000$. The `ChangeError::DustInputs` documentation itself acknowledges the determination is "potentially conservative". We flag this as conservative code behaviour against the ZIP wording, not an error in either (zcash_client_backend, `fees/common.rs`; zcash_client_sqlite, `wallet/common.rs`).

**Splitting change.** The strategy `MultiOutputChangeStrategy` fetches account metadata counting the wallet's existing notes above `min_split_output_value` (default fallback `SplitPolicy::MIN_NOTE_VALUE` = 500,000 zatoshis, documented as `MARGINAL_FEE` · 100); the split count shrinks until each split note is at least the minimum split value (or a quarter of the post-transaction average note value when no minimum is set); if fewer splits than targeted result, the fee is recomputed for the actual count. The division remainder is added to the first change output (zcash_client_backend, `fees.rs`, `fees/zip317.rs`, `fees/common.rs`).

**Example 3.7** (Fee of a split-change transaction)**.** Fix a five-way split policy with minimum split value 1,000,000 zatoshis and a wallet holding no other notes. Spending a 7,500,000-zatoshi Sapling note to pay 1,000,000 incurs a fee of 30,000 zatoshis — one spend against six outputs, the payment plus five change notes, is six logical actions — and leaves five change notes of 1,294,000 zatoshis each. Spending a 6,000,000-zatoshi note under the *same* policy shrinks the split: at the five-change fee of 30,000 the change totals 4,970,000 — 994,000 per note, below the minimum — so the strategy emits four change notes; the fee, recomputed for the payment plus four change notes (five logical actions), is 25,000, and each change note carries 1,243,750. The figures are recomputed by script from the ZIP-317 formula and the split rule above, and match the strategy's own tests (zcash_client_backend, `fees/zip317.rs`, test module). The same tests verify that the strategy refuses an exact-balance two-output transaction without change, requiring the change output's marginal fee, so that a recipient cannot reason about the sender's input values from the absence of change.

### 3.5 Input selection: the fee-escalation loop

Fee and input selection are mutually dependent: adding an input can raise the logical-action count, which raises the fee, which can require another input. The deployed selector, `GreedyInputSelector`, resolves the circularity by iteration rather than by solving for a fixed point directly (zcash_client_backend, `data_api/wallet/input_selection.rs`, `propose_transaction`). Starting from an empty selection, each round:

1. chooses source pools — Sapling inputs are used unless any payment is to Orchard and Orchard funds alone cover the requirement; Orchard inputs are added when Sapling funds are insufficient;

2. calls `ChangeStrategy::compute_balance` on the current selection (the fee evaluated on padded counts, prospective change included);

3. on `Ok`, emits the `Proposal` with the balanced fee; on `Err(DustInputs)`, adds the offending notes to an exclusion list and retries; on `Err(InsufficientFunds{required})`, sets the new target to *required* — which already embeds the fee recomputed for the enlarged, padded action count — and reselects notes from scratch with `TargetValue::AtLeast(required)`.

The loop terminates when a reselection fails to *strictly* increase the total selected value, returning `InsufficientFunds` (invariant comment at `input_selection.rs`). Escalation is thus implicit: each added note can raise the logical-action count by one, raising the required total by one marginal fee, which the next round's selection must cover.

The note store enforces the dust rule below the selector: both note-selection queries in zcash_client_sqlite filter `rn.value` > *min_value* with *min_value* = `zip317::MARGINAL_FEE` = 5000, so notes worth at most 5000 zatoshis are never even offered to selection. Value-targeted selection scans notes oldest-first under a running-sum window, taking every note while the running sum is below the target plus the single note that crosses it (zcash_client_sqlite, `wallet/common.rs`). The candidate notes themselves are the product of the synchronisation described in the *Orchard Guide* ("Transmission and detection of a note: encryption, trial decryption, and synchronisation"); this volume takes that spendable set as given.

**ZIP-320 (TEX) transactions.** A payment to a transparent-source (TEX) address builds two transactions: the first funds an ephemeral transparent output, which the second spends towards the TEX address. The selector prices the ephemeral input as a standard 150-byte P2PKH input and the ephemeral output as a 34-byte P2PKH output. The value the ephemeral output must carry is obtained by balancing the *second* transaction with a zero-valued ephemeral input (`EphemeralBalance::Input(0)`) and reading the `required` field of the resulting `InsufficientFunds` error (`zcash_client_backend`, `input_selection.rs`, `fees/common.rs`). In `propose_send_max`, the second transaction's one-transparent-in, one-transparent-out fee is `fee_required`$([\texttt{Known}(150)], [34], 0, 0, 0) = 10{,}000$ zatoshis — the grace minimum.

**Shielding.** The function `propose_shielding` computes a trial balance over all spendable UTXOs of the source addresses; on `DustInputs` it removes exactly the offending outpoints and recomputes once; the proposal is emitted only if the total reaches the caller's shielding threshold, else `InsufficientFunds` (`zcash_client_backend`, `input_selection.rs`).

### 3.6 Relay, mempool, and block template

The conventional fee binds economically through three node-side mechanisms, all specified in ZIP-317 and ZIP-401. None is a consensus rule. The zebra claims below are verified against its sources (`zebra-chain`, `transaction/unmined/zip317.rs` and `transaction/unmined.rs`; `zebra-rpc`, `types/get_block_template/zip317.rs`; `zebrad`, `mempool/storage/verified_set.rs`); the zcashd claims rest on the ZIP text alone.

**Definition 3.8** (Unpaid actions)**.** For a non-coinbase transaction,

$$\mathit{unpaid\_actions}(tx) = \max\big(0,\ \max(\mathit{grace\_actions},\ tx.\mathit{logical\_actions}) - \lfloor tx.\mathit{fee}/\mathit{marginal\_fee} \rfloor\big),$$

and 0 for coinbase; a block's unpaid actions are the sum over its transactions, bounded by $\mathit{block\_unpaid\_action\_limit} = 50$ in template construction (ZIP-317, *Recommended algorithm for block template construction*).

**Relay.** Nodes are expected to relay transactions paying at least the conventional fee. A transaction with more than $\mathit{block\_unpaid\_action\_limit} = 50$ unpaid actions will never be mined by the recommended template algorithm, and nodes MAY drop such transactions, or those above a configurable limit (ZIP-317, *Transaction relaying*). Concretely: zcashd v5.6.0 exposes `-txunpaidactionlimit` (default 50) and `-blockunpaidactionlimit` overriding the block limit, and its relay threshold — a separate, older mechanism — was simplified in v5.5.0 (zcash/zcash#6542) (ZIP-317, *Deployment*). The zebra node is stricter than the ZIP suggests: since v1.8.0 (ZcashFoundation/zebra#8638) it hard-codes `BLOCK_UNPAID_ACTION_LIMIT` $= 0$, and `mempool_checks` — run on every mempool candidate through `VerifiedUnminedTx::new` (`zebra-consensus`, `transaction.rs`) — rejects any transaction whose unpaid-action count exceeds that limit (`zebra-chain`, `transaction/unmined/zip317.rs`). By Definition 3.8, a transaction has zero unpaid actions iff it pays at least the conventional fee, so zebra admits to its mempool — and therefore relays — only transactions paying the full conventional fee. The same check retains a legacy fee-rate floor of 100 zatoshis per 1000 bytes, clamped to $[100, 1000]$ zatoshis; the source notes it cannot fail once the zero-unpaid-action check passes, the conventional fee being at least 10,000 zatoshis (same file).

**Mempool limiting (ZIP 401, as amended).** Each mempool transaction has

$$\mathit{cost} = \max(\text{memory size in bytes},\ 10{,}000), \qquad \mathit{eviction\_weight} = \mathit{cost} + \mathit{low\_fee\_penalty},$$

with $\mathit{low\_fee\_penalty} = 40{,}000$ if the transaction pays less than the ZIP-317 conventional fee and 0 otherwise, so low-fee transactions are evicted preferentially under memory pressure

(ZIP-401, *Specification*). The threshold was switched from the legacy fee to the ZIP-317 conventional fee in zcashd v5.5.0 and zebrad v1.0.0-rc.7 (ZIP-317, *Mempool size limiting*; ZIP-401, *Deployment*). In zebra, method `cost` returns max(serialized size, 10,000) and `eviction_weight` adds the 40,000-zatoshi penalty when the fee falls below the conventional fee (zebra-chain, `transaction/unmined.rs`); eviction samples at random by that weight (zebrad, `mempool/storage/verified_set.rs`, `evict_one`). Because admission already requires the full conventional fee, the penalty never fires in zebra's own mempool.

**Block template construction.** The recommended algorithm assigns each candidate transaction a weight

$$tx.weight\_ratio = \min\left(\frac{\max(1,\ tx.fee)}{conventional\_fee(tx)},\ weight\_ratio\_cap\right), \qquad weight\_ratio\_cap = 4,$$

the $\max(1, \cdot)$ avoiding an all-zero-weight candidate set. Phase 1 selects randomly, with probability proportional to weight ratio, among candidates paying at least the conventional fee, subject to the block size and sigop limits; phase 2 selects among the remaining candidates subject additionally to the block's unpaid actions staying at most *block_unpaid_action_limit* (Definition 3.8). Whatever an adversary submits, a block built this way carries at most 50 unpaid logical actions (ZIP-317, *Recommended algorithm for block template construction*). In zebra, the two-phase selection (`select_mempool_transactions`; zebra-rpc, `types/get_block_template/zip317.rs`) initialises the phase-2 budget to the hard-coded limit of 0, so a zebra template carries no unpaid actions at all. Overpaying beyond 4× the conventional fee therefore buys no further priority — a deliberate flattening of the fee market that keeps the conventional fee focal.

**Deployment.** ZIP-317 fees became the wallet default in zcashd v5.5.0; the block-template algorithm shipped in zcashd v5.5.0 and zebra v1.0.0-rc.3 (zebra has no wallet, so it computes no fees for construction) (ZIP-317, *Deployment*).

# 4 Transaction construction

The synchronisation procedure of the *Orchard Guide* (§ "Transmission and detection of a note: encryption, trial decryption, and synchronisation") leaves a wallet holding a set of unspent notes, each with a value, a position, and an authentication path available on demand. This section documents how the deployed wallet layer turns that state, together with a payment request, into signed transactions. The pipeline, implemented in zcash_client_backend (`src/data_api/wallet.rs`, module documentation and the two entry points), has two phases. Function `propose_transfer` takes a ZIP-321 payment request, runs input selection and fee-and-change computation, and returns a *proposal* — a complete plan, down to the zatoshi, of every transaction to be built. Function `create_proposed_transactions` executes the proposal: one transaction per step, each proved and signed, all stored together via `store_transactions_to_be_sent`; it returns the transaction ids, and submission to the network remains the caller's responsibility. The split places every decision that requires judgement — which notes to spend, what fee to pay, where change goes — into an inspectable value computed before any key is touched; execution is then deterministic given the plan.

## 4.1 The proposal

**The payment request.** The input is a ZIP-321 `TransactionRequest`: a `BTreeMap` from payment index to payment, implemented in librustzcash's `components/zip321` crate (`src/lib.rs`). A memo attached to a payment addressed to a transparent recipient is rejected — as

`Zip321Error::TransparentMemo` at parse time and again as `Error::MemoForbidden` when outputs are added to the builder (`zcash_client_backend/src/data_api/wallet.rs:1485--1504`) — because a transparent output has no ciphertext to carry one.

**Definition 4.1** (Proposal)**.** A *proposal* carries the fee rule under which it was computed, the minimum target height (the next block height), and a non-empty sequence of *steps* (type `Proposal<FeeRuleT, NoteRef>`; zcash_client_backend, `src/proposal.rs:167--348`), each step describing one transaction: the originating transaction request; a map from payment index to the pool that will pay it; the selected transparent inputs; the selected shielded inputs together with the anchor height at which their witnesses will be computed; references to outputs of prior steps to be spent; the *balance* — the proposed change values and the fee required — and a flag marking shielding steps. Validation of a multi-step proposal (`proposal.rs:397--560`) rejects forward references, spends of non-ephemeral change produced by a prior step, and intra-proposal double spends.

**Target and anchor heights.** In zcash_client_sqlite (`src/wallet.rs:2746--2794`), the heights are

$$\mathsf{target} := \mathsf{tip} + 1, \qquad \mathsf{anchor} := \min_{\text{pools}} \max\{\, h \le \mathsf{target} - c : h \text{ checkpointed} \,\},$$

the "mempool height" and, with $c$ the confirmation depth and "checkpointed" meaning present in the pool's shard tree, the highest checkpoint at least $c$ blocks below the target, minimised across Sapling and Orchard so a single height anchors both bundles. Anchor semantics and witness derivation are those of the *Orchard Guide* (§ "Proof of ownership of *some* valid note: the commitment tree", in particular "Anchor choice as the tree grows", which already develops the ZIP-315 3/10 defaults); here we record only what the pipeline feeds in.

**Definition 4.2** (Confirmations policy)**.** The `ConfirmationsPolicy` of ZIP 315 carries a *trusted* depth (default 3), an *untrusted* depth (default 10), and an `allow_zero_conf_shielding` flag (default true) (`zcash_client_backend/src/data_api/wallet.rs:394--405`; ZIP 315 § "Trusted and untrusted TXOs"). The constructor enforces $\mathsf{trusted} \le \mathsf{untrusted}$. An output is spendable at the trusted depth if its transaction is user-marked trusted or it was received on an internal-scope key, and at the untrusted depth otherwise; outputs of a shielding transaction inherit the mined height and trust of their transparent source inputs; and transparent UTXOs are spendable at zero confirmations for shielding when the flag is set (`wallet.rs:369--607`).

**Remark 4.3** (Anchor depth versus untrusted notes)**.** Function `propose_transfer` obtains its height pair with $\mathsf{minconf} = \mathsf{trusted}$ (`wallet.rs:634--642`) — even when the notes ultimately selected are untrusted and must themselves be 10 confirmations deep. The code comment explains the choice: the shallower anchor admits the maximum number of notes, and trusted-versus-untrusted filtering is deferred to note selection, which tests each note's own depth. The bundle-wide anchor therefore sits at depth $\ge 3$, not $\ge 10$ — which is what ZIP 315 itself recommends. A wallet SHOULD choose the anchor at the trusted confirmation depth: "if the current block is at height $H$, the anchor SHOULD reflect the final treestate of the block at height $H - 3$" (ZIP 315, § "Anchor selection"). The ZIP's stated rationale: a rollback past the anchor block invalidates the transaction while its revealed nullifiers link it to any future transaction spending the same notes; an anchor too many blocks back prevents recently received notes from being spent; and a *fixed* depth — rather than one conditioned on whether the spent notes are trusted — avoids leaking their trust status (ZIP 315, § "Rationale for anchor selection"). Neither the ZIP nor the code comment claims an anonymity-set benefit for the shallower anchor.

**Remark 4.4** (The ZIP-315 floor is advisory in the API)**.** ZIP 315 states that wallets SHOULD NOT permit spending outputs with fewer than 3 confirmations, but the deployed API exposes

`ConfirmationsPolicy::MIN` (1 confirmation) and constructors accepting lower values, gated only by a rustdoc warning about transaction distinguishability (`wallet.rs:369--607`). We present the ZIP recommendation as the norm and the constructors as a deliberate escape hatch.

## 4.2 Input selection

The deployed selector is zcash_client_backend's `GreedyInputSelector` (`src/data_api/wallet/input_selection.rs`); it buckets each payment by receiver type (lines 444–514): a transparent address becomes a transparent output; a TEX address is deferred to a second ZIP-320 step (below); a Sapling address becomes a Sapling output; a unified address is paid to its Orchard receiver if present, else its Sapling receiver, else its transparent receiver, and a unified address with no receiver the wallet supports is an error.

**The greedy loop.** Selection then iterates (`input_selection.rs:516--735`): starting with no inputs, call the change strategy's `compute_balance` (§4.3); on `InsufficientFunds{required}`, ask the backend for spendable notes covering at least `required` (`select_spendable_notes` with `TargetValue::AtLeast`, the target height, the confirmations policy, and the running exclusion list); on `DustInputs`, add the identified notes to the exclusion list and retry; on success, emit the proposal. The loop terminates because the selected value must strictly increase on every round, else the selector fails with `InsufficientFunds`. Pool preference inside the loop (`input_selection.rs:524--543`): Sapling inputs are used iff there are no Orchard outputs or Orchard inputs alone cannot cover the requirement, and Orchard inputs iff Sapling is not used or Sapling inputs alone cannot cover it — spend from the pool matching the outputs first, crossing pools only when needed, which keeps single-pool payments free of cross-pool linkage. Within a pool, the zcash_client_sqlite backend selects the *oldest* unspent notes first: a window-function running sum over notes ordered oldest-to-newest takes every note below the target plus the one note that crosses it (`zcash_client_sqlite/src/wallet/common.rs:299--344, 530--560`).

**ZIP-320 (TEX) payments.** A TEX address demands transparent funds whose provenance is a single ephemeral transparent address, so the selector builds *two* steps (`input_selection.rs:580--629, 974--1051`; ZIP 320 § "Design considerations for Senders"). It sizes the second step first: probing `compute_balance` over the TEX outputs with a zero-valued ephemeral input and catching `InsufficientFunds{required}` yields

$$v_{\mathsf{eph}} \;=\; \sum_{\text{TEX payments}} v \;+\; \mathsf{fee}(\mathrm{tr}_1),$$

where $\mathsf{fee}(\mathrm{tr}_1)$ is computed for exactly one standard P2PKH input (150 bytes) and the TEX payments as P2PKH outputs (34 bytes each) (`fees/common.rs:334--349`). The change-strategy contract requires the second step to balance exactly — $\mathsf{total} = \mathsf{fee\_required}$, no change — which the selector enforces with a runtime assertion (`input_selection.rs:625`). Step $\mathrm{tr}_0$ then replaces the TEX payments with a single ephemeral transparent "change" output of exactly $v_{\mathsf{eph}}$, and step $\mathrm{tr}_1$ spends it and pays the TEX addresses as P2PKH. Paying a TEX address from shielded inputs in a single step is rejected (`PaysTexFromShielded`, `wallet.rs:1539--1546`).

**Shielding.** Function `propose_shielding` (`wallet.rs:805--841`; `input_selection.rs:1066--1183`) targets the tip $+1$ and collects spendable UTXOs from the given source addresses — at zero confirmations under the default policy (Definition 4.2). Funds may be drawn from at most one ephemeral address at a time and never mixed with other addresses (`ProposalError::EphemeralAddressLinkability`), since co-spending would link the ephemeral address to the wallet's others. Dust UTXOs reported by `compute_balance` are dropped and the balance recomputed once; the proposal is built only if its total reaches the caller's shielding

threshold. The resulting single step has an empty transaction request, `is_shielding` set, and all value flowing to internal change.

### 4.3 Fees and change

Fees and change are computed *together*, by a `ChangeStrategy` whose one obligation is `compute_balance` (`zcash_client_backend/src/fees.rs:513--578`). The deployed strategies are `SingleOutputChangeStrategy` and `MultiOutputChangeStrategy` in the `fees::zip317` module; the `fees::standard` module aliases them over `StandardFeeRule`, whose only variant is `Zip317` (`fees/zip317.rs:61--265`; `fees/standard.rs:1--18`). Both delegate to a common routine, `single_pool_output_balance` (`fees/common.rs:204--557`), described below.

**Definition 4.5** (ZIP-317 conventional fee, as deployed)**.** The fee rule (zcash_primitives, `src/transaction/fees/zip317.rs:154--197`; ZIP 317 § "Fee calculation") is

$$\mathsf{fee} \;=\; \mathsf{marginal\_fee} \cdot \max(\mathsf{grace\_actions},\, \mathsf{logical\_actions}),$$

$$\mathsf{logical\_actions} \;=\; \max\!\left(\left\lceil \tfrac{\mathsf{in\_size}}{150} \right\rceil, \left\lceil \tfrac{\mathsf{out\_size}}{34} \right\rceil\right) + \max(n^{\mathsf{Sap}}_{\mathrm{spend}},\, n^{\mathsf{Sap}}_{\mathrm{out}}) + n^{\mathsf{Orch}}_{\mathrm{act}},$$

with $\mathsf{marginal\_fee} = 5\,000$ zatoshi, $\mathsf{grace\_actions} = 2$, the standard P2PKH input and output sizes 150 and 34 bytes, and hence a minimum fee of 10 000 zatoshi (`fees/zip317.rs:19--40`; the ZIP 317 parameter table). The shielded counts fed in are the *padded* counts of §4.5 — `BundleType::num_outputs` and `num_actions` — both here and in the final builder check (`fees/common.rs:299--332`; `zcash_primitives/src/transaction/builder.rs:713--753`).

Two deployment caveats. ZIP 317 Revision 0 additionally defines a Sprout contribution $2 \cdot n_{\mathrm{JoinSplit}}$; the wallet never constructs JoinSplits, so the deployed `FeeRule` omits the term (`fees/zip317.rs:42--53`) — code and ZIP agree on every transaction the wallet can produce, and we flag the omission rather than restate the ZIP formula as implemented. Second, the rule supports P2PKH transparent inputs plus P2SH inputs of known size only; a coin with an unknown P2SH redeem script fails with `FeeError::UnknownP2shInputs`, and the rule intentionally may slightly *overestimate* the fee for many transparent inputs — an overestimate the construction algorithms in zcash_client_backend rely on (`fees/zip317.rs:42--53, 154--197`).

**Change pool.** Change goes to Orchard if the transaction has any Orchard input or output flows; else to Sapling if it has any Sapling flows; else — fully transparent flows, as in shielding — to the caller-supplied fallback pool. The code marks this as deliberately naive (`fees/common.rs:117--138`).

**Balance computation.** Routine `single_pool_output_balance` (`fees/common.rs:204--557`) proceeds as follows. Let $\mathsf{in}$ be the sum of transparent, Sapling, and Orchard inputs (including any ephemeral input) and $\mathsf{out}$ the corresponding sum of outputs; let $\mathsf{fee}_0$ be the ZIP-317 fee with zero change outputs. If $\mathsf{in} < \mathsf{out} + \mathsf{fee}_0$, fail with $\mathsf{required} = \mathsf{out} + \mathsf{fee}_0$ (this is the value the greedy loop feeds back into note selection). If equality holds *and* the flows are fully transparent *and* no change memo is configured, emit no change with $\mathsf{fee} = \mathsf{fee}_0$. Otherwise recompute the fee with the target number of change outputs (re-derived if the split policy below yields fewer), set $\mathsf{change} = \mathsf{in} - (\mathsf{out} + \mathsf{fee})$, and divide it across the split count with the remainder added to the first output, each share a shielded change value in the selected pool carrying the configured change memo.

**Remark 4.6** (Zero-valued change is mandatory)**.** Every transaction with shielded flows produces a change output, even when inputs exactly balance outputs plus fee: a zero-valued shielded change output is created so that an observer cannot distinguish exact-balance transactions, and so that a shielded output always exists to carry the change memo (`fees/common.rs:352--399, 496--516`).

Zero-valued change is allowed even under the `Reject` dust action, and the code notes that zero-valued notes need no witness tracking. Only a fully-transparent-flow transaction without a change memo may omit change — the second ZIP-320 transaction is the deployed example. A configured change memo is attached to every proposed shielded change output, is deliberately discarded for the ZIP-320 step that spends the ephemeral input, and itself forces a change output to exist (`fees/common.rs:219--221, 257--258, 518--541`).

**Dust.** The default `DustOutputPolicy` is `DustAction::Reject` with no threshold override, delegating the threshold to the strategy, whose default is the marginal fee, 5 000 zatoshi (`fees.rs:340--344`; `fees/zip317.rs:132--141`; `fees/common.rs:491--494`). If the total change falls below the threshold: under `Reject`, fail with `InsufficientFunds` and the requirement raised by the shortfall (unless the change is exactly zero, which Remark 4.6 permits); under `AllowDustChange`, keep the dust output; under `AddDustToFee`, burn the change into the fee and omit the output (keeping a zero-valued one if a change memo exists) — *except* when the resulting fee would exceed the computed fee by more than $10 \cdot \mathsf{MINIMUM\_FEE} = 100\,000$ zatoshi (that is, when the folded dust exceeds 100 000), in which case the change output is kept as a defence against value loss (`fees/common.rs:491--548`).

**Uneconomic inputs.** An input is *dust* iff its value is at most the marginal fee; the check runs only when the marginal fee is positive (`fees/common.rs:260--290, 574--769`). Some dust rides for free: per pool $P$, up to

$$\min\big(\,|\mathrm{dust}_P|,\ (\mathsf{outputs}_P + \mathsf{change}_P) - \mathsf{nondust}_P\,\big) \quad \text{(saturating)}$$

dust inputs do not raise the padded action count, plus at most one extra *grace* input — tried transparent first, then Sapling, then Orchard — if the baseline logical-action count is below grace_actions and adding the candidate does not increase the padded count (the padding rules of §4.5 are invoked directly, with and without it). The allowance is the pointwise minimum over all possible change configurations; dust beyond it is returned as `ChangeError::DustInputs`, which the greedy loop converts into exclusions.

**Note management: splitting change.** The `MultiOutputChangeStrategy` splits change into up to `target_output_count` equal notes (remainder to the first) whenever the account holds fewer than that many notes of at least `min_split_output_value`; the note-count metadata comes from `InputSource::get_account_metadata` filtered by that minimum, or by the fallback $\mathsf{MIN\_NOTE\_VALUE} = 100 \cdot \mathsf{marginal\_fee} = 500\,000$ zatoshi (`fees.rs:346--450`; `fees/common.rs:236--255, 417--489`; `fees/zip317.rs:199--264`). The split count is

$$\mathsf{split} = \max\big(1,\ \mathsf{target} - \#\{\text{qualifying notes}\}\big),$$

decremented (floor 1) while $\mathsf{change}/\mathsf{split} <$ `min_split_output_value`; when the minimum is unset it defaults to $(\text{existing notes total} + \mathsf{change})/(4 \cdot \mathsf{target})$, a quarter of the projected average note value (`fees.rs:404--450`). If fewer than the target number of outputs survive, the fee is recomputed for the actual count. The `SingleOutputChangeStrategy` is the degenerate `SplitPolicy::single_output()`.

### 4.4 Executing the proposal

Function `create_proposed_transactions` executes the steps in order (zcash_client_backend, `src/data_api/wallet.rs:889--991, 1110--1137`). Only *transparent* outputs of prior steps may be spent by later steps: shielded chaining is impossible, because a note's position in the global commitment tree — and hence its witness (the *Orchard Guide*, § "Proof of ownership of *some* valid note: the commitment tree") — is unknown before the note is mined. Each ephemeral output must

be consumed by a later step exactly once, else `EphemeralOutputLeftUnspent`. All transactions are created first and then stored together, so a failure cannot leave a partially transmitted multi-step plan.

**Anchors and witnesses per step.** For each pool a step involves, the anchor is the shard-tree root at the proposal's anchor-height checkpoint (`root_at_checkpoint_id`), or the empty-tree anchor if the pool has outputs but no inputs; each selected note's Merkle path is computed with `witness_at_checkpoint_id_caching` at that same height (`wallet.rs:1139--1235`). The mechanics are those of the *Orchard Guide* (§ "Proof of ownership of *some* valid note: the commitment tree") and are not repeated here.

**Builder configuration.** The zcash_primitives builder is constructed with `BuildConfig::Standard` carrying both anchors, which selects `BundleType::DEFAULT` for Sapling and Orchard alike; the Orchard builder exists only if NU5 is active at the target height (`zcash_primitives/src/transaction/builder.rs:236--283, 454--521`). The expiry height is the target plus DEFAULT_TX_EXPIRY_DELTA = 40 blocks, the post-Blossom default of ZIP 203 (`builder.rs:63--65`; ZIP 203 § "Changes for Blossom"). The rustdoc on `Builder::new` still says "20 blocks"; code and ZIP agree, the rustdoc is stale — cite the constant, not the comment. The transaction version is `TxVersion::suggested_for_branch` of the target height's branch — V5 for the NU5, NU6, and NU6.1 branches (zcash_primitives, `src/transaction/mod.rs:229--238`) — and the lock time of built transactions is 0 (`builder.rs:1089`).

**Spends, outputs, and keys.** Sapling spends are added under the external or internal (change) full viewing key according to each note's recorded ZIP-32 scope; Orchard spends are added under the account's Orchard full viewing key, the scope being derived internally (`FullViewingKey::scope_for_address` inside `SpendInfo::new`) (`wallet.rs:1268--1290`; orchard 0.12.0 `src/builder.rs`). Payment outputs are added in payment-index order, then the change outputs from the proposal balance: Sapling change to `ufvk.sapling().change_address()` (internal scope), Orchard change to `ufvk.orchard().address_at(0, Scope::Internal)`, and ephemeral ZIP-320 change values to freshly reserved ephemeral transparent addresses (`reserve_next_n_ephemeral_addresses`; the reservation persists even if construction later fails, so an address is never silently reused) (`wallet.rs:1425--1643`). The signing keys marshalled for the build (`wallet.rs:1707--1753`): transparent secret keys derived per input address; *two* Sapling expanded spending keys — `usk.sapling()` and `usk.sapling().derive_internal()` — since external and internal notes sign under different keys; but a *single* Orchard spend-authorising key (`SpendAuthorizingKey`), because Orchard internal and external addresses share ask (the *Orchard Guide*, § "Keys: capabilities for spending and for viewing"). All randomness for the build is drawn from `OsRng`.

**Remark 4.7** (OVK policy: none for change)**.** Under the default `OvkPolicy::Sender`, external outputs are encrypted with the ovk selected by `UnifiedFullViewingKey::select_ovk` over the input sources — the Orchard ovk if any Orchard input is spent, else the Sapling ovk, else the external-scope ovk derived from the transparent account key per ZIP-316 — while wallet-internal change outputs get *no* ovk at all (`internal_ovk = None`) (`wallet.rs:1400--1413`; `zcash_client_backend/src/wallet.rs:654--697`; `zcash_keys/src/keys.rs:1139--1170`). A change output's $C_{\text{out}}$ is therefore unrecoverable by *any* outgoing viewing key; the wallet finds its change through the internal-scope ivk during scanning (trial decryption per the *Orchard Guide*, § "Transmission and detection of a note…"). This matches the `OvkPolicy::Sender` rustdoc but contradicts descriptions elsewhere of change "encrypted to the internal OVK"; the `None` is deliberate. Policy `OvkPolicy::Custom` supplies both keys explicitly; policy `OvkPolicy::Discard` uses none, so the sender cannot decrypt even its own external outputs.

### 4.5 Padding, dummies, and shuffling

The deployed bundle builders are orchard 0.12.0 and sapling-crypto 0.6.0, per librustzcash's `Cargo.lock` (workspace `Cargo.toml:68,71`); all line references below are to those releases.

**Orchard.** The wallet's bundles use `BundleType::DEFAULT`: flags `ENABLED` (spends and outputs both enabled) and `bundle_required = false` (orchard 0.12.0 `src/builder.rs:58--61`, `src/bundle.rs:85--88`). With $s$ requested spends and $o$ requested outputs, the action count is

$$n_{\text{act}}(s,o) \;=\; \begin{cases} 0 & \text{if } \max(s,o) = 0, \\ \max\big(\max(s,o),\, 2\big) & \text{otherwise,} \end{cases}$$

the floor being $\mathsf{MIN\_ACTIONS} = 2$ (`src/builder.rs:36, 75--107`). The spend list is extended to $n_{\text{act}}$ with dummy spends and the output list with dummy outputs; the two indexed lists are then shuffled *independently* and zipped pairwise into Actions, so the spend–output pairing within an action is random, and `BundleMetadata` records each requested spend's and output's post-shuffle action index (`src/builder.rs`, `build_bundle`).

**Orchard dummies.** A dummy spend (protocol specification § 4.8.3, "Dummy Notes (Orchard)"; orchard 0.12.0 `src/note.rs:208--224`, `src/tree.rs:116--124`, `src/note/nullifier.rs:33--35`, `src/builder.rs:262--286`) is built from a freshly random `SpendingKey`, its full viewing key's address at index 0 (external scope), and a zero-valued note whose $\rho$ is Extract of a random Pallas point (`Rho::from_nf_old(Nullifier::dummy)`), with a dummy Merkle path of a random u32 position and 32 random $\mathbb{F}_{p_{\mathsf{Pallas}}}$ siblings. Anchor consistency is skipped for zero-valued notes (`has_matching_anchor` returns true), which is exactly what lets a dummy coexist with any real anchor. A dummy output is a zero-valued note to a freshly random address, with no ovk and an all-zero memo.

**Per-action randomness.** At `ActionInfo` construction, the value-commitment trapdoor rcv is sampled uniformly from the Pallas scalar field — one per action, padding actions included (`src/builder.rs:426--432`; `src/value.rs:300--302`). When the action is built, the spend-auth randomiser $\alpha$ is sampled per spend, giving $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ (`src/builder.rs:293--310`); the output note's $\rho$ is the action's own $\mathsf{nf}_{\text{old}}$, and its rseed is freshly sampled by `Note::new`. The cryptographic roles of rcv, $\alpha$, and rseed (with its derived $\psi$, rcm, esk) are those of the *Orchard Guide* (§ "Conservation of value…" for cv/rcv; § "Authorisation of the spend: RedPallas" for ak/$\alpha$/rk; § "Transmission and detection of a note…" for the rseed derivations).

**Sapling.** Under `BundleType::DEFAULT` in sapling-crypto 0.6.0 (`src/builder.rs:39--44, 72--115`), a non-empty bundle pads outputs to $\max(o, 2)$ but never pads spends ($num\_spends = s$ when $s > 0$, else 0; the $\max(s, 1)$ floor applies only with `bundle_required` set), so an output-only bundle has zero spend descriptions; the two-output floor (MIN_SHIELDED_OUTPUTS) is a rule the source traces to `zcash/zcash` issue 3615. An empty, non-required bundle has zero of both. Spends and outputs are extended with dummies, independently shuffled, and the positions recorded in `SaplingMetadata`. Each spend samples its own rcv and $\alpha$ (uniform in the Jubjub scalar field) and each output its own rcv; the binding-signature key is $\mathsf{bsk} = \sum \mathsf{rcv}_{\text{spend}} - \sum \mathsf{rcv}_{\text{output}}$ (`src/builder.rs:236, 262--268, 420, 752--760`).

### 4.6 Build order and signatures

**Construction 4.8** (`Builder::build`)**.** The build proceeds in a fixed order (zcash_primitives, `src/transaction/builder.rs:826--883, 959--1189`):

1. check version compatibility;
2. compute the fee over the actual padded counts (Definition 4.5) and require the balance invariant below;
3. build the Sapling bundle and create its zk-proofs (proofs before signatures, because V4 transactions commit to proofs in the digest);
4. build the Orchard bundle, unproven;
5. assemble the unauthorised transaction data and compute the digest parts (`TxIdDigester`);
6. authorise the transparent inputs;
7. compute the shared shielded sighash;
8. apply the Sapling signatures;
9. create the Orchard proof and apply the Orchard signatures;
10. freeze the result into the final `Transaction`.

**Definition 4.9** (Balance invariant)**.** Before building, the builder requires, exactly,

$$\mathsf{value\_balance} - \mathsf{fee} = 0,$$

where the value balance sums the transparent, Sapling, and Orchard balances; a negative difference is `Error::InsufficientFunds` and a positive one `Error::ChangeRequired` (`builder.rs:677--707, 1020--1035`). The proposal's change outputs are precisely what make the invariant hold.

**One sighash for everything.** The message for *all* shielded spend-auth signatures and both binding signatures is the single 32-byte ZIP-244 sighash,

$$\mathsf{sighash} := \texttt{signature\_hash}(\mathrm{tx}_{\mathrm{unauth}}, \texttt{SignableInput::Shielded}, \mathsf{txid\_parts}),$$

(`builder.rs:1133--1162`; `zcash_primitives/src/transaction/sighash.rs:15--24`); the digest tree itself is constructed in the *Orchard Guide* (§ "Conservation of value…", "The transaction digest (ZIP-244)") and not repeated. For each Orchard action, with $\mathsf{rsk} = \mathsf{ask} + \alpha$ and $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ in the notation of the *Orchard Guide* (§ "Authorisation of the spend: RedPallas"),

$$\mathsf{spendAuthSig} := \mathrm{Sign}_{\mathsf{rsk}}(\mathsf{sighash}).$$

The binding key is $\mathsf{bsk} = \left(\sum_{\text{actions}} \mathsf{rcv}\right)$ converted via `into_bsk`, and the builder asserts consistency with $\mathsf{bvk} = \sum \mathsf{cv}^{\mathsf{net}} - \mathsf{ValueCommit}(\mathsf{rcv} = 0, \mathsf{value\_balance})$ (orchard 0.12.0 `src/builder.rs`, bundle finisher). In `Bundle::prepare`, the binding signature $\mathrm{Sign}_{\mathsf{bsk}}(\mathsf{sighash})$ is created and every dummy spend is signed immediately with its stored random $\mathsf{ask}$ randomised by that action's $\alpha$; `Bundle::sign` then signs every action whose $\mathsf{ak}$ matches the given authorising key, and `finalize` fails with `MissingSignatures` if any action remains unsigned (`src/builder.rs:984--1026`).

**Remark 4.10** (The proving key is rebuilt every time)**.** The Orchard proving key is reconstructed from scratch inside every `Builder::build` call — `orchard::circuit::ProvingKey::build()` runs Halo 2 key generation on the fly — and the deployed pipeline does not cache it (`builder.rs:1149--1153`; orchard 0.12.0 `src/circuit.rs:787--793`). The Sapling provers, by contrast, are passed in as arguments, typically a `LocalTxProver`.

### 4.7 After the build

The wallet immediately decrypts its own shielded outputs to record sent-note data: Orchard outputs via `decrypt_output_with_key` under the internal-scope ivk, Sapling via `try_sapling_note_decryption` likewise, using `BundleMetadata` and `SaplingMetadata` to map each requested output to its post-shuffle position (`wallet.rs:1755--1853`). Transparent outputs are *not* reordered — the source notes "there is no reason to do so because it would not usefully improve privacy" — so `vout` index $n$ is the $n$-th added transparent output. The fee recorded for each sent transaction is taken from the proposal's balance, not recomputed from the built transaction (`wallet.rs:1855--1861`).

**The PCZT path.** A partially-constructed-transaction path parallels the signing path: `build_for_pczt` produces the `PcztParts`, and the Creator, IO-Finalizer, and Updater roles attach ZIP-32 derivation paths (Orchard and Sapling $m/32'/coin'/account'$; transparent BIP-44 $m/44'/coin'/account'/scope/index$) and the proposal metadata under the proprietary fields `zcash_client_backend:proposal_info` and `:output_info`; only single-step proposals are supported (`wallet.rs:124--127, 1864--2140`; `zcash_primitives/src/transaction/builder.rs:314--336, 1192+`).

**Remark 4.11** (Deployed versus in-flight builders)**.** This section states the semantics of orchard 0.12.0 as resolved by librustzcash's `Cargo.lock`. The development head of the orchard repository already carries NU6.3-era builder changes — cross-address-transfer flags, an explicit `add_change_output`, and per-version proving keys — none of which exist in 0.12.0; they are design-stage and out of scope for the deployed layer documented here.

## 5 Partially created transactions (PCZT)

The *Orchard Guide* constructs a transaction as if one machine held everything at once: the notes and their authentication paths, the full viewing key, the spend authorizing key, the proving key, and a source of randomness. Deployed wallets do not enjoy that luxury. A hardware signer holds the spending key but cannot compute a Halo 2 proof and may not fit a whole transaction in memory; a threshold of co-signers must each contribute a share; a proof may be delegated to a machine that must never see spending authority. ZIP-374 resolves this by standardising the intermediate object those parties exchange: the *Partially Created Zcash Transaction* (PCZT), a format that carries the information each participant in transaction creation needs to perform its step, and that holds the accumulated proofs and signatures while the set is still incomplete — signers can be fully offline (ZIP-374, Abstract). The ZIP is Status Draft, Revision 0, owned by Jack Grigg, created 2024-12-09 (ZIP-374, header); throughout this section we document the *deployed* v1 surface — the pczt crate, version 0.5.1, in the librustzcash workspace — and classify the v2 material of the draft as *specified, not deployed* (§5.13).

The Motivation section of the ZIP names four problems the format solves: (i) offline and hardware signers that cannot compute Halo 2 or Groth16 proofs and may not hold a whole transaction in memory; (ii) proof delegation, where the prover needs the note's private data and the full viewing key but no spending key material; (iii) multiparty transactions — FROST threshold signing (RFC 9591), transparent multisig (ZIP-48), and inputs contributed by several parties, merged deterministically; and (iv) pre-authorization and deferred proving for v6 transactions, where anchors become authorizing data (ZIP-229) (ZIP-374, Motivation).

The design deliberately reuses the role structure of Bitcoin's PSBT (BIP 174 / BIP 370) with Zcash-specific additions. PSBT itself cannot be used: it has no representation for shielded inputs and outputs, no notion of proofs as distinct from signatures, and its key–value encoding tolerates unknown fields — undesirable for signers that must understand everything they authorize (ZIP-374, Motivation; pczt, `README.md`). The ZIP's Requirements section fixes the resulting contract:

the format must not require spending key material to be present (dummy-spend keys are the one exception, and must be removable before Signers see the PCZT); a signer must be able to determine from the PCZT alone exactly what it authorizes; merging must be deterministic and fail loudly on conflict; and the encoding must be compact enough for memory-constrained signing devices and strictly versioned (ZIP-374, Requirements).

Two scope restrictions follow from history. PCZTs do not support Sprout and cannot create v4-or-earlier transactions: the v4 format predates the ZIP-244 transaction digest, and the role separations — in particular proving independent of signing — do not hold for it (ZIP-374, Specification). Two PCZT versions are specified: v1 supports creation of v5 transactions (ZIP-225); v2 supports v5 and v6 (ZIP-229). A PCZT version does not pin a transaction version — a v2 PCZT may carry a v5 or a v6 transaction, a v1 PCZT only v5 (ZIP-374, Versioning). The deployed crate implements only v1: `Pczt::parse` rejects any version number other than 1 with `ParseError::UnknownVersion` (`pczt`, `src/lib.rs`).

## 5.1 Roles and lifecycle

**Definition 5.1** (ZIP-374 roles)**.** Transaction creation is divided among the following roles, each entitled to perform a specific step on a PCZT (ZIP-374, Roles; `pczt`, `src/roles.rs`):

**Creator (a single entity).** Initializes the global fields and the empty per-protocol bundles.

**Constructor.** Adds transparent inputs and outputs, shielded spends and outputs, and Orchard actions.

**IO Finalizer (anyone).** Declares the input/output set complete, computes the binding-signature keys, and signs dummy spends (§5.5).

**Updater (anyone).** Attaches metadata later roles need: derivation paths, full viewing keys, witnesses, anchors.

**Prover.** Attaches zero-knowledge proofs (§5.8).

**Signer.** Attaches signatures (§5.7).

**Combiner.** Merges parallel copies of a PCZT (§5.9).

**Redactor.** Removes fields that later entities do not need.

**Spend Finalizer.** Assembles the transparent `script_sig`s (§5.10).

**Transaction Extractor.** Produces and verifies the final transaction (§5.11).

A *Verifier* is "not a step in the transaction lifecycle, but a way of inspecting a PCZT" and checking its internal consistency on behalf of another entity that lacks the capacity to do so itself — for example a hardware signer (ZIP-374, Roles). The checks such a verifier runs are the per-action verification equations of Construction 5.8, exposed on the protocol crates' types rather than in the `pczt` crate's `verifier` module.

The lifecycle is not a fixed pipeline. Proofs and signatures slot in asynchronously: proof creation needs no spending key material, and for v5 transactions signatures commit to the anchor (through the sighash) but not to proofs, while proofs do not depend on signatures — so the Prover and Signer roles can run in either order or in parallel, their outputs merged by a Combiner (ZIP-374, Roles and Prover; `zcash_client_backend`, `src/data_api/wallet.rs`, `create_pczt_from_proposal` doc). From the v6 format onward the two roles become independent even of the anchor choice (§5.13).

## 5.2 Structure

**Definition 5.2** (Field kinds)**.** Every structure in a PCZT contains three kinds of fields (ZIP-374, Specification: Structure):

- *transaction effecting data*: required, part of the final transaction, always present;
- *transaction authorizing data*: initially empty, filled in as the PCZT is processed (proofs, signatures);
- *context data*: committed to by, or relevant to, the effecting data; present so that roles can be performed (openings, keys, paths).

The deployed `Pczt` struct is {`global`, `transparent`, `sapling`, `orchard`} with all three bundles non-optional (`pczt`, `src/lib.rs`). The bundles are not logically optional because a PCZT does not always contain a semantically valid transaction, and there may be phases where protocol-specific metadata must be stored before it is known whether that protocol has any inputs or outputs (ZIP-374, Structure; `pczt`, `src/lib.rs`, comment on `Pczt`).

**Global fields.** The `global` structure carries (ZIP-374, Global; `pczt`, `src/common.rs`): `tx_version` (`u32`; MUST be 5 in PCZT v1) and `version_group_id` (`u32`; must be consistent with `tx_version`); `consensus_branch_id` (`u32`; non-optional because it commits to the consensus rules that will apply to the transaction); `fallback_lock_time` (`Option<u32>`; assumed 0 if omitted); `expiry_height` (`u32`; ZIP-203); `coin_type` (`u32`, SLIP 44 — not part of the transaction and without consensus effect, present so later roles can check network-specific data such as derivation paths); `tx_modifiable` (a `u8` bitfield, next paragraph); and `proprietary`, a map from UTF-8 strings to byte strings.

**Definition 5.3** (The `tx_modifiable` bitfield)**.** Field `tx_modifiable` tracks which parts of the transaction are still open to change (`pczt`, `src/common.rs`; ZIP-374, Global):

| Bit | Mask | Meaning | Merge |
|---|---|---|---|
| 0 | `0b0000_0001` | Transparent Inputs Modifiable | AND |
| 1 | `0b0000_0010` | Transparent Outputs Modifiable | AND |
| 2 | `0b0000_0100` | Has `SIGHASH_SINGLE` | OR |
| 3–6 | — | reserved; must be zero (merge fails otherwise) | — |
| 7 | `0b1000_0000` | Shielded Modifiable | AND |

The Creator sets bits 0, 1, and 7 and clears bit 2; a Constructor checks the relevant bit before adding an input or output and may clear it; the IO Finalizer clears bits 0, 1, and 7 if there are shielded spends or outputs, and otherwise leaves the bitfield unchanged (§5.5). A Signer clears bit 0 when adding any signature that does not use `SIGHASH_ANYONECANPAY` — which includes *all* shielded signatures — and clears bit 1 for any signature not using `SIGHASH_NONE`; a `SIGHASH_SINGLE` signature also clears bit 1, because removing the paired output would not remove the signature. Bit 2 is set by any Signer using `SIGHASH_SINGLE` and tells Constructors to preserve the input/output pairing. Every Signer clears bit 7 (`pczt`, `src/common.rs`, `Global` doc, `merge`, and tests; ZIP-374, Global).

The split between transparent and shielded modifiable flags is deliberate: Zcash's sighash pins *all* non-transparent effecting data as soon as any input is signed, so separate flags keep purely-transparent PCZTs semantically close to PSBTs, where Constructor and Signer roles can interleave when inputs are not `SIGHASH_ALL` (ZIP-374, Rationale). The deep construction of the sighash itself is the ZIP-244 digest covered by the *Orchard Guide* (the transaction digest); this section only ever cites it.

**Transparent inputs and outputs.** A `TransparentInput` carries the outpoint (`prevout_txid` [u8; 32], `prevout_index u32`), `sequence` (`Option<u32>`, default `0xFFFFFFFF`), the lock-time constraints `required_time_lock_time` ( $\geq$ 500000000) and `required_height_lock_time` ( $>$ 0, $<$ 500000000), the `script_sig` (set by the Spend Finalizer), `value` (`u64`) and `script_pubkey` — both required, because the Transaction Extractor needs every input's UTXO to derive the shielded sighash for the binding signatures — `redeem_script` (P2SH only), `partial_signatures` (a map from 33-byte public keys to signatures), `sighash_type` (Signers MUST use it; Spend Finalizers MUST fail on signatures that mismatch it), `bip32_derivation`, four preimage maps (RIPEMD160/SHA-256/HASH160/HASH256), and `proprietary` (ZIP-374, TransparentInput; `pczt`, `src/transparent.rs`). A `TransparentOutput` carries `value`, `script_pubkey`, `redeem_script`, `bip32_derivation`, `user_address` (`Option<UTF8String>`, set by an Updater; Signers MUST parse it and confirm that it contains the recipient, directly or as a receiver within a Unified Address), and `proprietary` (ZIP-374, TransparentOutput). Transparent partial signatures are stored as DER-encoded ECDSA with the sighash-type byte appended, $sig\_bytes = \mathrm{DER}(sig) \,\|\, sighash\_type$, matching the Bitcoin/PSBT convention (zcash_transparent, `src/pczt/signer.rs`).

**The Sapling bundle.** A `SaplingBundle` carries lists of spends and outputs, a `value_sum` (`i128`, net spends minus outputs — wide enough that per-spend and per-output values can be redacted while the net remains), the `anchor` ([u8; 32]; in PCZT v1 set by the Creator and immutable, because the v5 sighash commits to it), and `bsk` (`Option`; `None` until the IO Finalizer runs, used by the Transaction Extractor for the binding signature). Each `SaplingSpend` requires `cv`, `nullifier`, and `rk` (effecting), and optionally carries a per-spend 192-byte Groth16 `zkproof` (Prover), a 64-byte `spend_auth_sig` (Signer), `recipient` (43 bytes), `value`, `rcm` *or* `rseed` (`rcm` only for pre-ZIP-212 notes; the Prover needs one of them), `rcv` (the IO Finalizer compresses these into `bsk`; the Prover requires it; it opens `cv`, hence is redactable), `proof_generation_key` (ak, nsk) (Updater sets, Prover requires), `witness` (a `u32` position and a 32-level path), `alpha`, `zip32_derivation`, `dummy_ask` (a Constructor chooses it; the IO Finalizer uses and clears it; Signers MUST reject PCZTs containing it), and `proprietary`. Each `SaplingOutput` requires `cv`, `cmu`, `ephemeral_key`, `enc_ciphertext`, and `out_ciphertext` (whose length depends on the transaction version), and optionally carries `zkproof`, `recipient`, `value`, `rseed` (the Prover requires it, in preference to disclosing the shared secret), `rcv`, `ock` (lets a Signer verify `out_ciphertext`; `None` under the OVK policy `None`), `zip32_derivation`, `user_address`, and `proprietary` (ZIP-374, SaplingBundle/SaplingSpend/SaplingOutput; `pczt`, `src/sapling.rs`).

**The Orchard bundle.** An `OrchardBundle` carries (`pczt`, `src/orchard.rs`; ZIP-374, OrchardBundle/OrchardAction):

**`actions`.** A list of `OrchardAction` structures.

**`flags (u8)`.** Bit 0 `enableSpends`, bit 1 `enableOutputs`; the remaining bits are reserved zeros in PCZT v1.

**`value_sum ((u64, bool))`.** The net value of spends minus outputs. The ZIP never states the boolean's meaning; only the reference implementation defines it (true = negative, via the orchard crate's sign mapping) — a spec gap we flag in §5.14.

**`anchor ([u8;32])`.** Set by the Creator; fixed for the life of a v1 PCZT.

**`zkproof (Option<Vec<u8>>)`.** A *single* Halo 2 proof for the whole bundle, set by the Prover — unlike Sapling's per-spend and per-output proofs.

**`bsk (Option<[u8;32]>)`.** The binding signing key, set by the IO Finalizer.

Each `OrchardAction` is $\{\mathsf{cv}^{\mathsf{net}}, \text{spend}, \text{output}, \mathsf{rcv}\}$: the net value commitment, a spend half, an output half, and *one* optional $\mathsf{rcv}$ per action, since an action carries a single net value commitment (the *Orchard Guide*, conservation of value).

An `OrchardSpend` carries: `nullifier` and `rk` (32 bytes each; effecting, required); `spend_auth_sig` (64 bytes; Signer); `recipient` (43 bytes, the raw Orchard payment address encoding; a Constructor sets it, the Prover requires it); `value` (the Prover requires it; a Signer may verify it against `cv_net` and confirm change amounts; redactable); `rho` and `rseed` (32 bytes each); `fvk` (96 bytes, $\mathsf{ak} \,\|\, \mathsf{nk} \,\|\, \mathsf{r_{ivk}}$ — the full viewing key of the note's recipient; an Updater sets it, the Prover requires it); `witness` (a `u32` position plus a 32-level, 1024-byte authentication path; Updater sets, Prover requires); `alpha` (32 bytes; a Constructor chooses it, the Signer requires it; redactable once `zkproof` and `spend_auth_sig` are set); `zip32_derivation`; `dummy_sk` (a Constructor chooses it for dummy notes; the IO Finalizer requires and clears it; Signers MUST reject PCZTs containing it); and `proprietary`. An `OrchardOutput` carries the effecting fields `cmx`, `ephemeral_key`, `enc_ciphertext`, and `out_ciphertext` (all required), plus `recipient`, `value`, `rseed` (the Prover requires it instead of disclosing the shared secret), `ock`, `zip32_derivation`, `user_address` (Signers MUST parse it and confirm that it contains the recipient), and `proprietary` (`pczt`, `src/orchard.rs`; ZIP-374, OrchardSpend/OrchardOutput). The witness, anchor, and the note-commitment machinery these fields feed are exactly those of the *Orchard Guide* (the commitment tree and authentication paths); we do not re-derive them here.

**ZIP-32 derivations.** A `Zip32Derivation` pairs a *seed_fingerprint* ([`u8`; 32], the ZIP-32 seed fingerprint) with a *derivation_path* (`List<u32>`), where each index may carry the hardened flag, bit 31 (*index* | `0x80000000`). Shielded spend paths are generally fully hardened; transparent paths generally include a non-hardened section matching BIP 44 or the ZIP-320 ephemeral-address path (`pczt`, `src/common.rs`). The `orchard` crate rejects any non-hardened component at parse time — Orchard does not support non-hardened derivation — and its method `extract_account_index` recognizes exactly the pattern $[32', \textit{coin\_type}', \textit{account}']$ under a matching seed fingerprint (`orchard`, `src/pczt.rs` and `src/pczt/parse.rs`).

## 5.3 Encoding

**Construction 5.4** (PCZT encoding)**.** A PCZT is encoded as

$$\underbrace{[\texttt{0x50}, \texttt{0x43}, \texttt{0x5A}, \texttt{0x54}]}_{\text{ASCII PCZT}} \,\|\, \text{I2LEOSP}_{32}(\textit{version}) \,\|\, \text{version-specific body},$$

an 8-byte header followed by the body. A parser rejects inputs shorter than 8 bytes (`TooShort`), a wrong magic (`NotPczt`), and — in the deployed crate — any version other than 1 (`UnknownVersion`); a parser encountering an unknown version MUST NOT parse the remainder. Files use the `.pczt` extension (ZIP-374, Encoding; `pczt`, `src/lib.rs`, `MAGIC_BYTES`, `parse`/`serialize`).

The body is a profile of the Postcard wire format — byte-for-byte the serde serialization of the `pczt` Rust crate, with the ZIP text canonical (ZIP-374, Encoding). Its primitives:

- varint($n$): emit $n$ seven bits at a time, least-significant group first, one byte per group; every byte except the last has bit 7 set; encoders MUST emit the shortest encoding. A $k$-bit integer occupies at most $\lceil k/7 \rceil$ bytes, and in a maximum-length encoding the final byte contributes only the high ($k \bmod 7$) bits — the fifth byte of a `u32` is at most `0x0F`; decoders MUST reject encodings exceeding either limit.
- Signed `i128` (the Sapling `value_sum`): encode($n$) = varint(zigzag($n$)) with zigzag($n$) = $(n \ll 1) \oplus (n \gg 127)$, the right shift arithmetic and the result reinterpreted as unsigned; a non-negative $n$ encodes as $2n$, a negative $n$ as $2|n| - 1$; at most 19 bytes.

- `Option`: a tag byte `0x00` (`None`) or `0x01` followed by the value.
- `List`: varint(*count*) then the elements.
- `Map`: varint(*count*) then the entries in strictly ascending lexicographic key order, duplicates forbidden.
- Structs: field concatenation in the ZIP's field order, with no framing or tags.

The v1 body is the concatenation

$$\texttt{Global} \,\|\, \texttt{TransparentBundle} \,\|\, \texttt{SaplingBundle} \,\|\, \texttt{OrchardBundle},$$

with the transparent bundle a pair of input and output lists, the Sapling bundle {spends, outputs, `value_sum`, `anchor`, `bsk`}, and the Orchard bundle {actions, `flags`, `value_sum`, `anchor`, `zkproof`, `bsk`} (ZIP-374, Encoding and Data type encodings).

Extensibility is deliberately narrow. New fields cannot be added within an existing PCZT version, because the Postcard encoding is positional; unlike PSBT there are no non-proprietary arbitrary key–value maps, so a Signer's view of a PCZT is complete. The `proprietary` maps are the only within-version extension point and are reserved for private use; format evolution happens only through new versions namespaced by the header's 4-byte version (ZIP-374, Extensibility and Proprietary Use fields). Serialization is deterministic: the crate's round-trip test checks serialize(parse(serialize($p$))) = serialize($p$) (pczt, `tests/end_to_end.rs`, `check_round_trip`).

### 5.4 Creation and construction

The deployed Creator, `Creator::new(consensus_branch_id, expiry_height, coin_type, sapling_anchor, orchard_anchor)`, defaults to the v5 transaction format — `tx_version` = 5 (`V5_TX_VERSION`), `version_group_id` = `0x26A7270A` (`V5_VERSION_GROUP_ID`); zcash_protocol, `src/constants.rs` — with initial `tx_modifiable` = `0b1000_0011` (bits 0, 1, 7 set), Orchard `flags` = `0b0000_0011` (spends and outputs enabled), and empty bundles with Sapling `value_sum` = 0 and Orchard `value_sum` = (0, true); builder options `with_fallback_lock_time` and `with_orchard_flags` adjust the defaults (pczt, `src/roles/creator/mod.rs`). The Orchard initialization is "negative zero" — harmless in v1 but an encoding nit against the v2 draft (§5.14).

The crate ships no Constructor implementation ("The roles currently without an implementation are: Constructor"; pczt, `src/roles.rs`). In the deployed stack the Constructor step is performed by the zcash_primitives transaction builder: `Builder::build_for_pczt(rng, fee_rule)` checks that the value balance minus the fee is exactly zero (failing with `InsufficientFunds` or `ChangeRequired` otherwise), then builds the per-pool PCZT bundles *without proving or signing*, returning a `PcztResult` whose `PcztParts` carry the parameters, version, branch id, `lock_time` = 0, expiry height, and the three bundles, together with the Sapling and Orchard bundle metadata (zcash_primitives, `src/transaction/builder.rs`, `build_for_pczt`, `PcztParts`). Then `Creator::build_from_parts(parts)` assembles the `Pczt`, setting `tx_modifiable` = `0b0000_0000` — nothing modifiable, plus the Has-`SIGHASH_SINGLE` bit if any input uses it — `fallback_lock_time` = `Some`(*lock_time*), and `coin_type` from the network parameters (pczt, `src/roles/creator/mod.rs`, `build_from_parts`).

The Orchard side of that construction produces, per action (orchard, `src/builder.rs`, `SpendInfo::into_pczt`, `OutputInfo::into_pczt`, `build_for_pczt`):

$$\mathsf{cv}^{\mathsf{net}} := \mathrm{ValueCommit}^{\mathsf{Orchard}}(v^{\mathsf{net}}, \mathsf{rcv}), \qquad \mathsf{nf}_{\mathrm{old}} := \text{the spent note's nullifier under } \mathsf{fvk},$$

a fresh uniformly random $\alpha$, and $\mathsf{rk} := \mathsf{Randomize}(\mathsf{ak}, \alpha)$ — the RedPallas re-randomization of the *Orchard Guide* (spend authorization) — populating the spend's `recipient`, `value`, `rho`, `rseed`,

`fvk`, `witness`, and `alpha` (all `Some`), plus `dummy_sk` for dummy spends. The output half builds the new note with $\rho := \mathsf{nf}_{\text{old}}$ of the paired spend, computes $\mathsf{cmx}$, and runs the note encryption of the *Orchard Guide*, populating `recipient`, `value`, and `rseed`. It currently sets `ock = None` with an in-code TODO ("Extract ock from the encryptor …"), so the deployed Constructor never fills `ock` even when an outgoing viewing key is used — a divergence from the ZIP's field description (§5.14).

Bundles are padded with dummy actions and their positions randomized, so a consumer must map logical spends and outputs to bundle positions through the builders' metadata: `spend_action_index(`$n$`)` and `output_action_index(`$n$`)` on the Orchard `BundleMetadata`, and `SaplingMetadata` for Sapling (orchard, `src/builder.rs`). The wallet backend uses exactly this to attach per-output metadata to the right PCZT actions (zcash_client_backend, `src/data_api/wallet.rs`, `create_pczt_from_proposal`).

## 5.5 IO finalization: the binding-signature key

Once the input/output set is complete, the IO Finalizer freezes it. The deployed role fails with `NoSpends`/`NoOutputs` if the transaction would have no spends or no outputs; if there are shielded spends or outputs it clears `tx_modifiable` bits 0, 1, and 7. It computes the ZIP-244 *shielded sighash*

$$\mathit{sighash} := \text{v5\_signature\_hash}(\mathit{tx\_data},\ \texttt{SignableInput::Shielded},\ \mathit{txid\_parts}),$$

where *tx_data* is the *effects-only* transaction data reconstructed from the PCZT — proofs and signatures do not affect the sighash — and *txid_parts* = *tx_data*.digest(`TxIdDigester`); only $(\texttt{tx\_version}, \texttt{version\_group\_id}) = (5, \texttt{0x26A7270A})$ is supported. The digest algorithm itself is the ZIP-244 construction of the *Orchard Guide* (the transaction digest). It then calls the per-pool `finalize_io(sighash, OsRng)` (pczt, `src/roles/io_finalizer/mod.rs`).

**Construction 5.5** (Orchard IO finalization)**.** Every action's $\mathsf{rcv}$ must be present. The role computes

$$\mathsf{bsk} := \sum_{a \in \text{actions}} \mathsf{rcv}_a \ \in \mathbb{F}_{p_{\mathsf{Vesta}}},$$

converted through its 32-byte little-endian representation into a RedPallas binding signing key (`ValueCommitTrapdoor::into_bsk`), and recomputes

$$\mathsf{bvk} := \Big(\sum_a \mathsf{cv}_a^{\mathsf{net}}\Big) - \text{ValueCommit}^{\mathsf{Orchard}}(\mathit{value\_sum}, 0),$$

where $\text{ValueCommit}^{\mathsf{Orchard}}(v, r) := [v]\,V^{\mathsf{Orchard}} + [r]\,R^{\mathsf{Orchard}}$ with

$$V^{\mathsf{Orchard}} = \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-cv}, \texttt{v}),$$
$$R^{\mathsf{Orchard}} = \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-cv}, \texttt{r})$$

— the net value commitment of the *Orchard Guide* (conservation of value). It fails with `ValueCommitMismatch` unless $\text{VerificationKey}(\mathsf{bsk}) = \mathsf{bvk}$: this is the ZIP's requirement that `value_sum` be consistent with the value commitments and trapdoors being aggregated. Finally, for each action with `dummy_sk` present, it *takes* the key (`Option::take`, guaranteeing that no spend authorizing key material remains after this role), derives $\mathsf{ask} = \text{SpendAuthorizingKey}(\mathsf{sk})$, and signs the action with the shielded sighash (orchard, `src/pczt/io_finalizer.rs`; ZIP-374, IO Finalizer).

The Sapling analogue is $\mathsf{bsk} := \sum_{\text{spends}} \mathsf{rcv} - \sum_{\text{outputs}} \mathsf{rcv}$ as a Jubjub scalar, made into a RedJubjub binding signing key, checked against $\mathsf{bvk} = (\sum_{\text{spends}} \mathsf{cv} - \sum_{\text{outputs}} \mathsf{cv})$ adjusted by `value_sum` as an `i64` (`InvalidValueSum` if it does not fit); dummy spends are then signed with their `dummy_ask`, which is taken and cleared (sapling-crypto, `src/pczt/io_finalizer.rs`; the workspace pins version 0.6, same algorithm).

**Remark 5.6** (Why a dedicated role)**.** The key bsk cannot be computed by the Transaction Extractor without preserving every rcv until extraction — revealing the opening of every value commitment to all intermediate participants — and cannot be maintained incrementally by Constructors, because with parallel construction and a Combiner the incremental update would be wrong. A dedicated role that runs once, after IO completion, resolves both. The same role signs dummy spends so that no Signer ever parses spending key material (ZIP-374, Rationale, "Adding an IO Finalizer role").

## 5.6 Updating

The deployed Updater wraps the protocol crates' updater types. Method `update_global_with` sets proprietary fields; the wrapped Orchard `ActionUpdater` exposes, per action, the setters `set_spend_zip32_derivation` and `set_spend_proprietary` on the spend half, and on the output half the setters `set_output_zip32_derivation`, `set_output_user_address`, and `set_output_proprietary` (with Sapling and transparent analogues). In PCZT v1 anchors are fixed at creation, so the deployed Updater cannot set anchors or witnesses — those come from the Constructor (pczt, `src/roles/updater/mod.rs` and `orchard.rs`; orchard, `src/pczt/updater.rs`).

## 5.7 Signing

**Construction 5.7** (The Signer role)**.** The deployed Signer (pczt feature `signer`), `Signer::new`, parses all bundles, extracts an effects-only `TransactionData`, and computes *txid_parts* and the shielded sighash of §5.5 *once*, cached across signatures. Its methods are `shielded_sighash()` and `transparent_sighash(index)` — the latter

$$\text{v5\_signature\_hash}(\textit{tx\_data},\ \texttt{SignableInput::Transparent}(\text{input } i),\ \textit{txid\_parts})$$

under the input's declared `sighash_type` — the signing methods `sign_transparent`, `sign_sapling`, and `sign_orchard`, their `append_`/`apply_` variants for externally produced signatures (a hardware device), and `finish()`. To sign an Orchard spend, `alpha` must be present; with the wallet's ask:

$$\mathsf{rsk} := \mathsf{ask}+\alpha \quad \text{(the RedPallas SpendAuth randomization)}, \qquad \text{require VerificationKey}(\mathsf{rsk}) = \mathsf{rk}$$

— failing with `WrongSpendAuthorizingKey` otherwise — then

$$\texttt{spend\_auth\_sig} := \text{RedPallas.Sign}(\mathsf{rsk},\ \textit{sighash}).$$

The external path `apply_orchard_signature` accepts a signature iff it verifies under the action's rk over the shielded sighash (`InvalidExternalSignature` otherwise). After any shielded signature all three modifiable bits are cleared; after a transparent signature the bits are updated per its sighash type as Definition 5.3 specifies (pczt, `src/roles/signer/mod.rs`; orchard, `src/pczt/signer.rs`).

All shielded signatures behave as committing to the whole transaction — there is no shielded `SIGHASH_ANYONECANPAY` — which is why every Signer clears every modifiable bit (ZIP-374, Global, `tx_modifiable`). Before signing an Orchard spend, the deployed Signer calls `verify_nullifier` (Construction 5.8) and treats `Missing{Recipient,Value,Rho,RandomSeed}` as acceptable — those fields may legitimately have been redacted for this signer — but hard-fails on any actual inconsistency; likewise for Sapling (pczt, `src/roles/signer/mod.rs`, `generate_or_apply_orchard_signature`).

**Construction 5.8** (Per-action verification equations)**.** The orchard crate exposes, for Signers and Verifiers (orchard, `src/pczt/verify.rs`):

$$\begin{aligned}
\texttt{verify_cv_net}: &\quad \mathsf{cv}^{\mathsf{net}} \stackrel{?}{=} \mathrm{ValueCommit}^{\mathsf{Orchard}}(v_{\mathrm{spend}} - v_{\mathrm{output}},\ \mathsf{rcv});\\
\texttt{verify_nullifier}: &\quad \text{rebuild the note from } (\text{recipient}, v, \rho, \mathsf{rseed}),\\
&\quad \text{require } \mathsf{fvk} \text{ owns it } (\texttt{fvk.scope_for_address} \text{ is } \texttt{Some}),\\
&\quad \text{and } \mathsf{nf} \stackrel{?}{=} \text{the note's nullifier under } \mathsf{fvk};\\
\texttt{verify_rk}: &\quad \mathsf{ak} \text{ from } \mathsf{fvk}, \quad \mathsf{rk} \stackrel{?}{=} \mathsf{Randomize}(\mathsf{ak}, \alpha);\\
\texttt{verify_note_commitment}: &\quad \text{rebuild the output note with } \rho := \mathsf{nf}_{\mathrm{old}} \text{ of the paired spend},\\
&\quad \mathsf{cmx} \stackrel{?}{=} \mathsf{Extract}\big(\mathsf{NoteCommit}_{\mathsf{rcm}}(\ldots)\big),
\end{aligned}$$

with the nullifier, randomization, and commitment constructions those of the *Orchard Guide*. For dummy notes ($v = 0$) an `expected_fvk` argument is ignored in favour of the spend's own `fvk` field.

A signer implementation "should only need to implement": Pedersen commitments using Jubjub/Pallas arithmetic (note and value commitments); BLAKE2b and BLAKE2s and the PRFs/CRHs built on them; the nullifier check; the KDF plus note decryption (AEAD_CHACHA20_POLY1305); the SignatureHash algorithm; RedJubjub/RedPallas signatures; and a source of randomness — no proving circuitry (pczt, `src/roles/signer/mod.rs`, module doc). For dependency-constrained environments a `low_level_signer::Signer` variant (not feature-gated) exposes `sign_orchard_with`/`sign_sapling_with`/`sign_transparent_with` closures over the parsed bundles plus `tx_modifiable` (pczt, `src/roles/low_level_signer/mod.rs`).

## 5.8 Proving

The Prover attaches proofs; per the ZIP it requires, for each Orchard action, the spend's `recipient`, `value`, `rho`, `rseed`, `fvk`, `witness`, and `alpha`, plus the action's `rcv` and the bundle anchor; delegating proving therefore reveals note private data, but not spending authority (ZIP-374, Prover). The deployed role (pczt feature `prover`) offers `requires_sapling_proofs` (any Sapling spend or output missing its `zkproof`) and `requires_orchard_proof` (actions non-empty and the bundle `zkproof None`), letting a wallet skip downloading the Sapling parameters when they are unneeded; `create_orchard_proof(pk)` parses the bundle and calls the orchard crate's `Bundle::create_proof` (pczt, `src/roles/prover/mod.rs` and `orchard.rs`).

**Construction 5.9** (Orchard proof creation from a PCZT)**.** With no actions the call is a no-op. Otherwise, for each action $i$ the prover rebuilds the spend information ($\mathsf{fvk}_i$, note$_i$, merkle path$_i$) — failing with `WrongFvkForNote` if $\mathsf{fvk}_i$ does not own the note — and the output note with $\rho_i := \mathsf{nf}_{\mathrm{old},i}$ (`RhoMismatch` if inconsistent), then forms

$$\begin{aligned}
\mathit{circuit}_i &:= \mathrm{Circuit}\big(\text{spend}_i,\ \text{output note}_i,\ \alpha_i,\ \mathsf{rcv}_i\big),\\
\mathit{instance}_i &:= \big(\text{anchor},\ \mathsf{cv}_i^{\mathsf{net}},\ \mathsf{nf}_i,\ \mathsf{rk}_i,\ \mathsf{cmx}_i,\ \text{flags}\big),
\end{aligned}$$

rejecting the identity rk (`IdentityRk`; forbidden by the consensus rule introduced in zcashd v6.12.1 / Zebra 4.3.1), and creates a *single* Halo 2 proof over all actions, stored as the bundle's `zkproof`. The proof commits to the anchor and the flags as public inputs; the statement proved is the Action relation of the *Orchard Guide* (orchard, `src/pczt/prover.rs`).

Because v5 signatures commit to the anchor but not to proofs, and proofs do not depend on signatures, the Prover and Signer can run in either order or in parallel; the zcash_client_backend documentation says to use the Combiner "if you create proofs and apply signatures in parallel" (ZIP-374, Prover; zcash_client_backend, `src/data_api/wallet.rs`).

## 5.9 Combining, redaction, and privacy

**Construction 5.10** (Merge algebra)**.** The deployed Combiner — `Combiner::new(Vec<Pczt>)` followed by `combine()` — left-folds pairwise merges; per-protocol bundles are merged before `Global`, "because each is only interpretable in the context of its own global". The rules (`pczt`, `src/roles/combiner/mod.rs`, `src/common.rs`, `src/orchard.rs`):

- Global effecting data (`tx_version`, `version_group_id`, `consensus_branch_id`, `fallback_lock_time`, `expiry_height`, `coin_type`) must be equal; `tx_modifiable` merges bit-by-bit per Definition 5.3.
- Optional fields: merge$(\ell, \texttt{None}) = \ell$; merge$(\texttt{None}, r) = r$; merge$(\texttt{Some}\,\ell, \texttt{Some}\,r) = \ell$ iff $\ell = r$, else FAIL. Maps merge by key union, with values required equal on collision.
- Bundle structure: flags must be equal; if either side's `bsk` is set (the IO Finalizer has run), action counts and `value_sum` must match; otherwise a strict prefix extension appends the extra actions *only if* the shorter side's global has Shielded Modifiable set — else FAIL. Anchors must be equal, and the per-action required fields ($\mathsf{cv}^{\mathsf{net}}$, `nullifier`, `rk`, `cmx`, `ephemeral_key`, `enc_ciphertext`, `out_ciphertext`) must be equal.

Combining must be functionally order-independent:

$$\mathrm{Combine}\big(f_A(p),\ f_B(p)\big) = f_A\big(f_B(p)\big) = f_B\big(f_A(p)\big).$$

The equality-or-fail rule intentionally differs from BIP 174, which on conflicting values says the combiner "must choose the value it considers correct" — in practice last-wins — silently discarding data. Because every PCZT field is typed and known up front, requiring equality is always possible and turns every conflict into a loud failure. A Combiner MUST NOT combine PCZTs that do not represent the same transaction; once IO is finalized, a PCZT is uniquely identified by the txid its effecting data determines (ZIP-374, Combiner and Rationale; `pczt`, `src/roles/combiner/mod.rs`, `merge_optional` comment).

**Redaction.** The Redactor provides per-bundle, per-field `clear` methods. The Orchard `ActionRedactor` can clear `spend_auth_sig`; the spend's `recipient`, `value`, `rho`, `rseed`, `fvk`, `witness`, `alpha`, `zip32_derivation`, and `dummy_sk`; the spend's proprietary entries (by key or all); the output's `recipient`, `value`, `rseed`, `ock`, `zip32_derivation`, and `user_address`; the output's proprietary entries; and `rcv`. The bundle-level redactor can clear `zkproof` and `bsk`. The intended use is multiple independent Signers, each given a PCZT with only the information it needs — for example, without the other signers' $\alpha$ values (`pczt`, `src/roles/redactor/mod.rs` and `orchard.rs`).

**Privacy.** A PCZT contains strictly more information than the transaction extracted from it: potentially counterparty addresses, values, note randomness, `rcv` (which opens `cv`), the spending account's full viewing key, ZIP-32 paths, `ock` (which opens `out_ciphertext`), and user-facing address strings. Whoever receives a PCZT at a given stage learns everything present at that stage; participants SHOULD send it only to entities required for the remaining roles and SHOULD use the Redactor to strip fields. Delegating the Prover reveals note private data — not spending authority. The extracted transaction contains none of this extra information (ZIP-374, Privacy Implications).

## 5.10 Transparent finalization and lock time

**Construction 5.11** (Script-sig assembly)**.** The Spend Finalizer constructs each transparent input's `script_sig`. Two forms are supported (`zcash_transparent`, `src/pczt/spend_finalizer.rs`; `pczt`, `src/roles/spend_finalizer/mod.rs`):

- *P2PKH*: require exactly one entry $(pubkey, sig)$ in `partial_signatures` satisfying

$$\mathrm{RIPEMD160}\big(\mathrm{SHA256}(pubkey)\big) = \text{the address hash};$$

  then

$$\texttt{script\_sig} := \mathrm{PUSH}(sig \,\|\, sighash\_type) \,\|\, \mathrm{PUSH}(pubkey).$$

- *P2SH-P2MS* ($m$-of-$n$ multisig):

$$\texttt{script\_sig} := \texttt{OP\_0} \,\|\, \mathrm{PUSH}(sig_1) \,\|\, \cdots \,\|\, \mathrm{PUSH}(sig_m) \,\|\, \mathrm{PUSH}(redeem\_script),$$

  where the leading `OP_0` is the dummy for the `OP_CHECKMULTISIG` bug and the signatures are the first $m$ available ones *in redeem-script pubkey order* (surplus signatures beyond the threshold are discarded; fewer than $m$ is a failure; pubkeys must be compressed 33-byte keys).

After finalization the role clears each input's required lock times, redeem script, partial signatures, BIP-32 derivations, and all preimage maps, keeping `value` and `script_pubkey` — the UTXO — so the Transaction Extractor can verify the final transaction.

**Construction 5.12** (Lock-time determination)**.** The final `nLockTime` follows BIP 370's rule (ZIP-374, Determining Lock Time; `pczt`, `src/common.rs`, `determine_lock_time`). Let $r$ hold if some input has a required time or height lock; let $t_u$ ("time unsupported") hold if some input requires a height lock (so a time lock cannot serve it), and $h_u$ symmetrically. Then:

$$\texttt{nLockTime} = \begin{cases} \text{FAIL} & r \wedge t_u \wedge h_u, \\ \max \text{ required height locks} & r \wedge \neg h_u, \\ \max \text{ required time locks} & r \wedge h_u \wedge \neg t_u, \\ \texttt{fallback\_lock\_time} \text{ or } 0 & \neg r, \end{cases}$$

height winning when both types remain possible; inputs specifying neither, or both, accept either type. The crate fails with `IncompatibleLockTimes` when one input requires only time and another only height.

## 5.11 Extraction and verification

The Transaction Extractor (`pczt` feature `tx-extractor`) checks transaction-version support and lock-time compatibility, then extracts the per-pool bundles, requiring every `script_sig`, every Sapling `zkproof` and `spend_auth_sig`, the Orchard bundle `zkproof` of canonical length for its action count, every Orchard `spend_auth_sig`, and each non-empty bundle's anchor and `bsk`; it rejects the identity `rk` and an invalid `ephemeral_key` (`pczt`, `src/roles/tx_extractor/mod.rs`).

On the Orchard side, extraction converts the PCZT bundle into a regular bundle whose authorization is `Unbound` $= \{\text{proof}, \mathsf{bsk}\}$; the canonical proof size is validated already at the `Unbound` stage, so the invariant "an `Authorized` bundle always has a canonical proof" holds across the binding step, and $value\_balance = \texttt{i64}(value\_sum)$ with `ValueSumOutOfRange` on overflow. The binding step is:

$$\text{verify every } \texttt{spend\_auth\_sig} \text{ under its } \mathsf{rk} \text{ over } sighash, \qquad binding\_sig := \mathrm{Sign}(\mathsf{bsk},\ sighash),$$

failing with the `pczt` crate's `Error::SighashMismatch` (`src/roles/tx_extractor/mod.rs`) if any spend authorization does not verify — the crate maps the `None` that `orchard`'s `apply_binding_signature` returns on failure; the `bsk`/`bvk` consistency established by the IO Finalizer (Construction 5.5) guarantees the binding signature verifies under the `bvk` derived from the transaction's value commitments and value balance (`orchard`, `src/pczt/tx_extractor.rs`, `apply_binding_signature`).

Finally the extractor *verifies* the completed transaction — proofs and signatures — "so that an invalid transaction is detected before it is broadcast" (ZIP-374, Transaction Extractor). Sapling proof verification requires caller-provided `SpendVerifyingKey`/`OutputVerifyingKey` (error `SaplingRequired` if absent but needed); the Orchard verifying key is generated on the fly if not provided via `with_orchard` (`pczt`, `src/roles/tx_extractor/mod.rs`).

### 5.12 The deployed wallet pipeline

The wallet layer drives the roles end to end. Function `create_pczt_from_proposal` — taking the wallet database, network parameters, spending account, OVK policy, and proposal (`zcash_client_backend` feature `pczt`, `src/data_api/wallet.rs`) — supports only single-step proposals. It runs the same internal transaction construction as its non-PCZT sibling `create_proposed_transactions`; then chains `Builder::build_for_pczt`, `Creator::build_from_parts`, and `IoFinalizer::finalize_io`; and finally runs an Updater pass that:

1. stores the proposal metadata $\{\mathit{from\_account}, \mathit{target\_height}\}$, postcard-encoded, under the Global proprietary key `zcash_client_backend:proposal_info`;
2. stores a reduced recipient enum — `External` (0), `EphemeralTransparent` (1), or `InternalAccount` (2), the latter two carrying the receiving account — under the per-output proprietary key `zcash_client_backend:output_info` on each shielded and transparent output;
3. sets `user_address` on external outputs; and
4. attaches derivation paths: to Orchard spends and non-dummy Sapling spends the fully hardened $[\,32', \mathit{coin\_type}', \mathit{account}'\,]$ under the account's seed fingerprint, and to transparent inputs the BIP-44 path $[\,44 \mid H,\ \mathit{coin\_type} \mid H,\ \mathit{account} \mid H,\ \mathit{scope},\ \mathit{address\_index}\,]$ with $H =$ `0x80000000` and the last two components non-hardened.

The PCZT is then handed to the caller for the Prover, Signer, and Combiner roles — on other devices if need be.

The return path is `extract_and_store_transaction_from_pczt(wallet_db, pczt, sapling_vk, orchard_vk)`: it runs the Spend Finalizer, reads back the proprietary proposal and output metadata — failing if `proposal_info` is missing, so the PCZT must have come from `create_pczt_from_proposal` — reconstructs each output's note from the PCZT's explicit `recipient`/`value`/`rseed` fields (outputs whose recipient is absent are dummies; outputs whose note fields were redacted are treated as deliberately unrecoverable and skipped), runs the Transaction Extractor, recovers memos via `try_output_recovery_with_pkd_esk` with `esk` re-derived from the note (post-ZIP-212; the derivation is the PRFexpand construction of the *Orchard Guide*, note encryption), computes the fee from the PCZT's transparent input values, and stores the result as a `SentTransaction`, returning the txid. The `sapling_vk` argument is optional so callers can first check `Prover::requires_sapling_proofs` and avoid downloading the Sapling parameters (`zcash_client_backend`, `src/data_api/wallet.rs`).

**Feature gating.** The `pczt` crate is `no_std` (alloc-based). The Creator, Verifier, Updater, Redactor, Combiner, and low-level Signer are always available; `io-finalizer`, `prover`, `signer`, `spend-finalizer`, and `tx-extractor` are cargo features; per-pool support is gated by the `transparent`, `sapling`, and `orchard` features; and `zcp-builder` enables `Creator::build_from_parts` from the `zcash_primitives` builder output (`pczt`, `Cargo.toml`, `src/lib.rs`, `src/roles.rs`). The crate's end-to-end tests exercise the full pipeline — Creator/Builder → IO Finalizer → Updater → Prover → Signer → Combiner → Spend Finalizer → Transaction Extractor — for the `transparent_to_orchard`,

`transparent_p2sh_multisig_to_orchard`, `sapling_to_orchard`, and `orchard_to_orchard` flows (`pczt`, `tests/end_to_end.rs`).

**ZIP-48 multisig.** ZIP-48 (transparent multisig) prescribes PCZT as its transaction-construction workflow: one participant constructs a PCZT spending multisig UTXOs, with change to a P2SH address at $m/48'/coin\_type'/account'/133000'/1/*$ (the $133000'$ component reads "Zcash P2SH"); the PCZT is conveyed out-of-band over a private authenticated channel; each participant MUST verify that change outputs are wallet-controlled, verifies that the spend is expected, adds their signature(s), and returns the signed PCZT; one participant combines a threshold of signed PCZTs and extracts the final transaction (ZIP-48, Transaction construction). The crate's `transparent_p2sh_multisig_to_orchard` test exercises exactly this flow.

**Tooling.** The zcash-devtool CLI exposes the full role pipeline as subcommands: `pczt create` / `create-max` / `shield` / `create-manual` / `pay-manual` (Creator through IO Finalizer via zcash_client_backend), `inspect`; `update-with-derivation`; `redact`; `prove`; `sign`; `update-with-signature`, which applies externally created signatures; `combine`; `extract`; `send`; `send-without-storing`; and `to-qr` / `from-qr` — animated QR rendering and webcam scanning of PCZTs (feature `pczt-qr`), illustrating the constrained-transport hardware-wallet flow. Its `sign` command selects which spends to sign by matching the seed fingerprint and account in the PCZT's `zip32_derivation` and `bip32_derivation` fields against the wallet's seed (`extract_account_index`), then calls the Signer's per-pool methods (zcash-devtool, `src/commands/pczt.rs` and `src/commands/pczt/sign.rs`).

## 5.13 PCZT v2: deferred anchors and the Ironwood bundle (specified, not deployed)

The draft's v2 (version number 2 in the header) extends the format for v6 transactions (ZIP-229): it adds an `IronwoodBundle`, adds a `note_version` field (an enum $\{V2, V3\}$) to Orchard-protocol bundles, makes shielded bundle anchors `Option<[u8;32]>`, and omits canonically-empty bundles from the encoding (ZIP-374, Versioning and v2 Encoding). Classification per our conventions: *specified* — present in the Draft ZIP, absent from the deployed `pczt` 0.5.1.

For `tx_version` 6, anchors and witnesses become deferrable: signatures commit to neither, an Updater may set or replace an anchor before proving — replacing an anchor MUST clear the bundle's proofs, which commit to it as a circuit public input — and witnesses must root to the anchor whenever both are present, checked by the Updater and the Prover. This enables re-anchoring fully signed transactions with the txid unchanged, and spending *unmined* Orchard-protocol notes: an Orchard nullifier depends only on note data, with $\rho$ fixed by the creating action rather than by tree position — impossible for Sapling even under v6, because Sapling nullifiers depend on the note's position. An all-zeroes anchor sentinel was rejected in favour of `Option` because 0 is a valid Pallas base element (ZIP-374, Anchors and pre-authorization; Rationale, "Optional anchors in PCZT v2").

From NU6.3, Orchard-pool actions must be created with cross-address transfers disabled — bundle flag bit 2, `enableCrossAddress`, MUST be 0 in PCZT v1 and in any Orchard bundle mined under NU6.3+ — and wallets pair each real spend with a *fabricated* zero-valued output to the spent note's own receiver, whose `enc_ciphertext` is random bytes. Signers MUST NOT reject a zero-valued output whose ciphertext does not decrypt on that basis alone, and SHOULD classify it as a tolerable dummy (optionally checking `cmx` against the explicit `recipient`/`value`/`rseed`). Conversely, a fabricated zero-valued spend paired with a real output is a *real* spend — no `dummy_sk`; it is signed via the normal Signer flow with the receiver's spend authorizing key; only fully-dummy actions carry `dummy_sk` (ZIP-374, Fabricated same-address outputs). The `IronwoodBundle` carries the Ironwood-pool component of a v6 transaction, structurally identical to `OrchardBundle` — ZIP-229 defines the Ironwood pool as a second value pool of the Orchard

protocol reusing the Action structure — differing in that `enableCrossAddress` is normally 1 (cross-address transfers are the usual case in the Ironwood pool), the anchor refers to the Ironwood note commitment tree, and `note_version` MUST be V3 (the ZIP-2005 quantum-recoverable note plaintext, lead byte `0x03`); a non-empty `IronwoodBundle` in a `tx_version`-5 PCZT MUST be rejected (ZIP-374, IronwoodBundle and Rationale).

The local orchard repository (0.14.0) already implements the NU6.3-era machinery — bundle-format-aware flag parsing (`Flags::from_byte(flags, format)`, `UnexpectedFlagBitsSet`); the check `verify_cross_address_restriction`, requiring every action's output addressed to the same $(\mathsf{g_d}, \mathsf{pk_d})$ as its spend when flag bit 2 is 0, enforced by `finalize_io` and `create_proof` and provable only with an Ironwood-version proving key; same-receiver comparison via `Address::same_receiver`; and change outputs paired with a fabricated same-receiver spend — while librustzcash still pins orchard 0.12 (orchard, `src/pczt/verify.rs`, `io_finalizer.rs`, `prover.rs`, `parse.rs`, and `src/builder.rs`; librustzcash `Cargo.toml`). When we cite orchard PCZT behaviour above, the claims are about the 0.12 line the deployed `pczt` 0.5.1 links against, except where marked.

### 5.14 Divergences between ZIP-374 and the deployed code

Per our conventions, we flag rather than resolve the following.

- *Dummy-key rejection is not implemented in the reference Signer.* ZIP-374 states that "Signers MUST reject PCZTs that contain `dummy_sk` values" (likewise `dummy_ask`), but no code path in `pczt` 0.5.1's Signer inspects those fields — only the Redactor offers `clear_spend_dummy_sk`/`clear_dummy_ask`, and the IO Finalizer clears the keys after use. The MUST therefore binds external signer implementations, such as hardware wallets, not the crate's own `Signer` type (ZIP-374, OrchardSpend.dummy_sk; `pczt`, `src/roles/signer/`).

- *A v4 PCZT is constructible but dead.* The ZIP forbids creating v4-or-earlier transactions, yet `Creator::build_from_parts` maps `TxVersion::V4` to a PCZT with `tx_version` = 4 (Sprout/V3 return `None`). Every subsequent role rejects anything but $(5, \texttt{0x26A7270A})$, so the state cannot progress (`pczt`, `src/roles/creator/mod.rs`; `src/roles/signer/mod.rs`).

- *Negative-zero initialization.* `Creator::new` sets the Orchard `value_sum` = (0, true), whereas the v2 draft's canonical empty `OrchardBundle` specifies (0, false). Numerically equal and harmless under v1, but the byte encodings differ, which would prevent the v2 empty-bundle-omission rule from round-tripping (`pczt`, `src/roles/creator/mod.rs`; ZIP-374, v2 Encoding).

- *The `ock` field is never populated.* The ZIP describes Sapling/Orchard `ock` as `None` only under the OVK policy `None`, but the deployed Orchard builder always sets `ock` = `None` (in-code TODO), so the Signer capability of verifying `out_ciphertext` is not currently exercisable for wallet-created PCZTs (orchard, `src/builder.rs`, `OutputInfo::into_pczt`).

- *Unspecified sign convention.* The ZIP declares the Orchard `value_sum` as `(u64, bool)` without stating the boolean's meaning; only the reference implementation defines it (true = negative) — a spec gap (ZIP-374, OrchardBundle; orchard, `src/pczt/parse.rs`).

- *Version skew.* The ZIP names `pczt` 0.7.0 as the release whose serialization defines the v1 encoding; the librustzcash checkout ships 0.5.1, and the local orchard 0.14 already implements draft-v2 machinery whose `Bundle::parse` signature is incompatible with the 0.5.1 call sites (ZIP-374, Reference implementation; librustzcash `Cargo.toml`; orchard, `Cargo.toml`).

# 6 Broadcast, expiry, and the transaction lifecycle

The *Orchard Guide* leaves off with a transaction fully proved, signed, and bound — an artefact whose internal validity is settled. This section follows that artefact through the rest of its life: serialisation and the identifier under which the network knows it, the deployed store-then-broadcast discipline, the expiry mechanism that bounds how long it can remain pending, the bookkeeping by which the wallet tracks it to one of its terminal states, the confirmation policy that governs when its outputs become spendable, and the truncate-and-rescan procedure that repairs the wallet's view after a chain reorganisation. Throughout, we develop the shielded path; the transparent-input monitoring machinery (`TransactionsInvolvingAddress` and its filters) is gated behind the `transparent-inputs` feature of zcash_client_backend and is mentioned only in passing, so for a shielded-only wallet the monitoring surface is exactly the two status queries of §6.4.

## 6.1 Serialisation and the transaction identifier

A built transaction is serialised for broadcast by `Transaction::write`, which dispatches on version: v5 transactions (version 5, version group id `0x26A7270A`; zcash_protocol, `src/constants.rs`) use `write_v5`, the ZIP-225 wire format of the protocol specification §7.1 (*Transaction Encoding and Consensus*), while v1–v4 use `write_v4` (zcash_primitives, `src/transaction/mod.rs`). The wallet places the resulting raw bytes in the `data` field of a `RawTransaction` gRPC message for submission (zcash-devtool, `src/commands/wallet/send.rs`).

The identifier under which the network, the mempool, and the wallet's own database all track the transaction depends on the version. For v5, the *txid* is the root of the ZIP-244 digest tree,

$$\mathsf{txid} := \text{BLAKE2b-256}\big(\texttt{ZcashTxHash\_} \,\|\, \mathsf{branchid},\ d_{\text{hdr}} \,\|\, d_{\text{trans}} \,\|\, d_{\text{Sap}} \,\|\, d_{\text{Orch}}\big),$$

where the personalisation is the 12-byte literal `ZcashTxHash_` (one underscore) followed by $\mathsf{branchid} = \text{LE}_{32}(\text{CONSENSUS\_BRANCH\_ID})$, the 4-byte little-endian consensus branch id, and an absent pool hashes the empty string under its own personalisation (ZIP-244, *TxId Digest*; zcash_primitives, `src/transaction/txid.rs`). The *Orchard Guide* assembles this tree in full ("The transaction digest (ZIP-244)", within its binding-signature section); we do not re-derive it. Two consequences matter for the lifecycle. First, the tree is computed over *effecting* data only, so the v5 txid is non-malleable: no third party can alter the transaction's serialisation without changing its identifier. For v1–v4 the txid is instead the double-SHA-256 of the serialised transaction, which *is* malleable and therefore reliable only once the transaction is mined (zcash_protocol, `src/txid.rs`). Second, because the personalisation bakes in the branch id, the same transaction has a *different* txid on parallel consensus branches — replay protection across forks (ZIP-244, *TxId Digest*). For a purely shielded transaction the sighash coincides with the txid, as the *Orchard Guide* already notes.

The header node of the tree is the one this section leans on:

$$d_{\text{hdr}} := \text{BLAKE2b-256}\big(\texttt{ZTxIdHeadersHash},\ \text{LE}_{32}(\mathsf{version}) \,\|\, \text{LE}_{32}(\mathsf{versionGroupId}) \,\|\, \text{LE}_{32}(\mathsf{branchid}) \,\|\, \text{LE}_{32}(\mathsf{lockTime}) \,\|\, \text{LE}_{32}(\mathsf{nExpiryHeight})\big),$$

with version including the overwinter flag (ZIP-244, node T.1). The field nExpiryHeight of §6.3 is therefore *effecting* data, committed by the txid: expiry cannot be malleated. Zebra's mempool relies on exactly this, matching and rejecting expired transactions by their non-malleable txid (zebrad, `src/components/mempool/storage.rs`).

One convention trips up every first-time implementer: txid bytes are stored and transmitted in protocol order but *displayed* byte-reversed and hex-encoded. The `Display` implementation of `TxId` reverses before encoding, and the zcash_client_sqlite view documentation states the same rule for its stored byte vectors (zcash_protocol, `src/txid.rs`; zcash_client_sqlite, `src/wallet/db.rs`).

## 6.2 Store, then broadcast

The deployed pipeline persists a transaction *before* the network ever sees it. The function `create_proposed_transactions` (zcash_client_backend, `src/data_api/wallet.rs`) constructs, proves, and signs each step of a proposal (§6.6 fixes the target height it builds against), then persists the results via `WalletWrite::store_transactions_to_be_sent`, whose contract reads "This must be called before the transactions are sent to the network" (zcash_client_backend, `src/data_api.rs`). The ordering is deliberate: a transaction the network might mine but the wallet has forgotten is unrecoverable spend history, whereas a stored transaction the network never saw is merely stale, and stored transactions "should be retransmitted while it is still possible that they could be mined" (zcash_client_backend, `src/data_api.rs`). In multi-step proposals (e.g. the ZIP-320 pairs of §6.5), storage happens only after *all* steps have been created, so that failure partway through creation cannot leave a transmittable prefix behind (zcash_client_backend, `src/data_api/wallet.rs`).

Persisting a created transaction does two things (zcash_client_sqlite, `src/wallet.rs`, `store_transaction_to_be_sent`). First, it records the transaction itself: txid, expiry height read from `tx.expiry_height()`, raw bytes, fee, target height, and `min_observed_height` := target height (Table 4). Second, it marks every spent input as spent — Sapling and Orchard nullifiers into the `*_received_note_spends` tables, transparent outpoints into `transparent_received_output_spends` — which locks those inputs against reselection by later proposals until the transaction is mined, or expires (§6.5).

Submission itself is one gRPC call: the lightwalletd `CompactTxStreamer` method `SendTransaction(RawTransaction) returns (SendResponse)`, where `errorCode` = 0 means accepted and any other value carries an `errorMessage` (zcash_client_backend, `proto/service.proto`; zcash-devtool, `src/commands/wallet/send.rs`). Zaino implements the same `CompactTxStreamer` service as a drop-in replacement for lightwalletd, so submission is identical against either (zaino, `README.md`).

**Remark 6.1** (The rebroadcast responsibility boundary)**.** The librustzcash documentation records the obligation to retransmit — stored transactions "should be retransmitted while it is still possible that they could be mined" (zcash_client_backend, `src/data_api.rs`), and no-expiry transactions "should be rebroadcast by the wallet until they are either mined or conflict with another mined transaction" (zcash_client_sqlite, `src/wallet.rs`) — but implements no rebroadcast loop itself. The obligation falls on the embedding wallet application; the deployed CLI reference (zcash-devtool) submits exactly once.

## 6.3 Expiry (ZIP-203)

An unmined transaction cannot be left dangling forever: an indefinitely pending payment is indeterminism for the user and dead weight for every mempool. ZIP-203 (Status Final, Category Consensus) resolves this with a consensus-enforced expiry height.

**Definition 6.2** (Transaction expiry, ZIP-203)**.** Every transaction from Overwinter onward carries a field nExpiryHeight of type `uint32` — always a block height, never a UNIX timestamp. A transaction $t$ is *expired* at block height $H$ iff

$$(t \text{ is not coinbase}) \ \wedge \ (t.\mathsf{nExpiryHeight} \neq 0) \ \wedge \ (H > t.\mathsf{nExpiryHeight}),$$

and the consensus rule states: if a transaction is not a coinbase transaction and its nExpiryHeight is nonzero, it MUST NOT be mined at a block height greater than its nExpiryHeight; a block containing an expired transaction is invalid (ZIP-203, *Specification*; protocol specification §7.1). Expiry at height $N$ thus makes block $N$ the last block that may include the transaction. The sentinel nExpiryHeight = 0 disables expiry, and for non-coinbase transactions nExpiryHeight $\leq$ 499999999 MUST hold. From NU5 the bound is removed for coinbase only, and the nExpiryHeight

of a coinbase transaction MUST instead equal its block height — this keeps v5 coinbase txids unique, since the ZIP-244 txid does not commit to the coinbase script height (ZIP-203, *Changes for NU5*; protocol specification §7.1).

**The deployed default: exactly 40 blocks.** ZIP-203 sets the default expiry delta at 20 blocks before Blossom and, from Blossom activation, 40 blocks from the current height — about 50 minutes at the 75-second target spacing — and permits nodes to override it (zcashd's `-txexpirydelta` option; ZIP-203, *RPC*, *Config*). The deployed light-wallet stack is stricter than the ZIP requires: the transaction builder has *no public expiry setter*. The constructor `Builder::new` unconditionally sets

$$\mathsf{nExpiryHeight} := \begin{cases} \text{targetHeight} & \text{coinbase,} \\ \text{targetHeight} + \texttt{DEFAULT\_TX\_EXPIRY\_DELTA} & \text{otherwise,} \end{cases}$$

with `DEFAULT_TX_EXPIRY_DELTA` = 40, the coinbase rule applied regardless of network upgrade, and the `expiry_height` field of `Builder` is private (zcash_primitives, `src/transaction/builder.rs`). Every deployed librustzcash-built wallet transaction therefore expires exactly 40 blocks after its target height. The rigidity is a feature: ZIP-315 (Draft; see Remark 6.6) says wallets SHOULD use the default 40-block expiry, and a wallet emitting idiosyncratic expiry deltas would fingerprint its transactions, exactly as an idiosyncratic fee would (§3).

**Remark 6.3** (Documentation divergence: the builder's stale delta)**.** The doc comment on `Builder::new` states that the expiry height is set to the target "plus the default transaction expiry delta (20 blocks)", while the constant it actually adds is `DEFAULT_TX_EXPIRY_DELTA` = 40 (zcash_primitives, `src/transaction/builder.rs`). The code matches ZIP-203 post-Blossom; the comment preserves the pre-Blossom value. We follow the code.

**Mempool eviction is node policy, mining is consensus.** The consensus rule of Definition 6.2 constrains block *contents*; what nodes do with their mempools is policy layered on top. ZIP-203 specifies that when a transaction expires it is removed from nodes' mempools, together with *all of its dependent transactions*. Zebra's `remove_expired_transactions` evicts every mempool transaction once

$$\text{tipHeight} \geq t.\mathsf{nExpiryHeight},$$

i.e. as soon as the tip reaches the expiry height — at which point no future block can include $t$ — and adds the evicted transaction's non-malleable txid to its rejection list with `SameEffectsChainRejectionError::Expired` (zebrad, `src/components/mempool/storage.rs`). The two formulations agree operationally with ZIP-203's "block $N$ or earlier; block $N + 1$ will be too late".

**The expiring-soon door policy.** The zcashd node additionally refuses (as denial-of-service mitigation, not consensus) to accept into its mempool, or relay, a transaction that is *expiring soon*: one that would be expired within `TX_EXPIRING_SOON_THRESHOLD` = 3 blocks of the next block height, where $\text{IsExpiredTx}(t, h) = (t.\mathsf{nExpiryHeight} \neq 0 \wedge t \text{ not coinbase} \wedge h > t.\mathsf{nExpiryHeight})$ (zcashd, `src/main.h`, `src/main.cpp`). The practical consequence for wallets: a transaction submitted with fewer than about four blocks of remaining life is refused at the mempool door. Zebra applies no such buffer: its only admission-time expiry check is the consensus rule itself, evaluated at the next block height. The mempool hands each candidate to the transaction verifier with height tipHeight + 1 (zebrad, `src/components/mempool/downloads.rs`), and the verifier's `non_coinbase_expiry_height` rejects only when that height exceeds $t.\mathsf{nExpiryHeight}$ (zebra-consensus, `src/transaction/check.rs`); no expiring-soon threshold appears anywhere in the mempool component. A transaction with a single block of remaining life is therefore accepted and relayed by Zebra but refused by zcashd.

**Expiry never straddles a network upgrade.** The zcashd node clamps a new transaction's nExpiryHeight to one less than the next network-upgrade activation height (zcashd, `src/main.cpp`), so a transaction never straddles an upgrade boundary. The librustzcash parser *relies* on this invariant when parsing transactions it holds in unmined form: since "a transaction can't be mined across a network upgrade boundary, … the expiry height must be in the same epoch", an unmined transaction with nonzero expiry is parsed under `BranchId::for_height`(nExpiryHeight); an unmined v4 transaction with expiry 0 carries no epoch hint at all and cannot be parsed until mined (zcash_client_sqlite, `src/wallet.rs`, `parse_tx`).

### 6.4 Tracking a pending transaction

Between submission and its terminal state, a transaction exists only as a row in the wallet database and an entry in remote mempools. The wallet-side record is the zcash_client_sqlite `transactions` table (Table 4); note that it may hold data unrecoverable from the chain, e.g. wallet-created transactions that expired unmined (zcash_client_sqlite, `src/wallet/db.rs`).

| Column | Semantics |
|---|---|
| `txid` | transaction identifier, protocol byte order (`UNIQUE`; reverse and hex-encode for display) |
| `created` | wallet-local creation time |
| `block` | height of the containing block, set only once that block has been *scanned* (foreign key to `blocks(height)`) |
| `mined_height` | mined height, settable from a status query without scanning the block; `CHECK` equal to `block` when `block` is non-null |
| `tx_index` | index of the transaction within its block |
| `expiry_height` | maximum height at which the transaction may be mined, if known |
| `raw` | serialised transaction bytes, if retrieved |
| `fee` | fee paid, if known |
| `target_height` | construction target height; set only for transactions this wallet created |
| `min_observed_height` | `NOT NULL`; minimum of the mempool-observation height and the mined height |
| `confirmed_unmined_at_height` | maximum height with positive proof of unminedness; `CHECK NULL` when `mined_height` is non-null |
| `trust_status` | user-assigned trust flag (§6.6) |

Table 4: Columns of the `transactions` table (zcash_client_sqlite, `src/wallet/db.rs`).

**Statuses and how they are learned.** The backend distinguishes three externally observable states, the enum `TransactionStatus`: `TxidNotRecognized`, `NotInMainChain` (in the mempool, or mined only on a reorged-away fork), and `Mined(height)` (zcash_client_backend, `src/data_api.rs`). Statuses arrive by evaluating `TransactionDataRequest` values the wallet emits: `GetStatus(txid)`; `Enhancement(txid)`, which downloads the full raw transaction and subsumes `GetStatus`; and, behind the `transparent-inputs` feature, `TransactionsInvolvingAddress`. These are typically served by lightwalletd's `GetTransaction` and `GetTaddressTxids` RPCs and fed back via `WalletWrite::set_transaction_status` or `decrypt_and_store_transaction` (zcash_client_backend, `src/data_api.rs`). Active polling is not optional even for a shielded wallet's own history: fully transparent transactions, and transactions containing neither shielded inputs nor shielded outputs belonging to the wallet, are invisible to the compact-block trial decryption of the *Orchard Guide* ("Transmission and detection of a note"), so scanning alone can never confirm their fate (zcash_client_backend, `src/data_api.rs`).

The wire encoding of a status reply hides a protobuf quirk. The `RawTransaction.height` field signals: 0 = in the mempool; `0xffffffffffffffff` = mined on a fork that is not the main chain; any other value = mined height in the main chain (zcash_client_backend, `proto/service.proto`). The deployed client maps

$$\begin{aligned} \text{gRPC } \texttt{NotFound} &\mapsto \texttt{TxidNotRecognized}, \\ \text{height} \notin [1, 2^{32}-1] &\mapsto \texttt{NotInMainChain}, \\ \text{otherwise} &\mapsto \texttt{Mined}(\text{height}) \end{aligned}$$

(zcash-devtool, `src/commands/wallet/enhance.rs`).

**Applying a status.** On `TxidNotRecognized` or `NotInMainChain` for an unmined transaction, `set_transaction_status` records positive proof of unminedness, setting `confirmed_unmined_at_height` to the current chain tip. On `Mined`(h) it sets `mined_height` to $h$, lowers `min_observed_height` to at most $h$, clears `confirmed_unmined_at_height`, links `block` if that block is already scanned, and updates transparent gap limits; in every case the pending `tx_retrieval_queue` entry is deleted (zcash_client_sqlite, `src/wallet.rs`). Independently, when chain scanning itself encounters a wallet-relevant transaction in a block, `put_tx_meta` upserts `block` and `mined_height` to the scanned height along with `tx_index`, takes the minimum for `min_observed_height`, and clears `confirmed_unmined_at_height` (zcash_client_sqlite, `src/wallet.rs`).

**When to stop asking.** A `GetStatus` request is (re)generated for every transaction with `mined_height` NULL while any of the following holds (zcash_client_sqlite, `src/wallet.rs`, `transaction_data_requests`):

$$\begin{aligned} &\texttt{confirmed\_unmined\_at\_height} \text{ IS NULL} \\ &\vee \big(\texttt{expiry\_height} > 0 \wedge \texttt{confirmed\_unmined\_at\_height} < \texttt{expiry\_height}\big) \\ &\vee \big(\texttt{expiry\_height} \text{ IS NULL} \\ &\qquad \wedge \texttt{confirmed\_unmined\_at\_height} < \texttt{min\_observed\_height} + \text{certaintyDepth}\big), \end{aligned}$$

where certaintyDepth := `PRUNING_DEPTH`+`DEFAULT_TX_EXPIRY_DELTA` = 100+40 = 140 blocks, and the result is unioned with explicit `tx_retrieval_queue` entries. A transaction with known nonzero expiry is thus polled until unminedness is confirmed at or beyond its expiry height; one with unknown expiry until 140 blocks past first observation; and one with expiry 0 indefinitely — such transactions "should be rebroadcast by the wallet until they are either mined or conflict with another mined transaction" (Remark 6.1). For display, the `v_transactions` view exposes the terminal flag

$$\texttt{expired\_unmined} := \texttt{mined\_height} \text{ IS NULL} \ \wedge\ 1 \le \texttt{expiry\_height} \le \max(\texttt{blocks.height})$$

(zcash_client_sqlite, `src/wallet/db.rs`), i.e. nonzero expiry at or below the maximum scanned height.

**Remark 6.4** (Documentation divergence: the retrieval-queue codes)**.** The table documentation for `tx_retrieval_queue` says `query_type` is "0 for raw transaction (enhancement) data, 1 for transaction mined-ness information" (zcash_client_sqlite, `src/wallet/db.rs`), but the authoritative encoder `TxQueryType::code()` maps `Status` $\mapsto$ 0 and `Enhancement` $\mapsto$ 1 and round-trips through `from_code` (zcash_client_sqlite, `src/wallet.rs`). The documentation is inverted relative to the code; the code governs.

## 6.5 Expiry and fund availability

Storing a transaction locked its inputs (§6.2); expiry is what unlocks them. Note selection excludes a note appearing in `*_received_note_spends` only when the spending transaction is

still *unexpired* in the following sense, evaluated at the target height $H$ of the new proposal (zcash_client_sqlite, `src/wallet/common.rs`, `tx_unexpired_condition`):

$$\begin{aligned} &\texttt{mined\_height} < H && \text{(mined)}\\ &\vee\ \texttt{expiry\_height} = 0 && \text{(never expires)}\\ &\vee\ \texttt{expiry\_height} \geq H && \text{(unexpired)}\\ &\vee\ \big(\texttt{expiry\_height}\ \text{IS NULL} \wedge \texttt{min\_observed\_height} + 40 \geq H\big) && \text{(default-delta guess).} \end{aligned} \tag{1}$$

A transaction that has expired unmined (`mined_height` NULL, $0 < \texttt{expiry\_height} < H$) fails every disjunct, so its inputs re-enter the spendable set for target heights beyond the expiry height. ZIP-315's known-spendable definition matches: a TXO must not be "committed to be spent in another unexpired transaction created by this wallet" (ZIP-315, Draft).

**Remark 6.5** (The unknown-expiry guess, and an inverted comment)**.** The final disjunct of (1) is a guess that the originating wallet used the default delta. Reading the predicate directly: if the sender in fact used a *larger* delta or disabled expiry, the transaction is treated as *expired* from height $\texttt{min\_observed\_height} + 41$ onward while it is actually still mineable — its inputs unlock prematurely, and a respend can conflict with a late mining of the original. If the sender used a *smaller* delta, the inputs merely stay locked slightly longer than necessary. Either misclassification persists only until the wallet observes the transaction mined or enhances it to learn the true expiry height. The doc comment on `tx_unexpired_condition` describes the first case as "treated as unexpired when it shouldn't be" (zcash_client_sqlite, `src/wallet/common.rs`), which is inverted relative to its own SQL; we state the direction the predicate implies and flag the comment rather than silently following either.

**Nullifiers outlive the lock.** Dropping a pending spend's inputs from the *spendable* set is not the same as forgetting them. The query `get_nullifiers(NullifierQuery::Unspent)` deliberately over-selects: a note spent in an unexpired-but-unmined transaction still contributes its nullifier, so that scanning (the *Orchard Guide*, "Transmission and detection of a note") detects the spend if the transaction is later mined; a note's nullifier leaves the watch set only once the spending transaction is mined or can never expire (zcash_client_sqlite, `src/wallet/common.rs`).

**Aftermath of an expiry.** ZIP-203 (*Wallet behavior and UI*) draws the user-facing conclusions: an expiring zero-confirmation transaction has even less inclusion guarantee, so UIs and services must never rely on zero-conf transactions in Zcash, and wallets should notify the user of expired transactions that must be re-sent. ZIP-315 (Draft) adds that sent transactions SHOULD be reported with confirmations and, while unmined with an expiry height, an estimate of time until expiry. Re-sending carries a privacy cost the wallet should recognise: ZIP-315 identifies repeated spends of the same TXOs across transactions — the typical aftermath of re-sending an expired payment — as an information leak linking the invalidated transaction to its replacement. For ZIP-320 (TEX) payments the deployed convention is to set the *same* expiry height on the first (shielded-to-ephemeral) and second (ephemeral-to-TEX) transactions, so another wallet on the same seed can infer when the second would have expired without observing it; if the second expires unmined, the ephemeral UTXO should be re-shielded before being re-spent (zcash_client_backend, `src/data_api.rs`).

## 6.6 Confirmations, trust, and spendability

Mining is not the end of caution: a freshly mined output can be un-mined by a reorganisation (§6.7), so the wallet imposes a confirmation policy before treating outputs as spendable.

**Remark 6.6** (ZIP-315 is a Draft)**.** ZIP-315 ("Best Practices for Wallet Handling of Multiple Pools") has Status Draft. The deployed stack nonetheless implements its central recommendations

— the 3/10 confirmation split, zero-conf shielding, the fixed anchor depth, the 40-block expiry — so throughout this section deployed behaviour cites the crates, and ZIP-315 is cited as the (draft) rationale, not as a finalised standard.

**Definition 6.7** (Confirmations policy)**.** A `ConfirmationsPolicy` holds two `NonZeroU32` counts, $c_{\mathrm{tr}}$ (*trusted*) and $c_{\mathrm{untr}}$ (*untrusted*), with the constructor enforcing $c_{\mathrm{tr}} \leq c_{\mathrm{untr}}$, plus a flag `allow_zero_conf_shielding`. The default is $c_{\mathrm{tr}} = 3$, $c_{\mathrm{untr}} = 10$, with zero-conf shielding allowed, per ZIP-315; the constant `ConfirmationsPolicy::MIN` is 1/1/`true`. Confirmations are counted since *and including* the block in which a note was produced (zcash_client_backend, `src/data_api/wallet.rs`).

The trusted/untrusted split is ZIP-315's: a *trusted* TXO is one received from a party where the wallet trusts it will remain mined in its original transaction — e.g. created by the wallet's own internal TXO handling — and everything else is untrusted. The recommended counts rest on two observations: reorganisations are anecdotally shorter than 3 blocks, and value from a trusted TXO is always recoverable after a rollback (the wallet can just re-create the transaction), whereas an untrusted TXO may require asking the sender to re-send (ZIP-315, Draft). Users may also explicitly mark specific transactions trusted via `WalletWrite::set_tx_trust`, persisted in the `trust_status` column — ZIP-315's "MAY enable users to mark specific external transactions as trusted" provision (zcash_client_backend, `src/data_api.rs`; zcash_client_sqlite, `src/wallet.rs`).

**Definition 6.8** (Confirmations until spendable)**.** Fix a target height $H$ and write $x \ominus y$ for saturating subtraction (`BlockHeight` subtraction in librustzcash yields a `u32` via `saturating_sub`; zcash_protocol, `src/consensus.rs`). Let $h_{\mathrm{tr}} := H \ominus c_{\mathrm{tr}}$ and $h_{\mathrm{untr}} := H \ominus c_{\mathrm{untr}}$, and for an output of a transaction with mined height $m$ (if any) let

$$\mathrm{confs}_{\mathrm{tr}} := \begin{cases} c_{\mathrm{tr}} & m \text{ unknown} \\ m \ominus h_{\mathrm{tr}} & \text{else,} \end{cases} \qquad \mathrm{confs}_{\mathrm{untr}} := \begin{cases} c_{\mathrm{untr}} & m \text{ unknown} \\ m \ominus h_{\mathrm{untr}} & \text{else.} \end{cases}$$

Then `confirmations_until_spendable` returns, and the output is spendable iff the result is 0 (zcash_client_backend, `src/data_api/wallet.rs`):

- *transparent*: 0 if `allow_zero_conf_shielding`; else $\mathrm{confs}_{\mathrm{tr}}$ if the transaction is trusted or the receiving key scope is internal; else $\mathrm{confs}_{\mathrm{untr}}$;
- *shielded*: $\mathrm{confs}_{\mathrm{tr}}$ if the transaction is trusted; else, for internal scope, if the maximum mined height $h_s$ of the transparent inputs to the shielding transaction is known, $h_s \ominus h_{\mathrm{tr}}$ when those inputs are trusted and $h_s \ominus h_{\mathrm{untr}}$ otherwise, and $\mathrm{confs}_{\mathrm{tr}}$ when $h_s$ is unknown; else $\mathrm{confs}_{\mathrm{untr}}$.

The saturation is what makes the count "decrease to a floor of zero" for long-mined outputs.

The internal-scope shielded branch encodes ZIP-315's rule that shielded funds produced by zero-conf shielding are unavailable until the *source UTXOs* have sufficient confirmations: spendability of a shielding output is computed from $h_s$ = `max_shielding_input_height` and the inputs' trust status, not from the shielding transaction's own mined height (zcash_client_backend, `src/data_api/wallet.rs`).

**Remark 6.9** (Deployed shielding rule vs. the ZIP-315 text)**.** The ZIP-315 text requires both that the shielding transaction have at least the trusted confirmation count *and* that its transparent inputs have at least $c_{\mathrm{untr}} - c_{\mathrm{tr}}$ confirmations. The deployed rule of Definition 6.8 tests only the input heights against the trusted/untrusted thresholds and imposes no separate requirement on the shielding transaction's own confirmations. The two rules do not coincide. Count confirmations as in Definition 6.8, write $h_s$ for the maximum input height and $h_m \geq h_s$ for the height at which the shielding transaction was mined, and let $d := h_m - h_s$. The ZIP composite admits the output

at target heights $H \geq h_s + \max(c_{\text{untr}} - c_{\text{tr}}, c_{\text{tr}} + d)$, the deployed rule (inputs untrusted) at $H \geq h_s + c_{\text{untr}}$; the thresholds agree only at $d = c_{\text{untr}} - c_{\text{tr}}$. Below that point — the common case of a promptly mined shielding transaction — the deployed rule is strictly stricter: at the default policy with $d = 1$, the ZIP releases the output at $H = h_s + 7$ while the code waits until $h_s + 10$. Above it, the deployed rule is strictly laxer: with $d = 9$ the code releases at $h_s + 10$, when the shielding transaction has a single confirmation, while the ZIP waits until $h_s + 12$. The deployed trusted-input branch ($H \geq h_s + c_{\text{tr}}$) and the unknown-$h_s$ fallback have no ZIP counterpart at all. The deployed rule is thus implementation policy diverging from the (Draft) ZIP text — conservative precisely when the shielding transaction confirms within $c_{\text{untr}} - c_{\text{tr}}$ blocks of its inputs.

**The full spendability gate.** Deployed note selection admits a shielded note as spendable only when all of the following hold (zcash_client_sqlite, `src/wallet/common.rs`): the note's transaction is in a *scanned* block (`t.block` non-null); its nullifier and commitment-tree position are known and the containing shard is fully scanned (witness data exists — the *Orchard Guide*, "Proof of ownership of *some* valid note: the commitment tree"); its mined height is at or below the anchor height; `confirmations_until_spendable` $= 0$ under the policy; and it is not excluded by the spent-notes clause — spent in a mined or unexpired pending transaction, condition (1).

**Target and anchor heights.** The heights every predicate above is evaluated against are fixed at proposal time (zcash_client_sqlite, `src/wallet.rs` and `src/wallet/commitment_tree.rs`; zcash_client_backend, `src/data_api/wallet.rs`):

$$\begin{aligned}
&\text{chainTip} := \max(\texttt{scan_queue.block_range_end}) - 1 \quad \text{(ranges are end-exclusive)},\\
&H := \text{chainTip} + 1 \quad \text{(the "mempool height")},\\
&\text{anchorHeight}_{\text{pool}} := \max\{\,\text{checkpoint} \;:\; \text{checkpoint} \leq H - \text{minConf}\,\},\\
&\text{anchor} := \min_{\text{pools with checkpoints}} \text{anchorHeight}_{\text{pool}} \quad \text{(Sapling, Orchard)},
\end{aligned}$$

and `propose_transfer` uses the trusted count, minConf $= c_{\text{tr}}$ (default 3); the target height that `create_proposed_transactions` hands to `Builder::new` is the proposal's `min_target_height()`. The anchor depth therefore *equals* the trusted confirmation count — a fixed depth for all transactions, matching ZIP-315's anchor-selection recommendation (anchor at the final treestate of block $H - 3$) so that anchor choice does not leak whether the notes being spent were trusted. The tree and anchor mechanics themselves are the *Orchard Guide*'s (its commitment-tree section); the wallet-layer content is only this choice of depth.

### 6.7 Reorganisations: truncate and rescan

A reorganisation invalidates the wallet's suffix of the chain: mined transactions become unmined, anchors vanish, and witness data beyond the fork point is wrong. The deployed stack detects this during scanning and repairs it by truncation.

**Detection.** Scanning reports continuity errors: the `ScanError` variants `PrevHashMismatch` (the parent hash of a new block does not match the current tip), `BlockHeightDiscontinuity`, and `TreeSizeMismatch` all satisfy `is_continuity_error()` = `true` (zcash_client_backend, `src/scanning.rs`). To surface reorganisations *promptly* rather than on the next unlucky scan, `update_chain_tip` marks the `VERIFY_LOOKAHEAD` = 10 blocks above the maximum scanned height with `ScanPriority::Verify` whenever that height is at or below the stable height tip − `PRUNING_DEPTH`; ranges marked `Verify` must always be scanned before `ChainTip` ranges. The choice of 10 blocks corresponds to roughly 12.5 minutes, within which a user plausibly received a transaction without their wallet having scanned the block (zcash_client_sqlite, `src/wallet/scanning.rs`, `src/lib.rs`).

**Recovery loop.** On a continuity error $e$, the documented algorithm is (zcash_client_backend, `src/data_api/chain.rs`):

1. choose a rewind height at least one block before the error height (the module docs' heuristic: $e$.`at_height()` $- 10$);
2. call `WalletWrite::truncate_to_height`, which may truncate *lower* than requested (below);
3. delete cached `CompactBlock`s above the returned truncation height, re-download, and rescan — `Verify` ranges first.

The truncation target is constrained by checkpoint availability: `truncate_to_height` truncates to the greatest height at most the requested one that has a note-commitment-tree checkpoint in *every* tree containing data, and errors with `RequestedRewindInvalid` — carrying the safe rewind height, the maximum over trees of their minimum checkpoint heights — if none exists. The bound `PRUNING_DEPTH` $= 100$ is the maximum number of blocks the wallet is allowed to rewind, consistent with the bound in zcashd; the variant `truncate_to_chain_state` additionally permits precise truncation from provided `ChainState` frontiers even when the target checkpoint was pruned (zcash_client_sqlite, `src/wallet.rs`, `src/lib.rs`; zcash_client_backend, `src/data_api.rs`).

**What truncation does.** Truncating to height $h$ (zcash_client_sqlite, `src/wallet.rs`):

- clamps the scan queue so the wallet's chain-tip view returns to $h$;
- resets or nulls transparent outputs' `max_observed_unspent_height` — a spending transaction may be entirely invalidated by a reorganisation that alters the anchors its shielded spends were built against;
- *un-mines* every transaction with `mined_height` $> h$: `block`, `mined_height`, `tx_index`, and `confirmed_unmined_at_height` are all set NULL, so the transaction reverts to the pending state of §6.4 and will either be re-observed as mined or expire;
- truncates both shard trees to the checkpoint at $h$ (tree and anchor mechanics: the *Orchard Guide*, its commitment-tree section);
- deletes blocks above $h$ and stale `tx_locator_map` entries.

Sent and received notes are deliberately *not* deleted: they may hold data, such as memos, unrecoverable from the chain. Balance computations must instead simply not count received notes whose transactions are no longer mined — which the spendability gate of §6.6 already guarantees, since an un-mined transaction has no scanned block and no qualifying mined height.

**Terminal states.** Every wallet-created transaction thus ends in exactly one of three states: *mined* with enough confirmations that truncation can no longer reach it (`PRUNING_DEPTH` $= 100$ blocks); *expired unmined*, its inputs released (§6.5) and its record retained (Table 4) with the user notified per ZIP-203; or, for the expiry-0 transactions the deployed builder never creates, *pending indefinitely* — polled and rebroadcast until mined or conflicted.

# 7 Payment requests (ZIP-321)

The *Orchard Guide* constructs everything a wallet needs once the sender knows where to pay: note encryption and detection, authentication paths, anchor selection, and the ZIP-244 transaction digest. What none of that settles is the step before any of it — how the payee communicates to the payer an address, an amount, and a memo, in a form a wallet can consume mechanically. ZIP-321 ("Payment Request URIs") standardises that step as a URI scheme; it is Status Active,

Category Standards/Wallet, created 2020-08-28, owned by Kris Nuttycombe and Daira-Emma Hopwood (ZIP-321, header). This section develops the format, its deployed parser — the zip321 crate in librustzcash — and the pipeline that turns a parsed request into a transaction proposal and finally a transaction.

## 7.1 Requests, payments, and the single-transaction rule

**Definition 7.1** (Payment; payment request)**.** A *payment* is a transfer of funds implemented by a single shielded or transparent output of a Zcash transaction. A *payment request* is a request for a wallet to construct a single Zcash transaction containing one or more payments (ZIP-321, Terminology).

A ZIP-321 URI therefore represents a request for the construction of *one* transaction; implementations SHOULD construct a single transaction paying all the specified addresses (ZIP-321, URI Semantics). The number of payments one transaction can carry is limited only by transaction-construction restrictions: the ZIP notes that if every payment were Sapling, the value-balance limit of the protocol specification, §4.12 (Balance and Binding Signature, Sapling), implies construction may fail above 2109 distinct payments, and that the effective limit may be lower when Orchard or future address types are involved. Recomputed from the specification, the bound is a transaction-size cap used inside that section's overflow argument, not a direct value-range consequence: each Sapling output contributes 948 bytes to the transaction — an `OutputDescriptionV4` comprises the 32-byte `cv`, `cmu`, and `ephemeralKey` fields, a 580-byte `encCiphertext`, an 80-byte `outCiphertext`, and a 192-byte proof; an `OutputDescriptionV5` is 756 bytes plus its 192-byte entry in `vOutputProofsSapling` — and a transaction must fit within the specification's 2 MB maximum transaction size (numerically equal to the 2 000 000-byte block-size limit), so at most $\lfloor 2\,000\,000/948 \rfloor = 2109$ outputs fit (protocol specification, §4.12; field sizes from §7.1, "Transaction Encoding and Consensus"). The specification uses the cap to show that the balance sum cannot overflow the type of committed values; ZIP-321 reuses it as the payment-count ceiling.

## 7.2 URI syntax

**Definition 7.2** (ZIP-321 URI grammar)**.** A ZIP-321 URI is a string derived by `zcashurn` in the following ABNF (RFC 5234), reproduced from ZIP-321 (URI Syntax); the productions `ALPHA`, `unreserved`, and `pct-encoded` are those of RFC 3986, and `base64url` is described below.

```
zcashurn         = "zcash:" ( zcashaddress [ "?" zcashparams ] / "?" zcashparams )
zcashaddress     = 1*( ALPHA / DIGIT )
zcashparams      = zcashparam [ "&" zcashparams ]
zcashparam       = [ addrparam / amountparam / assetparam / memoparam
                       / messageparam / labelparam / otherreqparam / otherparam ]
NONZERO          = %x31-39
DIGIT            = %x30-39
paramindex       = "." NONZERO 0*3DIGIT
addrparam        = "address" [ paramindex ] "=" zcashaddress
amountparam      = "amount" [ paramindex ] "=" 1*DIGIT [ "." 1*8DIGIT ]
assetparam       = "req-asset" [ paramindex ] "=" *base64url
labelparam       = "label" [ paramindex ] "=" *qchar
memoparam        = "memo" [ paramindex ] "=" *base64url
messageparam     = "message" [ paramindex ] "=" *qchar
paramname        = ALPHA *( ALPHA / DIGIT / "+" / "-" )
otherreqparam    = "req-" paramname [ paramindex ] [ "=" *qchar ]
otherparam       = paramname [ paramindex ] [ "=" *qchar ]
qchar            = unreserved / pct-encoded / allowed-delims / ":" / "@"
allowed-delims   = "!" / "$" / "'" / "(" / ")" / "*" / "+" / "," / ";"
```

Three global properties of the grammar deserve emphasis before the per-parameter semantics.

**No authority component.** A ZIP-321 URI is not a hierarchical URI: the grammar cannot derive `//` after the scheme, so there is no authority component (ZIP-321, URI Syntax note). For a single payment the recipient address SHOULD be placed directly in the hier-part, and a URI `zcash:<address>?...` MUST be treated as equivalent to `zcash:?address=<address>&...` (ZIP-321, URI Semantics); the query-parameter form with `addrparam` and distinct indices serves multiple recipients.

**Payment grouping by `paramindex`.** Addresses, amounts, labels, and messages sharing the same `paramindex` — including the empty index — belong to the same payment. An index MUST NOT have leading zeros, parameter ordering is insignificant, and index values need not be sequential (ZIP-321, URI Semantics). The grammar bounds indices to `"." NONZERO 0*3DIGIT`, that is $1 \leq i \leq 9999$; the empty index denotes a distinguished payment which the deployed crate stores at map key 0. Rendering and parsing are exact inverses on this range: index 0 renders as the empty suffix, and index $i \geq 1$ renders as `.` $\|$ dec($i$); on parse, an absent index maps to key 0, and the leading-zero strings `.0` and `.01` match no production and invalidate the URI (`zip321`, `src/lib.rs`, `param_index` and `indexed_name`; ZIP-321, Invalid Examples).

**Percent encoding is confined to `qchar`.** Only the values of `label`, `message`, `otherreqparam`, and `otherparam` — the `qchar` productions — may contain `pct-encoded` sequences. Addresses, amounts, and parameter *names* must appear literally: the ZIP's invalid examples include `zcash:taddr?amount=1%30` (encoded amount digit), `zcash:taddr?%61mount=1` (encoded parameter name), and `zcash:%74addr...` (encoded address) (ZIP-321, URI Syntax note and Invalid Examples).

**The base64url alphabet.** Productions `memoparam` and `assetparam` carry binary content encoded as base64url, RFC 4648 §5, with padding omitted: values 62 and 63 encode as `-` and `_`, and implementations MUST NOT accept the standard-base64-only characters `+`, `/`, `=` (ZIP-321, URI Syntax). The deployed crate decodes with the `BASE64_URL_SAFE_NO_PAD` engine (`zip321`, `src/lib.rs`).

### 7.3 Validity and address semantics

The ZIP imposes structural validity rules beyond the grammar (ZIP-321, URI Syntax and URI Semantics):

- a purported ZIP-321 URI that cannot be parsed according to the grammar MUST NOT be accepted;
- if any non-address parameter carries a given `paramindex`, the URI MUST contain an `address` parameter with that index; and
- there MUST NOT be more than one occurrence of a given parameter name at a given index.

Implementations SHOULD check that each `zcashaddress` is a valid encoding per the protocol specification, §5.6, other than a Sprout address: a transparent address (Base58Check, §5.6.1.1), a Sapling payment address (Bech32 per ZIP-173, §5.6.3.1), or a Unified Address (Bech32m per BIP 350, §5.6.4.1 and ZIP-316). New address formats SHOULD be supported whether or not ZIP-321 is updated for them; if network context is available, each address SHOULD be checked for the correct network; and every requirement of ZIP-316 applies to payments to Unified Addresses (ZIP-321, URI Semantics).

Sprout addresses MUST NOT be supported in payment requests. The rationale is economic rather than syntactic: since Canopy, ZIP-211 restricts adding value to the Sprout pool, so senders cannot generally satisfy a request to pay a Sprout address (ZIP-321, URI Semantics). Section 7.9 flags where the deployed stack enforces this.

**Forward compatibility.** A parameter name prefixed `req-` that the parser does not recognise renders the *entire* URI invalid (MUST); unrecognised parameters without the prefix SHOULD be ignored. The prefix is potentially part of a future parameter's name, not a modifier applicable to arbitrary parameters, and the original parameters `address`, `amount`, `label`, `memo`, `message` never carry it, because every conformant parser is required to understand them (ZIP-321, Forward compatibility).

**Backward compatibility.** Implementations of the informal pre-ZIP-321 `zcash:` URI scheme generally lacked the `req-` rule, the `paramindex` multi-payment facility, the `address=` query form for single payments, and the base64url memo treatment (ZIP-321, Backward compatibility).

### 7.4 Amounts

The `amount` parameter is a decimal number of ZEC. If a fraction is present, the separator MUST be a period and both the whole and fractional parts MUST be nonempty; grouping separators are forbidden; leading zeros in the whole part and trailing zeros in the fraction are ignored; the fraction carries at most 8 digits; and the amount MUST NOT exceed 21000000 ZEC (ZIP-321, ZEC Transfer Amount). Thus `50`, `050`, and `50.00` all denote 50 ZEC; `0.5` and `00.500` denote 0.5 ZEC; and `50,000.00`, `50.`, `.5`, and `0.123456789` are invalid.

**Construction 7.3** (Amount parsing and rendering)**.** Fix COIN $:= 10^8$ zatoshis per ZEC and MAX_MONEY $:= 21\,000\,000 \cdot \text{COIN} = 2\,100\,000\,000\,000\,000$ zatoshis (zcash_protocol, `src/value.rs`).

*Parse.* Given a string matching `1*DIGIT [ "." 1*8DIGIT ]`, let $c$ be the whole part read as an unsigned 64-bit integer, and let $z := \text{int}\big(\text{rpad}_8(f, \texttt{0})\big)$, the fraction $f$ right-padded with `0` to 8 digits ($z := 0$ when the fraction is absent). The value is

$$v \;:=\; c \cdot \text{COIN} + z$$

with checked (non-wrapping) multiplication and addition, and the parse succeeds iff $0 \le v \le$ MAX_MONEY (the `Zatoshis` range check) (zip321, `src/lib.rs`, `parse_amount`; zcash_protocol, `Zatoshis::from_u64`).

*Render.* Let $c := \lfloor v/\text{COIN} \rfloor$ and $z := v \bmod \text{COIN}$. Emit $\text{dec}(c)$; if $z \neq 0$, append `.` followed by $\text{dec}(z)$ left-padded with `0` to 8 digits and with trailing `0` characters trimmed (zip321, `src/lib.rs`, `render::amount_str`).

### 7.5 Memos, labels, and messages

The `memo` parameter carries the content of the shielded memo field — the fixed 512-byte field of the note plaintext constructed in the *Orchard Guide* (note encryption) — as base64url without padding. The decoded content MUST NOT exceed 512 bytes and, if shorter, is filled with trailing zeros to 512 bytes. A memo whose same-index address does not permit memos — a transparent address — makes the *entire* URI invalid (MUST) (ZIP-321, Query Keys).

**Construction 7.4** (Memo encoding and decoding)**.** Write $\mathsf{b64u}(\cdot)$ for RFC 4648 §5 encoding with padding omitted.

*Encode.* For a 512-byte memo $m$, the parameter value is $\mathsf{b64u}\big(\text{strip}_{\texttt{0x00}}(m)\big)$, where $\text{strip}_{\texttt{0x00}}$ removes trailing `0x00` bytes (`MemoBytes::as_slice`) (zip321, `src/lib.rs`, `memo_to_base64`).

*Decode.* Let $b := \mathsf{b64u}^{-1}(\text{value})$, rejecting any input containing `+`, `/`, or `=`; require $|b| \le 512$; then

$$m \;:=\; b \,\|\, \texttt{0x00}^{\,512-|b|}$$

(`MemoBytes::from_bytes` zero-pads) (zip321, `src/lib.rs`, `memo_from_base64`; zcash_protocol, `src/memo.rs`).

An absent `memo` parameter yields no memo; at output construction the wallet then substitutes the "no memo" sentinel `MemoBytes::empty()` = $\texttt{0xF6} \,\|\, \texttt{0x00}^{511}$ (zcash_protocol, `src/memo.rs`; zcash_client_backend, `src/data_api/wallet.rs`).

The `label` parameter names an address (for example, the receiver's name): a client SHOULD display it alongside the address, and it MUST be identifiable as distinct from the address itself. The `message` parameter is descriptive text the client may display to describe the payment (ZIP-321, Query Keys). Both are `qchar` values, so arbitrary UTF-8 reaches them only through percent encoding (§7.2).

## 7.6 Custom assets: `req-asset` (specified, not deployed)

The `req-asset` parameter extends payment requests to ZSA Custom Assets (ZIP-227), encoding an Asset Identifier and an amount in a single value; absent `req-asset`, the payment is for ZEC. A URI containing both `amount` and `req-asset` at the same index MUST be rejected. Only Orchard receivers are valid for Custom Asset payments, so a Custom-Asset request MUST use a Unified Address encoding at least one Orchard receiver (ZIP-321, Custom Assets).

**Definition 7.5** (`req-asset` value encoding)**.** The parameter value is

$$\texttt{b64u}\big(\texttt{0x00} \,\|\, \mathrm{I2LEOSP}_{64}(\mathrm{value}) \,\|\, \mathrm{EncodeAssetId}(\mathrm{AssetId})\big) :$$

a version byte `0x00`, the amount as an unsigned 64-bit little-endian integer of the asset's minimal units, and the 66-byte encoded Asset Identifier of ZIP-227 — a 75-byte payload, hence 100 base64url characters. Future versions may define different structures; implementations MUST dispatch on the version byte (ZIP-321, Custom Assets).

Classification per our conventions: *specified*. The parameter appears in the Active ZIP-321 text, but it depends on ZIP-227, which is Status Draft, and the deployed parser rejects every unrecognised `req-` parameter — including `req-asset` — so a Custom-Asset request is invalid to the deployed stack today (zip321, `src/lib.rs`, `to_indexed_param`).

## 7.7 The deployed parser: the zip321 crate

The reference implementation is the zip321 crate in librustzcash (`components/zip321`, a single file `src/lib.rs`), published at version 0.6.0; release 0.1.0 (2024-08-20) factored it out of zcash_client_backend 0.10.0, which still re-exports it as a deprecated module `zcash_client_backend::zip321` (`components/zip321/Cargo.toml` and `CHANGELOG.md`; zcash_client_backend, `src/lib.rs`). The Guide's baseline is the local checkout — librustzcash `main` at commit `62ee526f`, carrying the pending 0.7.0 changes — as for every other librustzcash citation in this volume; the published 0.6.0 crate differs in one observable respect, its non-optional amount defaulting to zero when the parameter is absent, recorded as divergence (D8) in §7.9.

**Definition 7.6** (Types `Payment` and `TransactionRequest`)**.** Type `Payment` holds a recipient address (`zcash_address::ZcashAddress`), an optional amount (`Option<Zatoshis>`), an optional memo (`MemoBytes`), an optional label and message, and a list `other_params` of retained unrecognised parameters. An absent amount means the *sender* chooses the amount — semantics introduced by the pending 0.7.0 change; the published 0.6.0 crate instead defaults the field to zero, a behaviour ZIP-321 leaves unspecified (zip321, `src/lib.rs`; `CHANGELOG.md`, 0.7.0 entry).

Type `TransactionRequest` wraps a `BTreeMap<usize, Payment>`, map key 0 denoting the empty `paramindex`. Constructor `new` rejects more than 9999 payments (`Zip321Error::TooManyPayments`) and validates every non-empty request by round-tripping it through `to_uri`/`from_uri`; constructor `from_indexed` rejects any index key above 9999 (zip321, `src/lib.rs`).

**Construction 7.7** (Parsing: `TransactionRequest::from_uri`)**.** Parsing proceeds in four stages (zip321, `src/lib.rs`: `from_uri`, `lead_addr`, `zcashparam`, `to_indexed_param`, `to_payment`).

(1) Match the literal `zcash:` (lowercase; nom `tag`), then take the characters up to `?` as the hier-part address. If nonempty, it must decode as a `ZcashAddress` and is recorded as an address parameter at payment index 0.

(2) If input remains, require `?` followed by a list, separated by `&` and consuming the rest of the input, of `name[.index]=value` pairs, each mapped to a typed parameter (address, amount, label, message, memo, or other); any unrecognised `req-` name is a hard error ("Required parameter {name} not recognized").

(3) Group parameters by payment index, rejecting any duplicate — the same parameter kind, or the same other-parameter name, twice at one index (`Zip321Error::DuplicateParameter`).

(4) For each index, build a `Payment`. An address parameter must be present, an explicit amount of 0 must not target a transparent-only address, and a memo requires `can_receive_memo()` on its address; the corresponding errors are `RecipientMissing`, `ZeroValuedTransparentOutput`, and `TransparentMemo`. Labels, messages, and other parameters are collected.

**Construction 7.8** (Rendering: `TransactionRequest::to_uri`)**.** An empty request renders as `zcash:`. A request with exactly one payment stored at key 0 renders in hier-part form, `zcash:<address>` followed by `?` and the payment's parameters if any. Otherwise every payment renders in query-parameter form: `zcash:?` followed by, in ascending key order, an address parameter and then that payment's amount, memo, label, message, and other parameters, the index suffix omitted for key 0 (zip321, `src/lib.rs`, `to_uri`).

**Definition 7.9** (Payment validity predicate)**.** Constructor `Payment::new`$(a, v, m, \ldots)$ returns `None` (invalid) iff

$$\big(\mathsf{transparentOnly}(a) \wedge v = \mathrm{Some}(0)\big) \;\vee\; \big(m \neq \mathrm{None} \wedge \neg\, \mathsf{canReceiveMemo}(a)\big),$$

and `Some` of the payment otherwise (zip321, `src/lib.rs`, `Payment::new`). The two predicates live in zcash_address (`src/lib.rs`): canReceiveMemo is true for Sprout, Sapling, and Unified addresses containing a Sapling or Orchard receiver, false for P2PKH, P2SH, and TEX; transparentOnly is true for P2PKH, P2SH, and TEX, and for Unified addresses that have a transparent receiver but neither a Sapling nor an Orchard receiver.

**Character sets.** On parse, a raw `qchar` value may contain alphanumerics plus `- . _ ~ ! $ ' ( ) * + , ; : @ %`, and a parameter name is `ALPHA` followed by `ALPHA/DIGIT/+/-`; the value is then percent-decoded and validated as UTF-8. On render, values are UTF-8 percent-encoded over the complement of `qchar`: the ASCII controls together with space and `" # % & / < = > ? [ \ ] ^ ` { | }` (the crate's `QCHAR_ENCODE` set) (zip321, `src/lib.rs`). Unknown parameters *without* the `req-` prefix are retained: their percent-decoded values land in `Payment::other_params` and are re-rendered, percent-encoded, by `to_uri`.

**Totals.** Method `TransactionRequest::total()` sums the payment amounts with overflow checking against MAX_MONEY, returning `Ok(None)` when any payment lacks an amount (the total is undefined) and `Err(BalanceError::Overflow)` on an out-of-range sum (zip321, `src/lib.rs`).

**Test vectors.** The crate's test module encodes the ZIP's own valid and invalid examples: `amount=1` parses to $10^8$ zatoshis and `amount=123.456` to 12 345 600 000; the boundary `21000000` is valid and `21000000.00000001` invalid; `i64`-overflow and `u64`-wraparound amounts are invalid; `paramindex` 10000 is invalid; `amount=123.` is invalid; and `req-unknown=x` is invalid (zip321, `src/lib.rs`, test module).

### 7.8 From request to proposal to transaction

A parsed `TransactionRequest` enters the wallet through the function `propose_transfer` in module `zcash_client_backend::data_api::wallet`. It takes the wallet database, network parameters, the account to spend from, an input selector, a change strategy, the request, and a confirmations policy, and returns a `Proposal<FeeRule, NoteRef>` by delegating to the input selector's `propose_transaction`; the proposal is later executed by `create_proposed_transactions`. The convenience wrapper `propose_standard_transfer_to_address` places a single `Payment` into a one-element request and uses `GreedyInputSelector` with `SingleOutputChangeStrategy` (zcash_client_backend, `src/data_api/wallet.rs`).

The selector requires every payment to carry an amount: a missing amount yields `ProposalError::PaymentAmountMissing(idx)`, so a wallet UI must fill in user-chosen amounts before proposing (zcash_client_backend, `src/data_api/wallet/input_selection.rs` and `src/proposal.rs`).

**Construction 7.10** (Payment-to-pool assignment)**.** For each (idx, payment) in the request, `GreedyInputSelector` converts the recipient address to the wallet's network and assigns an output pool (zcash_client_backend, `src/data_api/wallet/input_selection.rs`):

- a transparent address maps to the transparent pool, emitting a `TxOut`;
- a TEX address maps to the transparent pool of the *second* step of a two-step proposal (below), its payment re-wrapped with no memo;
- a Sapling address maps to the Sapling pool;
- a Unified Address maps to Orchard if it has an Orchard receiver and Orchard is supported; else to Sapling if it has a Sapling receiver and Sapling is supported; else to its transparent receiver; else the selection fails (`GreedyInputSelectorError::UnsupportedAddress`).

The selector then chooses spendable shielded notes to at least the required amount, preferring the pool matching the outputs, computes change under the change strategy, and returns a proposal whose steps embed the originating request and the resulting map payment_pools : idx $\mapsto$ PoolType.

**TEX addresses (ZIP-320).** A payment to a TEX address is consumed as a two-step proposal: step $\mathrm{tr}_0$ excludes the TEX outputs and instead creates one ephemeral transparent change output; step $\mathrm{tr}_1$ spends that ephemeral output to the TEX recipients, with the memo forced to `None`. Paying a TEX address from shielded inputs in one step is rejected (`ProposalError::PaysTexFromShielded`) (zcash_client_backend, `src/data_api/wallet/input_selection.rs` and `src/data_api/wallet.rs`).

**Step validation.** Constructor `Step::from_parts` enforces that the keys of payment_pools exactly match the request's payment indices, that each selected pool satisfies `recipient_address().can_receive_as(pool)`, that every payment has an amount, and the balance equation

$$v^{\mathrm{in}}_{\mathrm{transparent}} + v^{\mathrm{in}}_{\mathrm{shielded}} + v^{\mathrm{in}}_{\mathrm{prior\ steps}} \;=\; \mathrm{total}(R) \;+\; \mathrm{total}(\Delta), \tag{2}$$

where $R$ is the embedded request, $\Delta$ the proposed balance — change, including ephemeral outputs, plus fee — and every sum is checked in `Zatoshis`; a violation is a `BalanceError` (zcash_client_backend, `src/proposal.rs`).

**Serialization.** A proposal serialises its transaction request *as its ZIP-321 URI*: the protobuf field `transactionRequest` of message `ProposalStep` (string, field 1, package `cash.z.wallet.sdk.ffi`) is produced by `to_uri` and parsed back with `from_uri`, alongside `PaymentOutputPool{paymentIndex, valuePool}` entries for the pool map (zcash_client_backend, `proto/proposal.proto` and `src/proto.rs`). This round-trip is why the crate accepts and renders the empty request `zcash:` — a shielding step has no payments — a deliberate divergence from the ZIP flagged in §7.9.

**Execution.** At transaction creation, for each (idx, pool) in the step's pool map the corresponding `Payment` is fetched from the embedded request. A shielded output uses the payment's memo, or the sentinel $\texttt{0xF6} \,\|\, \texttt{0x00}^{511}$ when absent (§7.5); a memo on a transparent output raises `Error::MemoForbidden`. For a Unified Address, the receiver actually paid is the one recorded in payment_pools — the Orchard, Sapling, or transparent receiver extracted from the UA (zcash_client_backend, `src/data_api/wallet.rs`). From here the construction is that of the *Orchard Guide*: note encryption seals each output's plaintext, and the spend and binding signatures sign the ZIP-244 sighash.

### 7.9 Divergences between ZIP-321 and the deployed stack

Our conventions require flagging every point where the deployed code and the ZIP disagree; ZIP-321 accumulates several, all in the parser.

(D1) *Empty requests.* The crate accepts `zcash:` (and `zcash:?`) as a valid empty `TransactionRequest` and renders empty requests as `zcash:` (zip321, `src/lib.rs`), but the grammar of Definition 7.2 requires either an address or `? zcashparams` after the scheme, and the ZIP defines a request as having one or more payments. The divergence is deliberate: proposal serialisation round-trips shielding steps — which have no payments — through the URI form (§7.8).

(D2) *Sprout recipients.* ZIP-321 says Sprout addresses MUST NOT be supported (§7.3), yet the crate parses a Sprout recipient without error — `ZcashAddress::try_from_encoded` decodes Sprout (zcash_address, `src/encoding.rs`) and `to_payment` performs no Sprout check; indeed canReceiveMemo is true for Sprout, so even a memo to one is accepted. Rejection occurs only downstream, at proposal time, when address conversion fails with `ConversionError::Unsupported("Sprout")` because `zcash_keys::address::Address` has no Sprout variant (zcash_address, `src/convert.rs`; zcash_client_backend, `src/data_api/wallet/input_selection.rs`). The deployed stack satisfies the payment-level requirement but not URI-level rejection.

(D3) *Case sensitivity.* ABNF literal strings (RFC 5234) are case-insensitive, so a strict reading makes `ZCASH:` — and even `AMOUNT=` — grammar-valid; the deployed parser matches only lowercase (zip321, `src/lib.rs`, `tag("zcash:")` and the literal parameter names). The ZIP text never addresses case, which matters for QR alphanumeric-mode URIs, which are uppercase. We adopt the deployed, lowercase-only behaviour as normative and treat the ABNF case-insensitivity as a defect of the ZIP text: an uppercased URI is unparseable to the deployed stack, so a producer wanting QR alphanumeric mode must not obtain it by uppercasing the URI.

(D4) *Valueless parameters.* The grammar makes `=` optional for `otherparam` and `otherreqparam` (a bare `nonce` is derivable), but the crate requires `name=value`, rejecting such grammar-valid URIs (zip321, `src/lib.rs`, `zcashparam`).

(D5) *Bare percent signs.* The grammar admits `%` in a value only as `pct-encoded` (`% HEXDIG HEXDIG`), but the parser accepts `%` as a raw character and its percent decoder passes invalid

sequences through unchanged, so `message=100%` is accepted — laxer than the grammar (`zip321`, `src/lib.rs`, `qchars` and `percent_decode`). A direct test against the crate confirms it: the values `100%`, `100%zz`, and `100%2` all parse, the malformed sequences surviving percent-decoding verbatim, while `100%25` decodes to `100%`.

(D6) *Empty list elements.* The production `zcashparam` can derive the empty string, making `a=1&&b=2` arguably grammar-valid, while the crate rejects it (its list elements must be nonempty) — a divergence in the opposite direction of (D4) and (D5).

(D7) *Zero-valued transparent outputs.* The crate rejects `amount=0` to a transparent-only address (Definition 7.9); ZIP-321 has no such rule. The crate's changelog asserts such outputs are disallowed by consensus (`zip321`, `CHANGELOG.md`, 0.7.0 entry). The assertion is inaccurate: consensus admits zero-valued transparent outputs — `CheckTransaction` rejects only negative values and values above MAX_MONEY (`zcashd`, `src/main.cpp`), and the protocol specification constrains transparent outputs no further, requiring only a nonnegative remaining transparent value pool. What rejects them in practice is *standardness*: a zero-valued spendable output falls below the dust threshold (zero only for unspendable scripts, which no address payment produces), so `zcashd`'s `IsStandardTx` (`src/policy/policy.cpp`) and Zebra's equivalent mempool check (`zebrad`, `src/components/mempool/storage.rs`) refuse to admit or relay it. The crate's rule is sound — such an output is unrelayable — but on standardness, not consensus, grounds.

(D8) *Version skew.* The local librustzcash main (0.7.0-PENDING, commit `62ee526f`) makes the payment amount optional; the published 0.6.0 crate defaults an absent amount to zero (§7.7).